



МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(национальный исследовательский университет)»

Институт № 3 «Системы управления, информатика и электроэнергетика»

Кафедра 305 Группа МЗО-606С-20

Специальность 24.05.05 «Интегрированные системы летательных аппаратов»

Специализация Приборы и измерительно-вычислительные комплексы ЛА

Квалификация инженер

ДИПЛОМНЫЙ ПРОЕКТ

На тему: Разработка стенда полунатурного моделирования системы управления движением
квадрокоптера

Автор проекта Кравцов Максим Валентинович

(фамилия, имя, отчество полностью)

Руководитель Нгуен Ныы Ман

(фамилия, имя, отчество полностью)

Консультанты

а) Нгуен Ныы Ман

(фамилия, имя, отчество полностью)

б) Хайрнатов Камилль Зайнутдинович

(фамилия, имя, отчество полностью)

в) Галкина Елена Евгеньевна

(фамилия, имя, отчество полностью)

г) Ермолаева Елена Николаевна

(фамилия, имя, отчество полностью)

Рецензент Смирнов Сергей Витальевич

(фамилия, имя, отчество полностью)

К защите допустить

Заведующий кафедрой 305 Алешин Борис Сергеевич

(№ каф)

(фамилия, имя, отчество полностью)

«30» 01 2026 г.

Москва 2026 г.



МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(национальный исследовательский университет)»

Институт № 3 «Системы управления, информатика и электроэнергетика»

Кафедра 305 Группа МЗО-606С-20

Специальность 24.05.05 «Интегрированные системы летательных аппаратов»

Специализация Приборы и измерительно-вычислительные комплексы ЛА

Квалификация инженер

УТВЕРЖДАЮ

Заведующий кафедрой

305

(№ каф.)

« 13 »

(подпись)

Б.С. Алешин

(инициалы, фамилия)

2025 г.

ЗАДАНИЕ
на дипломный проект

Обучающийся Кравцов Максим Валентинович

(фамилия, имя, отчество полностью)

Руководитель Нгуен Ныы Ман

(фамилия, имя, отчество полностью)

к.т.н., доцент, доцент кафедры 305 МАИ

(ученая степень, ученое звание, должность и место работы)

1. Наименование темы Разработка стенда полунатурного моделирования системы управления движением квадрокоптера

2. Срок сдачи обучающимся законченной работы 04.01.2026 г.

3. Задание и исходные данные к работе

Разработать систему управления угловым движением квадрокоптера на базе инерциальной системы измерения MPU-6050. Разработать программу визуального моделирования движения квадрокоптера в среде Unreal Engine. Разработать стенд полунатурного моделирования работы системы управления движением.

4. Перечень подлежащих разработке разделов и этапы выполнения работы

№ п/п	Наименование раздела или этапа	Трудоёмкость в % от полной трудоёмкости работы	Срок выполнения	Примечание
1	Введение	5	17.10.25	
2	Специальная часть	70	15.12.25	

2.1	Обзор современных подходов к тестированию алгоритмов оптической навигации для квадрокоптера	10	31.10.25	
2.2	Описание стенда полунатурного моделирования	30	14.11.25	
2.3	Разработка программы визуального моделирования движения квадрокоптера в среде Unreal Engine	25	5.12.25	
2.4	Анализ результатов	5	15.12.25	
3	Технологическая часть	10	18.12.25	
4	Охрана труда и окружающей среды	5	20.12.25	
5	Экономическая часть	5	23.12.25	
6	Заключение	5	25.12.25	

5. Исходные материалы и пособия

1. ГОСТ 7.32-2017 Отчет о научно-исследовательской работе
2. ГОСТ 19.701-90 Схемы алгоритмов, программ, данных и систем
3. ГОСТ Р 7.05-2008 Библиографическая ссылка

6. Консультанты по работе с указанием относящихся к ним разделов работы

Раздел	Консультант	Задание выдал (подпись, дата)
Специальный	Нгуен Ныы Ман	
Экономический	Ермолаева Елена Николаевна	
Охрана труда и окружающей среды	Галкина Елена Евгеньевна	
Технологический	Хайрнасов Камиль Зайнутдинович	

7. Дата выдачи задания 29.09.2025г.

Руководитель

(подпись)

Задание принял к исполнению

(подпись)



МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(национальный исследовательский университет)»

О Т З Ы В РУКОВОДИТЕЛЯ

Обучающийся Кравцов Максим Валентинович
(фамилия, имя, отчество полностью)
Институт № 3 «Системы управления, информатика и электроэнергетика»
Кафедра 305 Группа МЗО-606С-20
Специальность 24.05.05 «Интегрированные системы летательных аппаратов»
Специализация Приборы и измерительно-вычислительные комплексы ЛА
Квалификация инженер
Наименование темы Разработка стенда полунатурного моделирования системы управления движением квадрокоптера
Руководитель Нгуен Ныы Ман, к.т.н., доцент каф. 305
(фамилия, имя, отчество полностью, ученая степень, ученое звание, должность и место работы)

Одним из главных этапов разработки систем управления полетом является верификация работоспособности их алгоритмов управления на основе обработки результатов имитационного моделирования и данных летных испытаний. При этом с одной стороны, чисто математическое моделирование не позволяет учесть все особенности динамики управляемого процесса. С другой стороны, проведение испытаний связано с повышенным риском повреждения оборудования и сложностью воспроизведения одинаковых условий эксперимента, что существенно ограничивает возможности исследования.

Эффективным подходом к решению данной проблемы является применение стендов полунатурного моделирования, объединяющих реальные аппаратные компоненты и виртуальную среду. Использование таких стендов позволяет исследовать работу системы управления движением в воспроизводимых и безопасных условиях без выполнения реальных полетов. В связи с этим дипломный проект на тему «Разработка стенда полунатурного моделирования системы управления движением квадрокоптера» является актуальным.

В соответствии с заданием дипломантом разработан стенд полунатурного моделирования, состоящий из квадрокоптера, закрепленного на кардановом подвесе с тремя степенями свободы, и персонального компьютера, на котором работает среда моделирования и визуализации AirSim. С использованием данного стенда проведено исследование работы системы стабилизации углового движения квадрокоптера. Кроме того, реализованы алгоритмы передачи информации об угловом движении квадрокоптера в виртуальную среду Airsim, на основе которой проведено моделирование поступательного движения квадрокоптера с целью тестирования алгоритмов обнаружения препятствий. Полученные результаты позволили оценить корректность работы системы управления и сформировать задел для дальнейшего развития системы автономного управления полетом путем интеграции системы обнаружения препятствий в контур управления.

В процессе выполнения данного проекта Кравцов М.В. продемонстрировал высокий уровень владения теоретическими знаниями, а также навыки разработки программного

обеспечения для интеграции виртуальной и реальной среды. Все разработанные алгоритмы и программные модули реализованы самостоятельно дипломантом, что подтверждает высокий уровень профессиональной подготовки и умение применять комплексный подход к решению инженерных задач.

Проект выполнен в полном объеме в соответствии с заданием. Дипломантом показаны глубокие теоретические знания и практические навыки, подтверждающие успешное решение поставленных задач по разработке и исследованию системы полунатурного моделирования управления квадрокоптером.

Проект проверен на объем заимствования. % заимствования – 3.58

Заключение: Дипломный проект выполнен на высоком научно-техническом уровне. Кравцов М.В. заслуживает присвоения квалификации «Инженер» по специальности 24.05.05 «Интегрированные системы летательных аппаратов».

« 24 » 01 2026 г.

Руководитель


(подпись)



ЗАКЛЮЧЕНИЕ РЕЦЕНЗЕНТА О ДИПЛОМНОЙ РАБОТЕ

Обучающийся Кравцов Максим Валентинович
(фамилия, имя, отчество полностью)
Институт № 3 «Системы управления, информатика и электроэнергетика»
Кафедра 305 Группа МЗО-606С-20
Специальность 24.05.05 «Интегрированные системы летательных аппаратов»
Специализация Приборы и измерительно-вычислительные комплексы ЛА
Квалификация инженер
Наименование темы Разработка стенда полунатурного моделирования системы управления движением квадрокоптера
Рецензент Начальник отдела 1410, к.т.н., Смирнов Сергей Викторович, АО «ЦНИИАГ»
(фамилия, имя, отчество полностью, ученая степень, ученое звание, должность и место работы)

Отмеченные достоинства: Дипломный проект Кравцова М.В. посвящен актуальной теме, связанной с разработкой стенда полунатурного моделирования, предназначенного для безопасного тестирования систем управления беспилотными летательными аппаратами типа квадрокоптера в условиях, максимально приближенных к реальным, при ограниченных возможностях проведения натурных испытаний.

В соответствии с поставленной задачей в проекте разработан испытательный стенд в виде карданного подвеса, обеспечивающий отработку алгоритмов управления угловым движением жестко закрепленного на нем квадрокоптера. Так же разработано программное обеспечение, обеспечивающее обмен данными между квадрокоптером и персональным компьютером, на котором реализуется математическое моделирование продольного движения, соответствующего реальному угловому движению квадрокоптера. Кроме того, создано программное обеспечение для визуализации полного движения квадрокоптера и реализации алгоритмов обнаружения препятствий в программной среде Airsim.

По результатам проведенных экспериментальных исследований сделаны обоснованные выводы о работоспособности разработанного стенда и возможности его практического применения.

Отмеченные недостатки: В рамках выполненного проекта не рассмотрен вопрос реализации контура управления поступательным движением квадрокоптера с использованием результатов детекции препятствий.

Заключение: Дипломный проект Кравцова М.В. выполнен в полном объеме в соответствии с установленным заданием. Считаю, что дипломный проект заслуживает высокой оценки, а дипломант Кравцов М.В. – присуждения квалификации «Инженер» по специальности 24.05.05 «Интегрированные системы летательных аппаратов».

« 05 » 02 2026г.

Рецензент


(подпись)

Справка о проверке на наличие заимствований

Имя файла: ДП_Кравцов_MB.docx

Автор: Кравцов М.В.

Заглавие: Дипломный проект

Год публикации: 2026

Комментарий: Не указан



Коллекции: Интернет 2.0, Научные статьи 2.0, Википедия, Англоязычная Википедия, Коллекция Энциклопедий, Библиотека Либрусек, Университетская библиотека, Коллекция КФУ, ВКР Российского университета кооперации, Коллекция АПУ ФСИН, Коллекция ПГУТИ, Репозиторий открытого доступа СПб гос. ун-та, Научная электронная библиотека "КиберЛенинка", ЦНМБ Сеченова, Авторефераты ВАК, Диссертации ВАК, Диссертации РГБ, Авторефераты РГБ, Готовые рефераты, ФИПС. Изобретения, ФИПС. Полезные модели, ФИПС. Промышленные образцы, Коллекция Руконт, Библиотека им. Ушинского, Готовые рефераты (часть 2), Открытые научные источники, НЭБ, БиблиоРоссика, Правовые документы I, Правовые документы II, Правовые документы III, Собрание законодательства Российской Федерации

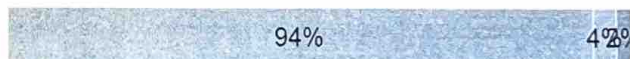
Результат проверки

Оценка оригинальности документа: 94%

Оригинальные фрагменты: 93,93%

Обнаруженные заимствования: 3,58%

Цитирование: 2,49%



Работу проверил: Антонов Д.А.

Дата: 27.01.2026

Подпись:

Подпись Антонова Д.А. заверяю
Зам. директора Института №3



Косынова О.С.

СОДЕРЖАНИЕ

ПЕРЕЧЕНЬ СОКРАЩЕНИЙ И ОБОЗНАЧЕНИЙ	5
ВВЕДЕНИЕ.....	7
1 Специальная часть.....	11
1.1 Обзор современных подходов к тестированию алгоритмов оптической навигации для квадрокоптера.....	11
1.1.1 Современные методы детекции препятствий в квадрокоптере	11
1.1.2 Современные методы управления и навигации квадрокоптера	16
1.1.3 Обоснование выбора YOLO как метода детекции препятствий.....	19
1.1.4 Обоснование выбора среды AirSim для моделирования	23
1.2 Описание стенда полунатурного моделирования	26
1.2.1 Назначение и общая структура стенда полунатурного моделирования	26
1.2.2 Схема информационного обмена и синхронизации данных.....	29
1.2.3 Система управления угловым движением квадрокоптера на базе MPU-6050	32
1.2.4 Полетный контроллер на базе микроконтроллера ESP32	43
1.2.5 Двухсторонняя передача данных ориентации от ESP32 на ПК по протоколу UDP ...	48
1.2.6 Обработка данных ориентации на ПК и синхронизация движения стенда	54
1.3 Разработка программы визуального моделирования движения квадрокоптера в среде Unreal Engine.....	61
1.3.1 Подготовка среды моделирования.....	61
1.3.2 Настройка виртуального окружения и карты в Unreal Engine	69
1.3.3 Управление бортовыми камерами и формирование визуальных данных	76
1.3.4 Результаты визуального моделирования и тестирование системы	90
1.4 Анализ результатов	95
1.4.1 Режимы функционирования и сценарии использования полунатурного стенда	95
1.4.2 Результаты моделирования	101
1.4.3 Вывод.....	106
2 Технологическая часть	110
2.1 Введение.....	110
2.2 Разработка и конструкция полунатурного стенда	110
2.2.1 Технологический процесс сборки стенда	110
2.2.2 Конструктивно-технологическое описание стенда.....	112
2.3 Установка и описание квадрокоптера-рамы.....	115
2.4 Подбор микроэлектромеханических датчиков и микроконтроллера.....	118
2.4.1 Микроконтроллер.....	118
2.4.2 Микроконтроллер ESP32	119

2.4.3 Микроэлектромеханические системы	121
2.4.4 Инерциальный измерительный модуль MPU-6050.....	122
2.4.5 Схема подключения	124
2.5 Вывод	125
3 Охрана труда и окружающей среды.....	126
3.1 Введение.....	126
3.2 Анализ условий труда.....	127
3.3 Микроклимат	130
3.4 Шум	132
3.5 Освещенность	133
3.6 Тяжесть трудового процесса.....	135
3.7 Напряженность трудового процесса	136
3.8 Итоговый класс условий труда	138
3.9 Проектирование системы освещения	138
3.10 Вывод	140
4. Экономическая часть	141
4.1 Введение.....	141
4.2 Оценка целесообразности выполнения разработки на основе определения ее технической прогрессивности.....	141
4.2.1 Оценка научно-технической прогрессивности.....	141
4.2.2 Оценка новизны НТПр и уровня научно-технического прогресса (НТП).....	144
4.3 Планирование разработки	146
4.3.1 Определение трудоемкости разработки	147
4.3.2 Календарное планирование	149
4.4 Определение затрат, себестоимости и цены.....	151
4.5 Определение и оценка показателей экономической эффективности.....	155
4.6 Вывод по оценке экономической эффективности разработки программного продукта	157
ЗАКЛЮЧЕНИЕ	159
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ.....	162
ПРИЛОЖЕНИЕ А	164

ПЕРЕЧЕНЬ СОКРАЩЕНИЙ И ОБОЗНАЧЕНИЙ

БПЛА – Беспилотный летательный аппарат;

LiDAR (Light Detection and Ranging) – система дистанционного зондирования на основе лазерного излучения;

UE / Unreal Engine – игровой движок;

AirSim – платформа моделирования беспилотных летательных аппаратов;

ESP32 – микроконтроллер;

ИМИ – инерциальный модуль измерения;

SLAM (Simultaneous Localization and Mapping) – одновременная локализация и построение карты;

NN (Neural Network) – нейронная сеть;

CNN (Convolutional Neural Network) – сверточная нейронная сеть;

YOLO (You Only Look Once) – одноэтапный алгоритм детекции объектов;

NMS (Non-Maximum Suppression) – подавление немаксимальных предсказаний;

IoU (Intersection over Union) – метрика пересечения и объединения ограничивающих рамок;

PID-регулятор – пропорционально-интегрально-дифференциальный регулятор;

MPC (Model Predictive Control) – предиктивное управление на основе модели;

RL (Reinforcement Learning) – обучение с подкреплением;

SSD (Single Shot Detector) – одноэтапный алгоритм детекции объектов;

GPU (Graphics Processing Unit) – графический процессор;

TPU (Tensor Processing Unit) – специализированный процессор для нейронных вычислений;

API (Application Programming Interface) – программный интерфейс приложения;

RGB-камера – камера, формирующая изображение в цветовом пространстве

Depth-камера – камера глубины;

Segmentation-камера – камера, формирующая изображение с семантической сегментацией;

GPS (Global Positioning System) – глобальная система позиционирования;

DMP (Digital Motion Processor) – цифровой процессор обработки движений;

PWM-сигналы (Pulse Width Modulation) – сигналы широтно-импульсной модуляции;

I²C (Inter-Integrated Circuit) – последовательная шина обмена данными;

ESC (Electronic Speed Controller) – электронный регулятор скорости;

MEMS-датчики (Micro-Electro-Mechanical Systems) – микроэлектромеханические датчики;

UDP (User Datagram Protocol) – протокол передачи данных без установления соединения;

TCP (Transmission Control Protocol) – протокол передачи данных с установлением соединения;

Bluetooth – беспроводная технология передачи данных малого радиуса действия;

Wi-Fi – беспроводная технология передачи данных по локальной сети;

ПК – персональный компьютер;

ИС – измерительная система;

САУ – система автоматического управления;

ИБК – измерительно-вычислительный комплекс;

NED (North-East-Down) – навигационная система координат;

FPV (First Person View) – режим управления от первого лица;

FPS (Frames Per Second) – количество кадров в секунду;

CPU (Central Processing Unit) – центральный процессор;

Датасет – набор данных для обучения и тестирования моделей;

CVAT (Computer Vision Annotation Tool) – инструмент разметки данных компьютерного зрения;

mAP (mean Average Precision) – средняя точность детекции объектов;

ССК – связанная система координат.

ВВЕДЕНИЕ

В последние годы беспилотные летательные аппараты прочно вошли в различные сферы деятельности. Они используются для съёмки с воздуха и мониторинга наземных объектов, а также в спасательных операциях и автоматизированной доставки грузов. Технический прогресс и снижение стоимости производства сделали БПЛА доступными не только для крупных организаций, но и для частных пользователей, что привело к резкому росту их использования. Согласно данным на 2022 год, только в Китае было зарегистрировано около 958 тысяч БПЛА, выполняющих в среднем 57 тысяч часов полетов ежедневно [1].

Среди всего многообразия беспилотных летательных аппаратов особое распространение получили квадрокоптеры, благодаря их манёвренности, компактности конструкции и относительной простоте управления. Массовое использование квадрокоптеров привело к увеличению нагрузки на воздушное пространство, особенно в городских условиях, характеризующихся высокой плотностью зданий, линий электропередач и других потенциальных препятствий. При этом многие современные квадрокоптеры обладают ограниченными возможностями автономной навигации и зачастую требуют постоянного вмешательства оператора.

Этот стремительный рост привел к увеличению нагрузки на воздушное пространство, особенно в городах, где присутствует высокая плотность зданий, линий электропередач и других потенциальных препятствий. При этом многие современные квадрокоптеры обладают ограниченными возможностями автономной навигации и зачастую требуют вмешательства оператора. Это создает дополнительные сложности при эксплуатации, особенно в условиях низкой видимости, ограниченного пространства или высокой концентрации движущихся объектов, таких как птицы или другие беспилотные аппараты. В таких ситуациях особенно важны надежные системы детекции и обхода препятствий, обеспечивающие безопасный полет.

Исследования Европейского агентства авиационной безопасности подтверждают, что при полетах на высотах до 120 метров вероятность столкновений значительно возрастает из-за ограниченного воздушного пространства и высокой концентрации объектов [2].

На сегодняшний день разработано множество методов детекции препятствий. Среди них можно выделить:

- Лидар – использует лазерные импульсы для создания трехмерной карты окружающего пространства.
- Стереозрение – анализ изображений с двух камер для вычисления глубины сцены.
- Ультразвуковые датчики – работают по принципу эхолокации, измеряя время прохождения звуковых волн.
- Компьютерное зрение на основе нейросетей – анализ изображений с использованием алгоритмов глубокого обучения, что позволяет эффективно обнаруживать и классифицировать объекты.

В таблице В.1 представлено сравнение основных подходов.

Таблица В.1 – Сравнение методов детекции препятствий для квадрокоптеров

Метод	Принцип работы	Преимущества	Недостатки
Лидар	Лазерные импульсы для создания 3D-карты пространства	Высокая точность, надёжность	Высокая стоимость, чувствительность к погоде
Стереозрение	Анализ изображений с двух камер	Эффективно для динамических объектов	Ограничения при слабом освещении
Ультразвуковые датчики	Эхолокация с использованием звуковых волн	Низкая стоимость, компактность	Ограниченная дальность и точность
Компьютерное зрение	Анализ изображений с помощью нейросетей	Высокая скорость, универсальность	Требует больших вычислительных ресурсов

Системы обнаружения препятствий играют ключевую роль в обеспечении безопасности автономного полёта квадрокоптера. Однако одной лишь детекции недостаточно, необходима также система управления, способная адаптировать траекторию движения в зависимости от поступающей информации и предотвращать возможные столкновения. Для проверки и отладки подобных алгоритмов широко применяются системы моделирования и имитации, которые позволяют воспроизводить поведение квадрокоптера в контролируемых условиях, оценивать реакцию на различные сценарии и снижать риски при реальных испытаниях.

Одним из перспективных направлений является создание полунатурных стендов моделирования, сочетающих физические и виртуальные компоненты. Такие системы позволяют одновременно учитывать реальные характеристики аппаратной части, например, динамику подвеса и поведение датчиков, и гибко управлять параметрами моделируемой среды. Это делает возможным проведение полноценных экспериментов без необходимости полевых испытаний, что особенно важно на этапах разработки и отладки алгоритмов автономного управления.

Таким образом, разработка интегрированной системы, объединяющей физический стенд и виртуальную среду моделирования, является актуальной задачей, направленной на повышение безопасности, надёжности и автономности полёта квадрокоптера.

Целью данного дипломного проекта является разработка интегрированной системы моделирования, которая позволит тестировать управление алгоритмы управления и обнаружения препятствий с возможностью взаимодействия с виртуальной средой AirSim и физическим стендом.

Для достижения поставленной цели решаются следующие задачи:

- Провести анализ современных подходов к детекции препятствий, а также методов моделирования и управления беспилотными летательными

аппаратами – для обоснования выбора оптимальных технологий, применяемых в проекте.

- Разработать систему детекции препятствий на основе современных методов обработки информации и компьютерного зрения – для обеспечения автономного восприятия окружающей среды.

- Создание полунатурного стенда моделирования – для воспроизведения реальных динамических характеристик квадрокоптера и получения данных с датчиков в реальном времени.

- Интегрировать физическую и виртуальную части системы – для реализации гибридного моделирования, обеспечивающего взаимодействие между аппаратной и программной составляющими.

- Провести тестирование и оценку эффективности системы – для определения точности, стабильности и надёжности работы в различных условиях моделирования.

1 Специальная часть

1.1 Обзор современных подходов к тестированию алгоритмов оптической навигации для квадрокоптера

1.1.1 Современные методы детекции препятствий в квадрокоптере

Одним из ключевых компонентов системы восприятия и навигации квадрокоптера является модуль детекции препятствий. Данный модуль предназначен для формирования информации об окружающей среде и используется в составе подсистем планирования траектории и принятия решений при автономном управлении летательным аппаратом.

Корректность выполнения полётной миссии определяется возможностью своевременного обнаружения статических и динамических объектов. В связи с этим модуль детекции препятствий является обязательным элементом системы автономного управления квадрокоптером. Современные системы детекции, как правило, основываются на комбинированном использовании аппаратных сенсоров и алгоритмических методов обработки данных, включая методы машинного и глубокого обучения, что позволяет повысить надёжность и устойчивость восприятия в различных условиях эксплуатации [3].

Детекция препятствий для квадрокоптера реализуется двумя взаимодополняющими направлениями:

- аппаратные сенсоры,
- алгоритмические методы обработки сенсорных данных.

Аппаратные сенсоры представляют собой сенсорное оборудование, устанавливаемое на летательный аппарат и обеспечивающее получение первичной информации об окружающей среде, такой как расстояние до препятствий, глубина сцены и движение объектов [5].

В таблице 1.1 представлена таблица-сводка основных типов сенсоров, применяемых в системах детекции препятствий:

Таблица 1.1 – Сравнительная таблица аппаратных сенсоров

Сенсор	Основная функция	Преимущества	Ограничения
LiDAR	Измерение расстояния и формирования облака точек	Высокая точность по глубине, работает в темноте	Стоимость, вес, чувствителен к погодным условиям
Стерео-камера	Получение глубины через корреляцию двух камер	Относительно недорогой, пассивный	Снижение точности в условиях слабой текстуры/освещенности
Моно-камера и нейросеть	Оценка глубины и/или идентификация объектов	Лёгкая установка, быстрое обновление модели	Аппроксимация глубины, требует обучения и данных
Радар	Радиолокационное сканирование препятствий	Работает при плохой погоде, дальнейшее обнаружение	Низкое пространственное разрешение для мелких квадрокоптера
Ультразвук	Ближняя дистанция (несколько метров)	Простота, малый вес	Ограниченный радиус действия, множество ложных срабатываний
ИМИ / акустика	Косвенная информация	Дополняет визуал/лидар	Не обеспечивает явной карты препятствий

После анализа основных типов сенсоров, применяемых в системах детекции препятствий, можно отметить, что для малогабаритных квадрокоптеров наибольший практический интерес представляют визуальные сенсоры, в частности камеры. Это обусловлено их малой массой, низким энергопотреблением, универсальностью и возможностью одновременного получения информации как о геометрии сцены, так и о типах объектов. Обзорные работы подчёркивают, что для квадрокоптеров приоритетными являются лёгкие и энергоэффективные сенсоры с малой задержкой реагирования, что делает камеры одним из наиболее рациональных вариантов для систем автономной навигации [3], [4].

При использовании камер в качестве основного сенсора ключевой задачей становится обработка визуальной информации и выделение препятствий в изображении. В этом контексте центральное место занимает задача детекции объектов в изображении.

Детекция объектов в изображении – это задача, заключающаяся в нахождении всех объектов интереса на изображении, определении их классов и координат ограничивающих рамок. В отличие от задач классификации, где необходимо установить только принадлежность изображения к одному из классов, детекция требует выделения нескольких объектов разного типа и формы в пределах одного кадра [5].

Формально задача заключается в предсказании множества ограничивающих рамок и их классов:

$$\{(x_i, y_i, w_i, h_i, c_i)\}_{i=1}^N, \quad (1.1)$$

где x_i, y_i – координаты центра ограничивающей рамки;

w_i, h_i – ширина и высота рамки;

c_i – метка класса;

N – количество объектов на изображении.

При этом отдельные сенсоры, как правило, не обеспечивают полной картины окружающей среды и требуют дальнейшей алгоритмической обработки полученных данных.

Алгоритмические методы обработки данных – после получения сенсорных данных следующей задачей становится их обработка – выделение препятствий, классификация, оценка расстояния, формирование безопасной траектории. Существуют два основных направления:

- классические (геометрические) методы,
- методы, основанные на глубоком обучении.

Классические (геометрические) методы включают стерео-корреспонденция, оптический поток, SLAM с визуальными или данными с лидаров. Они нацелены на построение глубинной карты и локализацию по сцене. Такие методы хорошо работают в текстурированных средах, но чувствительны к условиям освещения и структуре сцены [6].

До появления глубоких нейросетей, детекция объектов в изображениях основывалась на ручном извлечении признаков и применении скользящего окна. Основная схема включала:

- Генерацию кандидатных окон – сканирование изображения с различными размерами.
- Извлечение признаков – преобразование каждого окна в вектор признаков.
- Классификацию – определение наличия объекта с помощью классификаторов.
- Фильтрацию результатов – устранение пересечений методом NMS.

Однако такие методы имели серьёзные ограничения: низкая скорость из-за обработки тысяч окон, ограниченная обобщающая способность ручных признаков и плохая масштабируемость.

Методы глубокого обучения произвели революцию в этой области. С появлением сверточных нейронных сетей (CNN) подходы были заменены обучаемыми системами, которые сами извлекают признаки и строят карту классов и рамок. Современные подходы используют CNN как основу для автоматического извлечения признаков и классификации. Существуют два основных архитектурных подхода:

- Двухэтапные детекторы – генератор регионов предлагает области интереса. Каждая область передаётся в классификатор, который определяет класс и уточняет координаты рамки.
- Одноэтапные детекторы (например, YOLO) избегают явной генерации регионов, что делает их существенно быстрее. Вместо этого изображение делится на сетку, и каждая ячейка предсказывает несколько ограничивающих рамок и классов объектов. Это особенно важно для задач реального времени, таких как навигация квадрокоптера.

Алгоритм YOLO является ключевым примером одноэтапного детектора. Его ключевая идея заключается в разделении изображения на сетку $S * S$, где каждая ячейка отвечает за предсказание ограничивающих рамок, если центр объекта попадает в неё [7].

Выход модели представляет собой тензор:

$$S * S * (B * (5 + C)), \quad (1.2)$$

где S – количество ячеек сетки;

B – количество предсказаний рамок на ячейку;

5 – координаты (x_i, y_i, w_i, h_i) и c_i ;

C – количество классов.

Все предсказания объединяются и фильтруются с помощью алгоритма подавления немаксимумов (NMS). Этот алгоритм сравнивает рамки по метрике пересечения и объединения (IoU – Intersection over Union):

$$IoU(A, B) = \frac{|A \cap B|}{|A \cup B|} \quad (1.3)$$

Если значение IoU между двумя рамками превышает установленный порог, оставляется только рамка с наибольшей уверенностью. Это позволяет устранить избыточные и дублирующие предсказания.

YOLO представляет собой единый сверточный блок, обучаемый end-to-end, что делает его чрезвычайно эффективным для задач реального времени, таких как навигация квадрокоптеров [7].

Современные методы детекции препятствий для квадрокоптера представляют собой мультисенсорные системы, где аппаратная часть (LiDAR, камеры, радар и др.) обеспечивает базовую геометрию и информацию об окружении, а алгоритмическая часть (от классических геометрических методов до современных нейросетевых архитектур, таких

как YOLO) отвечает за интерпретацию этой информации, классификацию препятствий и принятие решений. В условиях автономного полёта, особенно в неизвестной среде, предпочтение отдаётся лёгким, быстродействующим и адаптивным решениям, способным работать в реальном времени и в разнообразных условиях [5],[7].

1.1.2 Современные методы управления и навигации квадрокоптера

Системы управления и навигации квадрокоптера обеспечивают выполнение заданной траектории, стабилизацию в пространстве и предотвращение отклонений, вызванных внешними возмущениями (ветром, турбулентностью, неидеальностью исполнительных механизмов). Ключевой задачей таких систем является поддержание устойчивости и точного позиционирования квадрокоптера в реальном времени при ограниченных вычислительных и энергетических ресурсах.

В современной литературе выделяются три ключевых класса методов управления, которые отображены в таблице 1.2

Таблица 1.2 – Сравнение основных классов методов управления квадрокоптера

Класс методов	Особенности	Преимущества	Недостатки
Классические (ПИД регулятор)	Используют линейные модели и обратную связь по ошибке	Простота, высокая надёжность, малая вычислительная нагрузка	Плохо адаптируются к нелинейностям и сильным возмущениям
Оптимизационные (линейно-квадратичные регуляторы)	Прогноз поведения системы, минимизация функционала отклонений	Учёт ограничений, высокая точность	Высокая вычислительная сложность
Интеллектуальные (нейросетевые, гибридные)	Обучение на данных, адаптация к среде	Самообучение, устойчивость к неопределённости	Требуют обучения и ресурсов, риск переобучения

Наиболее распространённым методом стабилизации квадрокоптера остаётся ПИД-регулятор. Он формирует управляющее воздействие u на основе текущей ошибки $e(t)$ между заданным и измеренным значением по следующей формуле:

$$u = f(e(t)) = K_p e(t) + K_I \int_0^t e(t) dt + K_D \frac{d}{dt} e(t), \quad (1.4)$$

где K_x – коэффициент пропорциональности;

K_I – коэффициент интегрирования;

K_D – коэффициент дифференцирования;

$\int_0^t e(t) dt$ – интеграл ошибки;

$\frac{de(t)}{dt}$ – дифференциал ошибки.

Первое слагаемое (1.4) $K_p e(t)$ – вычисляет управляющий сигнал пропорционально разнице между текущим значением переменной и заданным уровнем. Данная величина определяет, насколько сильно регулятор реагирует на ошибку в данный момент времени.

Второе слагаемое $K_I \int_0^t e(t) dt$ – пропорционально сумме всех прошлых значений ошибки и позволяет компенсировать постоянные или долговременные отклонения от желаемого значения и помогает уменьшить установочное время.

Последнее слагаемое, т.е. $K_D \frac{de(t)}{dt}$ – пропорционально зависит от скорости изменения ошибки и предназначено для предотвращения перегрузки системы и помогает предотвратить колебания системы при изменении нагрузки или других факторов.

Таким образом, ПИД-регулятор объединяет в себе три компонента: пропорциональный, интегральный и дифференциальный. Это позволяет более точно и эффективно управлять движением динамической системы. ПИД-регулятор непрерывно вычисляет величину ошибки $e(t)$ как разность

между требуемым заданным значением и измеренным параметром процесса и применяет коррекцию, основанную на пропорциональном, интегральном и дифференциальном коэффициентах

На рисунке 1.1 показана структурная схема ПИД-регулятора.

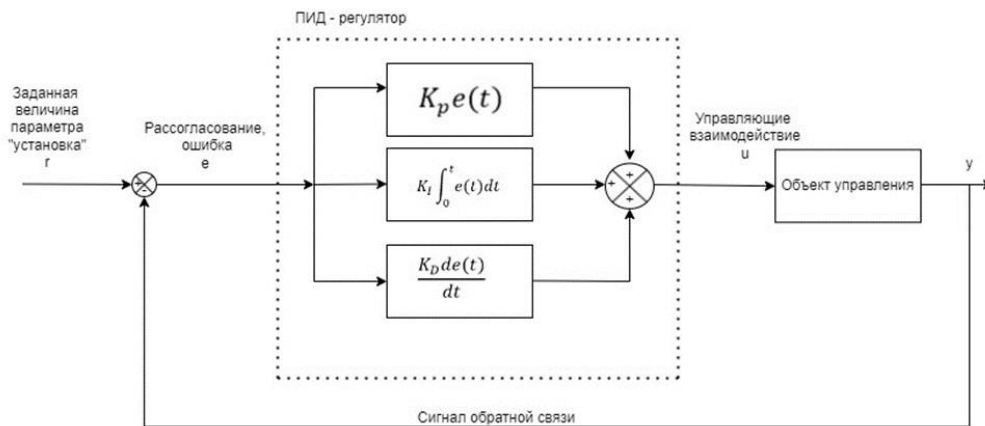


Рисунок 1.1 – Схема ПИД-регулятора

Метод модельно-прогнозирующего управления (MPC) основан на решении задачи оптимизации на каждом шаге времени. Цель – минимизация функционала при ограничениях на входы и состояния. MPC учитывает динамику системы и ограничения на тягу, углы и скорость. Преимущество метода – возможность предсказывать поведение квадрокоптера и корректировать управление до появления ошибки. Недостаток – высокая вычислительная сложность, особенно при большом горизонте предсказания. Тем не менее, использование ускорителей позволяет применять MPC в реальном времени для малых аппаратов.

В последние годы активно развиваются нейросетевые и гибридные подходы к управлению квадрокоптерами, объединяющие классические регуляторы и методы обучения с подкреплением (Reinforcement Learning, RL). Например, методы глубокого обучения с подкреплением (Deep Reinforcement Learning, DRL) позволяют агенту формировать управляющие воздействия на основе взаимодействия с виртуальной средой, в том числе в симуляторе AirSim. Комбинированные архитектуры, такие как ПИД+NN или

MPC+RL, демонстрируют повышенную адаптивность и устойчивость в условиях неопределённости параметров модели и наличия динамических препятствий [6], [8].

Вместе с тем, несмотря на активное развитие интеллектуальных методов управления, ПИД регулятор остаётся наиболее распространённым и широко применяемым средством управления движением беспилотных летательных аппаратов, в частности квадрокоптеров. Это обусловлено его простой структурой, наглядной физической интерпретацией параметров, низкими вычислительными затратами и возможностью устойчивой работы в режиме реального времени на встраиваемых вычислительных платформах. Кроме того, ПИД-регуляторы не требуют предварительного обучения и обладают высокой надёжностью при управлении угловым движением летательного аппарата, что делает их предпочтительным выбором для практических систем.

Таким образом, современная тенденция в области управления квадрокоптерами заключается в интеграции традиционных регуляторов, прежде всего ПИД-регуляторов, с модельными и интеллектуальными методами управления. В сочетании с модулем детекции препятствий такие гибридные архитектуры формируют основу для построения устойчивых и адаптивных систем автономной навигации квадрокоптеров.

1.1.3 Обоснование выбора YOLO как метода детекции препятствий

Современные задачи распознавания и локализации объектов в реальном времени требуют алгоритмов, обеспечивающих высокую скорость обработки и точность при ограниченных вычислительных ресурсах. В контексте систем управления квадрокоптером эти требования становятся особенно критичными, поскольку задержка в обнаружении препятствия может привести к аварийной ситуации. Среди актуальных подходов к детекции

объектов выделяются такие методы, как Faster R-CNN, SSD и YOLO, каждая из которых имеет свои преимущества и ограничения. Сравнительная характеристика архитектуры описана в таблице 1.3

Таблица 1.3 – Сравнение основных архитектур детекции

Метод	Принцип работы	Преимущества	Недостатки	Ср. скорость (FPS)	Ср. точность (mAP)
Faster R-CNN	Двухэтапный метод: генерация областей интереса и классификация	Высокая точность	Низкая скорость; высокая вычислительная сложность	5–7	75–80%
SSD	Одноэтапный метод, использующий несколько уровней признаков	Хороший баланс между скоростью и точностью	Менее точен для мелких объектов	20–40	70–75%
YOLO	Одноэтапная модель, разделяющая изображение на сетку и предсказывающая классы и координаты	Высокая скорость; низкие требования к ресурсам; стабильная точность	Возможны ошибки при перекрытии объектов	45–150	75–85%

В отличие от двухэтапных методов, таких как Faster R-CNN, YOLO выполняет детекцию и классификацию в один проход нейронной сети, что значительно повышает производительность. Алгоритм обрабатывает изображение целиком, а не фрагментами, благодаря чему контекстные взаимосвязи между объектами учитываются более эффективно.

Система детекции препятствий в квадрокоптерах требует алгоритма, способного работать в реальном времени с минимальными задержками и без значительных потерь в точности. В этом контексте архитектура YOLO демонстрирует оптимальное соотношение скорости и качества, что делает её предпочтительным выбором для интеграции в системы визуальной навигации.

На рисунке 1.2 представлена общая схема архитектуры YOLO, включающая свёрточные слои, блоки извлечения признаков и выходные

карты, содержащие вероятности классов и координаты ограничивающих рамок.

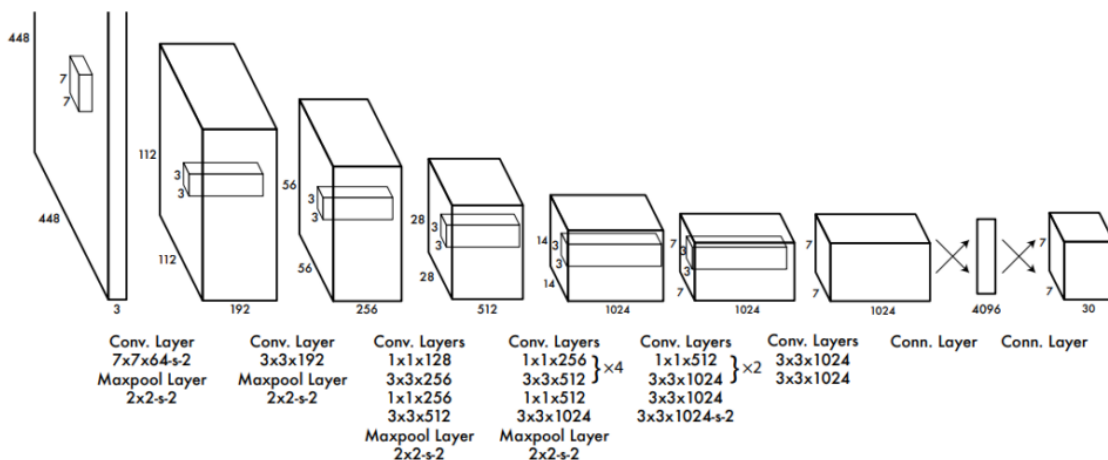


Рисунок 1.2 – Архитектура модели YOLO

Кроме того, современные версии модели, такие как YOLOv5 и YOLOv8, обладают модульной архитектурой, что позволяет адаптировать их под конкретные задачи:

- уменьшить размер сети для работы на встраиваемых системах (варианты YOLOv5n, YOLOv8n),
- использовать улучшенные механизмы якорей и функции потерь, повышающие точность для мелких объектов,
- обеспечивать совместимость с GPU и TPU, что ускоряет обучение и вывод [6].

Преимущества YOLO для систем навигации квадрокоптера:

- Высокая скорость обработки – до 150 кадров в секунду, что позволяет осуществлять визуальную навигацию в реальном времени.
- Низкая вычислительная сложность – возможность работы на компактных устройствах, таких как NVIDIA Jetson или Raspberry Pi.
- Устойчивость к шуму и изменению освещения, что особенно важно при полётах в различных условиях.
- Простота интеграции в существующие системы компьютерного зрения и управления полётом [7].

На данный момент активно используются несколько поколений моделей YOLO – от YOLOv3 до YOLOv9. Каждая последующая версия включает архитектурные улучшения, направленные на повышение точности и стабильности. Однако при выборе конкретной реализации необходимо учитывать не только показатели качества, но и ресурсные ограничения вычислительной платформы, на которой выполняется обработка данных. Характеристики моделей приведены в таблице 1.4.

Таблица 1.4 – Сравнение моделей алгоритма детекции YOLO

Версия модели	Основные особенности	Скорость (FPS)	Точность (mAP@0.5)	Требования к ресурсам
YOLOv5	Устойчивая и лёгкая модель, поддерживает вариации (n/s/m/l/x)	120–140	76–80%	Низкие
YOLOv7	Улучшенные блоки E-ELAN, высокая точность	90–120	80–84%	Средние
YOLOv8	Современная архитектура, поддержка ONNX/TensorRT, автоаннотация, улучшенные функции потерь	100–130	82–85%	Средние
YOLOv9	Максимальная точность, усиленные трансформерные блоки	60–90	86–88%	Высокие

Для решения задачи детекции препятствий выбрана YOLOv8, разработанная компанией Ultralytics, поскольку она сочетает высокую точность и стабильность работы при умеренных вычислительных затратах. По сравнению с YOLOv5, новая версия обладает улучшенной функцией потерь, что повышает точность распознавания мелких объектов и устойчивость к изменению освещения. В то же время, более новые версии, такие как YOLOv9, требуют значительно больших ресурсов, что делает их менее подходящими для применения в реальных системах с ограниченной вычислительной мощностью [7].

Выбор модели YOLOv8 обусловлен сочетанием следующих факторов:

- высокая производительность при сохранении точности,
- адаптивность к различным вычислительным средам,

- совместимость с современными инструментами вывода,
- возможность дообучения на собственных наборах данных, что особенно важно при разработке детекторов для конкретных типов препятствий.

Таким образом, использование YOLO в качестве базового алгоритма детекции препятствий является обоснованным решением, обеспечивающим компромисс между скоростью, точностью и вычислительными затратами. Применение YOLOv8 позволяет повысить надёжность навигации квадрокоптера в условиях ограниченного времени реакции и ресурсов обработки данных, обеспечивая корректное распознавание препятствий в реальном времени [6],[7].

1.1.4 Обоснование выбора среды AirSim для моделирования

Одним из ключевых этапов при разработке и тестировании систем автономного управления квадрокоптером является выбор программной среды моделирования. Использование симуляторов позволяет проводить исследования, отладку алгоритмов навигации и детекции препятствий без необходимости проведения дорогостоящих и потенциально опасных натурных испытаний.

Современные симуляторы квадрокоптера должны удовлетворять ряду требований:

- реалистичное моделирование физических процессов (аэродинамика, столкновения, поведение сенсоров),
- возможность интеграции пользовательских алгоритмов управления,
- поддержка визуальных сенсоров (RGB, глубинная камера, LiDAR),
- возможность автоматизации сценариев и экспорта данных,

– кроссплатформенность и открытость архитектуры для модификации.

Среди наиболее распространённых решений выделяются Gazebo, AirSim, Microsoft Project Bonsai, RotorS и PX4 SITL. Сравнительные характеристики симуляторов представлены в таблице 1.5

Таблица 1.5 – Сравнение ПО симуляторов

Среда моделирования	Физическая модель	Графика / визуализация	Интеграция ИИ / Python	Основные преимущества	Недостатки
Gazebo (ROS)	Достаточно точная, Open Dynamics Engine	Средняя, ограниченные визуальные эффекты	Поддержка через ROS и Python API	Активное сообщество, модульность	Ограниченная графика, сложность настройки
RotorS (ROS)	Простая модель полёта на основе MAVLink	Базовая	Через ROS	Простота и открытость	Отсутствие визуальной среды и реалистики
PX4 SITL / QGround Control	Физика полёта и телеметрия	Минимальная	MAVSDK, C++, Python	Точное моделирование автопилота PX4	Отсутствие 3D-визуализации
Microsoft Project Bonsai	Моделирование с элементами ИИ	Высокая (Unity)	Python SDK	Автоматизация обучения агентов	Закрытая экосистема, требует лицензии
AirSim (Unreal Engine)	Реалистичная физика и динамика полёта	Фотореалистичная (Unreal Engine)	Поддержка Python, C++, API RPC	Реализм, гибкость, поддержка ИИ и глубокого обучения	Высокие требования к оборудованию

Согласно обзорам и практическим исследованиям, AirSim, разработанный Microsoft Research, является одной из наиболее полнофункциональных и гибких платформ для симуляции автономных систем. Она сочетает фотореалистичную графику, точное физическое моделирование и удобный API для взаимодействия с ИИ и алгоритмами управления [9].

Основные преимущества AirSim:

- Фотореалистичная среда: благодаря использованию движка Unreal Engine, достигается высокая точность визуализации, что особенно важно при обучении и тестировании моделей компьютерного зрения (например, YOLO, CNN).
- Реалистичная физика: встроенная модель аэродинамики обеспечивает приближённое к реальности поведение квадрокоптера, включая моменты инерции, силу тяги, сопротивление воздуха и турбулентность.
- Гибкая архитектура – поддержка работы через Python API, C++ и RPC-интерфейсы, что позволяет интегрировать собственные алгоритмы навигации, стабилизации и анализа данных.
- Наличие виртуальных сенсоров – RGB, Depth, Segmentation, LiDAR, ИМИ, GPS – это делает систему удобной для исследований в области распознавания и автономного управления.
- Открытый исходный код и активное сообщество разработчиков, позволяющие вносить изменения и адаптировать симулятор под конкретные задачи.

На рисунке 1.3 представлен пример визуализация работы симулятора AirSim с моделью квадрокоптера в среде Unreal Engine.



Рисунок 1.3 – Пример визуализации полёта квадрокоптера в AirSim

Одним из ключевых преимуществ AirSim является тесная интеграция с графическим движком Unreal Engine, что обеспечивает не только визуальную достоверность, но и возможность моделирования сложных физических взаимодействий и освещения сцены. Unreal Engine поддерживает динамические тени, отражения и текстуры высокого разрешения, что позволяет создавать максимально приближённые к реальности условия восприятия для систем компьютерного зрения.

AirSim в сочетании с Unreal Engine обеспечивает оптимальный баланс между реализмом визуализации, точностью физического моделирования и гибкостью архитектуры для научных и инженерных задач. Это делает данную связку подходящей для разработки и тестирования систем навигации и детекции препятствий, особенно при использовании нейросетевых моделей, таких как YOLOv8.

Таким образом, выбор среды AirSim для моделирования полёта и восприятия окружающей среды является обоснованным решением. Эта платформа сочетает реалистичную физику, фотореалистичное изображение, удобные инструменты интеграции и широкие возможности расширения. Использование Unreal Engine в качестве графического движка обеспечивает визуальную достоверность и возможность проведения полунатурных экспериментов в условиях, максимально приближённых к реальным.

1.2 Описание стенда полунатурного моделирования

1.2.1 Назначение и общая структура стенда полунатурного моделирования

Для проведения исследований и отработки алгоритмов управления полётом квадрокоптера был разработан стенд полунатурного моделирования системы управления движением. Основным назначением стенда является создание экспериментальной среды, позволяющей исследовать и тестировать

алгоритмы управления, стабилизации и визуального моделирования движения квадрокоптера без выполнения реальных лётных испытаний.

Полунатурное моделирование представляет собой компромисс между чисто математическим моделированием и натурными экспериментами. В рамках данного подхода реальные аппаратные компоненты используются совместно с виртуальной средой моделирования, что позволяет учитывать особенности работы датчиков, задержки передачи данных и влияние шумов, сохраняя при этом высокий уровень безопасности и воспроизводимости экспериментов.

Разработанный стенд обеспечивает измерение углов ориентации квадрокоптера по трём осям – крену, тангажу и рысканию – с использованием инерциального измерительного модуля MPU-6050, установленного на механической платформе. Измеренные данные передаются на микроконтроллер ESP32, который выполняет первичную обработку информации и обеспечивает беспроводную передачу данных на персональный компьютер.

На персональном компьютере осуществляется приём данных ориентации, их программная обработка и формирование управляющих воздействий, которые передаются в симуляционную среду AirSim, интегрированную с игровым движком Unreal Engine. Таким образом, движение механической части стенда напрямую отражается в виртуальной модели квадрокоптера, обеспечивая визуальную синхронизацию между реальными измерениями и цифровым представлением.

Общая структурная схема стенда полунатурного моделирования представлена на рисунке 1.4.

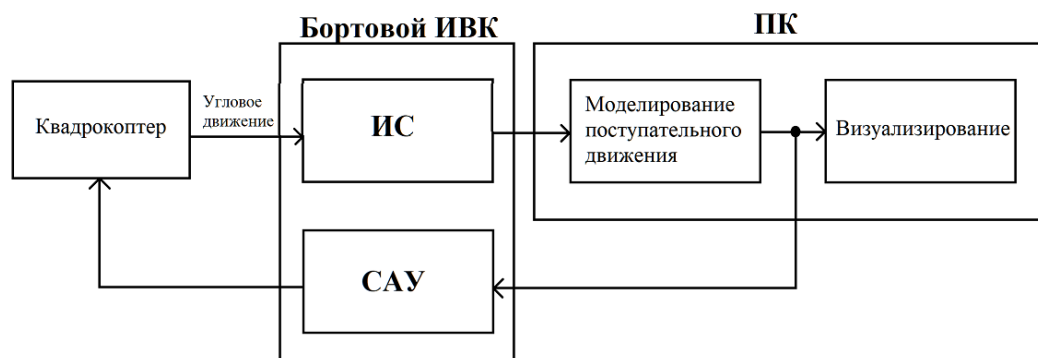


Рисунок 1.4 – Общая схема стенда полунатурного моделирования

На рисунке 1.4 обозначены следующие основные компоненты:

- Квадрокоптер,
- Бортовой измерительно-вычислительный комплекс (ИВК),
- Измерительная система (ИС),
- Система автоматического управления (САУ),
- Персональный компьютер (ПК).

Подробное описание конструкции стенда, его механической реализации, схемотехнических решений, способов крепления и используемых аппаратных компонентов приведено в технологической части работы (раздел 2).

Таким образом, разработанный стенд полунатурного моделирования представляет собой целостную аппаратно-программную систему, объединяющую физическую часть, вычислительную среду и программное обеспечение моделирования. Он обеспечивает условия для комплексного исследования алгоритмов управления, визуализации движения и анализа поведения квадрокоптера, выступая самостоятельным средством экспериментальной отработки систем управления беспилотными летательными аппаратами.

1.2.2 Схема информационного обмена и синхронизации данных

В отличие от классических симуляторов, где движение летательного аппарата формируется исключительно на основе математических моделей, в разработанной системе в качестве первичного источника ориентационных данных используется физический квадрокоптер, установленный на механической платформе с тремя степенями свободы. Это позволяет учитывать реальные инерционные и динамические эффекты, возникающие при изменении ориентации объекта, и повышает достоверность моделирования.

На рисунке 1.5 представлена схема полного цикла взаимодействия аппаратной и программной частей системы, отражающая процессы сбора данных, их обработки, передачи, визуализации и формирования обратной связи.



Рисунок 1.5 – Схема полного цикла работы программы

Физическая часть стенда представлена натурным оборудованием, включающим в себя квадрокоптер, закрепленный на механической платформе, с установленным инерциальным измерительным модулем MPU6050. Конструкция платформы обеспечивает вращение квадрокоптера вокруг трёх осей: крена, тангажа и курса, что соответствует ориентационным степеням свободы летательного аппарата.

При угловом движении квадрокоптера его угловые скорости и линейные ускорения регистрируются MPU6050. Считанные значения таких

физических величин передаются на микроконтроллер ESP32, где осуществляется:

- фильтрация шумов инерциальных измерений,
- коррекция смещений гироскопа и акселерометра,
- приведение данных к единому формату и системе координат.

В памяти микроконтроллера заложен алгоритм расчет углов ориентации с использованием оптимального фильтра Калмана. В результате получаются оценки углов курса (pitch), крена (roll) и курса (yaw)

$$\vartheta_{\text{текущ}} = \begin{bmatrix} \theta_{\text{текущ}} \\ \varphi_{\text{текущ}} \\ \psi_{\text{текущ}} \end{bmatrix},$$

где θ – угол тангажа;

φ – угол крена;

ψ – угол курса.

Полученные данные вместе с измеренными значениями угловых скоростей являются базовыми входными величинами всей системы и отражают фактическое состояние физического объекта.

После получения оценок углов ориентации и угловых скоростей квадрокоптера в бортовом измерительно-вычислительном комплексе данные передаются по беспроводному каналу связи на персональный компьютер. Выбор беспроводного канала обусловлен необходимостью исключения механических ограничений подвижной платформы стенда и обеспечения гибкости при проведении экспериментальных исследований.

На персональном компьютере функционирует программный комплекс полунатурного моделирования, интегрированный со средой AirSim и движком Unreal Engine. Угловая ориентация и угловые скорости виртуальной модели квадрокоптера в среде AirSim принимаются идентичными измеренным значениям, поступающим с физического стенда.

Таким образом, в разработанной архитектуре моделирование углового движения квадрокоптера осуществляется физической частью стенда, тогда как в виртуальной среде AirSim выполняется расчёт поступательного движения летательного аппарата. Для этого используются измеренные углы ориентации, угловые скорости, а также расчётное значение суммарной силы тяги. Поступательное движение вычисляется на основе уравнений динамики квадрокоптера в связанной системе координат, приведённых в выражениях (1.7).

В результате интегрирования данных уравнений в среде AirSim определяются линейные скорости и координаты виртуального квадрокоптера. Полученные координаты сравниваются с заданным целевым положением, определяющим сценарий полёта. Разность между текущими и требуемыми координатами используется для формирования требуемых углов ориентации квадрокоптера, необходимых для продолжения движения к целевой точке.

Сформированное $\vartheta_{\text{тр}}$ – требуемое значение угла ориентации сравнивается с $\vartheta_{\text{текущ}}$ – текущим измеренным углом физического квадрокоптера. Разность между ними определяет ошибку управления ориентацией:

$$\Delta\vartheta = \vartheta_{\text{текущ}} - \vartheta_{\text{тр}} \quad (1.5)$$

Полученная ошибка используется для выработки управляющего воздействия, направленного на изменение ориентации физического квадрокоптера на механической платформе стенда.

Особенностью разработанной архитектуры является наличие замкнутого контура обратной связи, в котором виртуальная среда AirSim используется не только для визуализации, но и для вычисления координат, линейных скоростей и формирования требуемых ориентаций, это позволяет:

- учитывать динамику поступательного движения квадрокоптера,

- моделировать влияние ориентации на траекторию полёта,
- обеспечивать согласованную работу физической и виртуальной частей стенда.

В результате AirSim выступает в качестве активного вычислительного элемента полунатурной системы моделирования, обеспечивающего расчёт поступательного движения квадрокоптера в режиме, близком к реальному времени.

Схема, представленная на рисунке 1.5, обобщает архитектуру полунатурного моделирования и отражает:

- использование математической модели поступательного движения квадрокоптера,
- принципы получения и передачи данных ориентации с модуля MPU-6050,
- взаимодействие физического стенда с виртуальной средой AirSim,
- формирование замкнутого контура управления.

Представленный подход обеспечивает целостное описание системы полунатурного моделирования и служит основой для дальнейшего рассмотрения конструктивной и аппаратной реализации стенда в технологической части работы.

1.2.3 Система управления угловым движением квадрокоптера на базе MPU-6050

1.2.3.1 Математическая модель углового движения квадрокоптера

Квадрокоптер представляет собой беспилотный летательный аппарат с четырьмя соосными двигателями, расположенными в вершинах крестообразной рамы. Два винта вращаются по часовой стрелке, два –

против, что обеспечивает компенсацию реактивных моментов и позволяет аппарату выполнять взлёт, зависание, маневрирование и посадку. Управление движением осуществляется изменением угловых скоростей вращения винтов. Схема показана на рисунке 1.6.

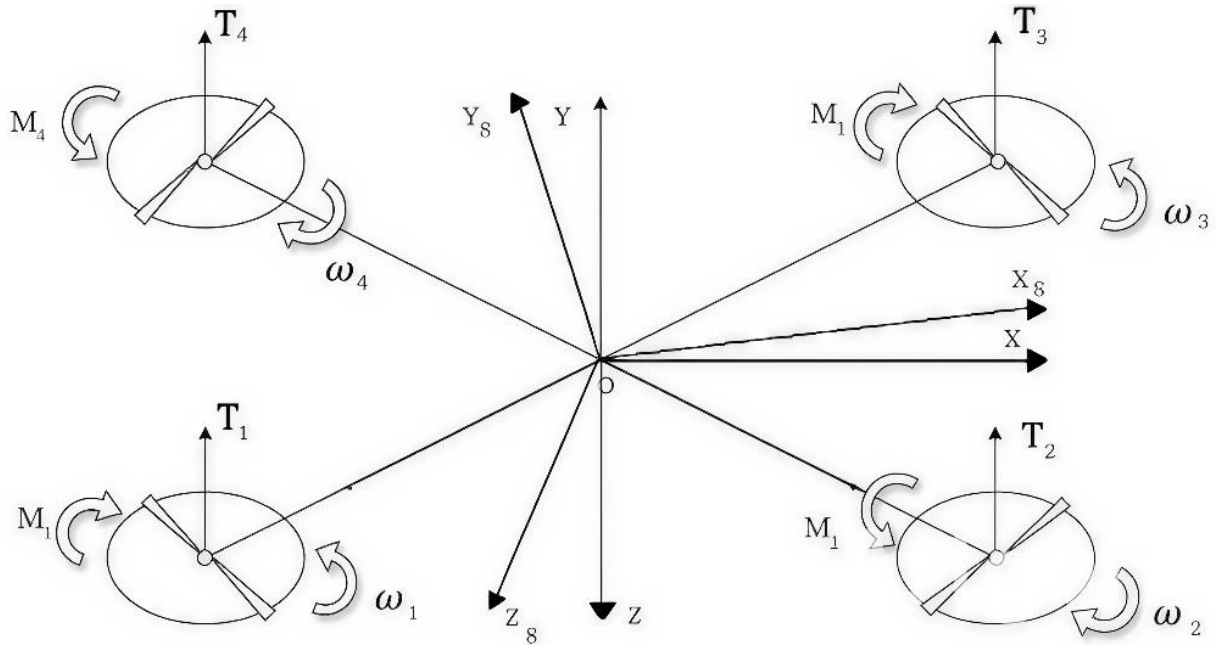


Рисунок 1.6 – Кинематическая схема квадрокоптера в ССК и NED

На рисунке 1.6. представлена схема квадрокоптера, где ω_i – угловая скорость i -ого винта; T_i – сила тяги i -ого винта; M_i – момент сопротивления i -ого винта.

Динамика квадрокоптера описывается системой нелинейных дифференциальных уравнений, включающих поступательные и вращательные движения.

Поступательное движение центра масс:

$$m \cdot \ddot{\vec{r}} = T + mg + F_{\text{аэро}}, \quad (1.6)$$

где $\vec{r} = [x, y, z]^T$ – координаты в пространстве;

m – масса аппарата;

T – суммарная тяга от двигателей;

mg – сила тяжести;

$$F_{\text{аэро}} = \begin{bmatrix} k_v V_{xb}^2 \\ -k_v V_{yb}^2 \\ k_v V_{zb}^2 \end{bmatrix} - \text{аэродинамическая сила;}$$

k_v – приведенный коэффициент аэродинамического сопротивления.

Уравнение (1.6) в ССК имеет следующий вид:

$$\begin{cases} \dot{V}_{xb} = -\frac{k}{m} V_{xb}^2 + \omega_y V_{yb} - g \cdot \sin \theta, \\ \dot{V}_{yb} = \frac{T - k V_{yb}^2}{m} - \omega_z V_{xb} + \omega_x V_{zb} - g \cdot \cos \theta \cos \varphi, \\ \dot{V}_{zb} = -\frac{k}{m} V_{zb}^2 + \omega_x V_{yb} - \omega_y V_{xb} - g \cdot \cos \theta \sin \varphi. \end{cases} \quad (1.7)$$

где V_{xb} , V_{yb} , V_{zb} – проекции линейной скорости в связанной системе координат;

ω_x , ω_y , ω_z – проекции угловой скорости;

$T = T_1 + T_2 + T_3 + T_4$ – сумма сил тяги моторов, T_i – сила тяги i -ого мотора.

Вращательное движение описывается уравнением Эйлера:

$$I \cdot \dot{\omega} + \omega \times (I \cdot \omega) = \tau, \quad (1.8)$$

где I – тензор инерции;

ω – угловая скорость;

τ – управляющий момент от роторов.

Эти уравнения формируют основу для разработки модулей стабилизации и управления, обеспечивающих реализацию устойчивого полёта.

Моменты инерции считаются диагональными, так как корпус квадрокоптера на стенде симметричен, тензор инерции принимается диагональным:

$$I = \begin{pmatrix} I_x & 0 & 0 \\ 0 & I_y & 0 \\ 0 & 0 & I_z \end{pmatrix} \quad (1.9)$$

С учетом этого вращательное движение описывается системой дифференциальных уравнений:

$$\begin{cases} I_x * \dot{\omega}_x = (I_y - I_z)\omega_y\omega_z + \tau_x, \\ I_y * \dot{\omega}_y = (I_z - I_x)\omega_x\omega_z + \tau_y, \\ I_z * \dot{\omega}_z = (I_x - I_y)\omega_x\omega_y + \tau_z. \end{cases} \quad (1.10)$$

где τ_x, τ_y, τ_z – управляющие моменты, создаваемые моторами.

Суммарные моменты, создаваемые моторами квадрокоптера:

$$\begin{cases} \tau_x = l(T_2 - T_4), \\ \tau_y = l(T_3 - T_1), \\ \tau_z = k(T_1 - T_2 + T_3 - T_4). \end{cases} \quad (1.11)$$

где l – плечо силы тяги;

k – приведенный коэффициент реактивного момента винта.

Углы ориентации квадрокоптера в пространстве определяются по следующим уравнениям:

$$\begin{cases} \dot{\phi} = \omega_x + \omega_y \sin \varphi \tan \theta + \omega_z \cos \varphi \tan \theta, \\ \dot{\theta} = \omega_y \cos \varphi - \omega_z \sin \varphi, \\ \dot{\psi} = \omega_y \sin \varphi / \cos \theta + \omega_z \cos \varphi / \cos \theta. \end{cases} \quad (1.12)$$

Благодаря симметричности квадрокоптера можно считать, что движения по тангажу, крену и курсу могут быть реализованы независимо. Следовательно, можем приблизительно переписать уравнения (1.12) в следующих:

$$\begin{cases} \varphi(t) = \int w_x dt, \\ \theta(t) = \int w_y dt, \\ \psi(t) = \int w_z dt. \end{cases}$$

1.2.3.2 Каскадные системы управления угловым движением квадрокоптера

Для реализации устойчивого и эффективного управления угловым движением квадрокоптера применяется каскадная система автоматического управления, обеспечивающая стабилизацию ориентации по трем осям: тангажу, крену и курсу. Использование каскадной структуры позволяет разделить задачи управления на несколько уровней, каждый из которых отвечает за свою динамику и масштаб изменений.

Каскадная система управления угловым движением квадрокоптера, как правило, включает два контура управления:

- внешний контур, формирующий требуемые угловые скорости на основе ошибки по углам ориентации,
- внутренний контур, обеспечивающий стабилизацию угловых скоростей и формирование управляющих воздействий на исполнительные органы.

Структурная схема каскадной системы управления угловым движением квадрокоптера представлена на рисунке 1.7.

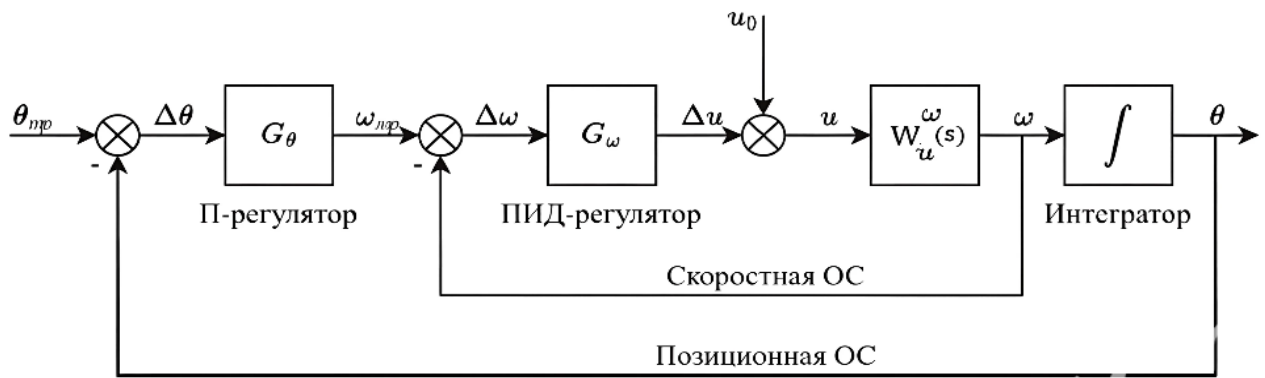


Рисунок 1.7 – Структурная схема системы управления угловым движением квадрокоптера

На рисунке 1.7 указаны следующие обозначения, где $\theta_{тр}$ – требуемый угол, на который следует отклонить квадрокоптер;

θ – текущий угол отклонения квадрокоптера;

$\Delta\theta$ – отклонение текущего угла от требуемого;

$\omega_{тр}$ – требуемая угловая скорость поворота квадрокоптера;

ω – текущая угловая скорость поворота квадрокоптера;

$\Delta\omega$ – отклонение текущей угловой скорости от требуемой;

Δu – корректирующее управление;

u_0 – сигнал управления, необходимый для зависания квадрокоптера на заданной высоте;

u – командный сигнал для объекта;

$W_u^\omega(s)$ – передаточная функция по угловой скорости.

Во внешнем контуре осуществляется сравнение заданного и текущего углов ориентации квадрокоптера. На основе ошибки по углу формируется требуемая угловая скорость, которая передается во внутренний контур. Внешний контур, реализуется с использованием пропорционального регулятора и отвечает за плавность и точность ориентации квадрокоптера в пространстве.

Внутренний контур управления предназначен для стабилизации угловых скоростей. Он обрабатывает ошибку между требуемой и текущей угловой скоростью и формирует управляющее воздействие, поступающее на

исполнительные органы квадрокоптера. Для реализации внутреннего контура широко применяется ПИД-регулятор, обеспечивающий достаточную быстроедействие и устойчивость системы при внешних возмущениях.

Передаточные функции регуляторов в каскадной системе управления могут быть представлены в следующем виде:

$$G_{\theta}(s) = \frac{\omega_{\text{тр}}(s)}{\Delta\theta(s)} = K_x,$$
$$G_{\omega}(s) = \frac{\Delta u(s)}{\Delta\omega(s)} = K_P + \frac{K_I}{s} + K_D s.$$

Коэффициент K_x представляет собой коэффициент пропорционального регулятора внешнего контура, осуществляющего преобразование ошибки по углу ориентации в задание по угловой скорости.

Внутренний контур управления реализован в виде ПИД-регулятора угловой скорости с коэффициентами K_P , K_I и K_D , физический смысл и назначение которых были рассмотрены в п. 1.1.2.

Применение каскадной структуры управления позволяет существенно улучшить динамические характеристики системы по сравнению с одноконтурными схемами. Разделение управления на контур ориентации и контур угловых скоростей снижает взаимное влияние каналов, упрощает настройку системы и повышает устойчивость при воздействии внешних возмущений.

Каскадные системы управления угловым движением получили широкое распространение в современных квадрокоптерах благодаря своей надёжности, относительной простоте реализации и эффективности при работе в реальном времени. Они успешно применяются как в реальных бортовых системах управления, так и в средах моделирования и полунатурных стендах, используемых для отладки и тестирования алгоритмов стабилизации и навигации.

1.2.3.3 Измерительная система

В качестве измерительной системы, обеспечивающей алгоритму управления данные о текущем положении квадрокоптера, используется инерциальный модуль измерения (ИМИ) MPU-6050, состоящий из трех датчиков угловых скоростей (ДУС) и трех датчиков линейных ускорений.

Использование ИМИ MPU-6050 в данном проекте обусловлено сочетанием нескольких факторов:

- высокая частота измерений (до 1000 Гц),
- встроенный DMP-фильтр, уменьшающий шумы,
- простой интеграцией через интерфейс I²C и микроконтроллер ESP32,
- низкая стоимость и массовая распространённость,
- наличие готовых библиотек, ускоряющих интеграцию,
- достаточная точность для задач визуализации и стенда.

ИМИ MPU-6050 представляет собой инерциальный измерительный модуль, совмещающий трёхосевой гироскоп и трёхосевой акселерометр, что позволяет получать полную информацию об угловых скоростях и линейных ускорениях квадрокоптера. Подробные аппаратные характеристики датчика, включая диапазоны измерений, чувствительность и параметры чувствительных элементов, приведены в технологической части (см. раздел 2). В контексте разработанного полунатурного стенда для визуализации поведения квадрокоптера данные MPU-6050 служат основой для определения текущих угловых координат – крена, тангажа и рыскания, которые затем синхронизируются с моделью в Unreal Engine.

Для корректной интерпретации данных важно учитывать ориентацию чувствительных осей. На плате MPU-6050 они обозначены следующим образом: ось X направлена вдоль длинной стороны модуля, Y – поперёк, а Z – перпендикулярна плате. В работе был выполнен монтаж датчика таким

образом, чтобы его оси совпадали с системой координат стенда. Любое отклонение от идеального выравнивания приводит к появлению постоянных смещений, что характерно для MEMS-датчиков.

MPU-6050 поддерживает выбор шкал чувствительности. Для акселерометра доступные диапазоны: $\pm 2g$, $\pm 4g$, $\pm 8g$ и $\pm 16g$. Для гироскопа: ± 250 , ± 500 , ± 1000 и $\pm 2000^\circ/\text{с}$. В проекте использовались шкалы $\pm 2g$ и $\pm 250^\circ/\text{с}$, поскольку стенд имитирует только угловые повороты и не создаёт экстремальных ускорений. Эти настройки обеспечивают наименьший шум и наибольшую точность.

Работа MPU-6050 сопровождается типичными для MEMS-устройств погрешностями:

- Дрейф гироскопа – проявляется как медленное изменение нулевого смещения при отсутствии вращения;
- Акселерационная погрешность – акселерометр реагирует на любые ускорения, а не только на гравитацию. Поэтому при быстрых движениях измерения «загрязняются» динамическими ускорениями;
- Белый шум – проявляется как мелкие высокочастотные колебания измерений;
- Монтажный уклон – если датчик установлен под углом даже $2-3^\circ$, это приводит к постоянному смещению;
- Резонансные вибрации – характерны для платформ с моторами.

Даже при полной неподвижности гироскоп показывает небольшие ненулевые значения угловых скоростей. В акселерометре основной источник ошибок – реакция на любые ускорения помимо гравитации. В условиях стенда эти факторы выражены слабо, однако их влияние остаётся заметным при длительной работе.

Для уменьшения этих погрешностей в MPU-6050 встроен цифровой процессор DMP, способный самостоятельно выполнять часть вычислений: компенсировать дрейф гироскопа, вычислять ориентацию на основе

объединения данных от гироскопа и акселерометра и даже формировать кватернионы положения в пространстве. Использование DMP позволяет отказаться от реализации фильтра Калмана или комплементарных схем, которые потребовали бы значительных вычислительных ресурсов и сложной настройки. В данной работе DMP выполняет функцию первичной фильтрации и стабилизации измерений.

Одним из преимуществ DMP является то, что он предоставляет ориентацию устройства сразу в виде кватерниона – компактного математического представления вращения в трёхмерном пространстве. Кватернион удобнее классических углов Эйлера, поскольку не имеет сингулярностей. Формально кватернион записывается в виде:

$$q = (w, x, y, z), \quad (1.13)$$

где w – скалярная часть;

x, y, z – векторная часть.

Преобразование кватерниона в углы рыскания, крена и тангажа выполняется через тригонометрические выражения. В библиотеке MPU6050 эта операция реализована автоматически.

Работа алгоритма чтения кватерниона начинается с запроса данных из FIFO-буфера, куда DMP помещает уже обработанные значения. В программе это выглядит следующим образом (рисунок 1.8):

```
// ===== Считывание с DMP =====
if (!dmpReady) return;

if (mpu.dmpGetCurrentFIFOPacket(fifoBuffer)) {
    mpu.dmpGetQuaternion(&q, fifoBuffer);
    mpu.dmpGetGravity(&gravity, &q);
    mpu.dmpGetYawPitchRoll(ypr, &q, &gravity);
}
```

Рисунок 1.8 – Пример кода обработки кватерниона

В этом фрагменте считывается очередной пакет обработанных данных; извлекается кватернион ориентации q вычисляется направление гравитации; из последних двух величин формируются углы yaw-pitch-roll.

Для дальнейшей визуализации в Unreal Engine выполняется перевод из радиан в градусы (рисунок 1.9):

```
// Радианы → градусы
float yaw = ypr[0] * 180.0 / M_PI;
float pitch = ypr[1] * 180.0 / M_PI;
float roll = ypr[2] * 180.0 / M_PI;
```

Рисунок 1.9 – Пример кода перевода в градусы

Именно эти три величины формируют ориентацию квадрокоптера и затем передаются на компьютер (рисунок 1.10):

```
// === Отправка по UDP (если активировано) ===
if (active && millis() - lastSend >= 50) { // ~20 Гц
    char msg[64];
    snprintf(msg, sizeof(msg), "%.2f,%.2f,%.2f", yaw, pitch, roll);
    udp.beginPacket(pcIP, pcPort);
    udp.print(msg);
    udp.endPacket();
    lastSend = millis();
}
```

Рисунок 1.10 – Пример кода передачи пакетов по UDP

Из этого видно, что MPU-6050 в собранной системе играет роль первичного измерителя угловой ориентации. ESP32 обеспечивает:

- инициализацию и калибровку датчика,
- получение кватернионов и вычисление углов,
- передачу данных по беспроводной сети на компьютер с Unreal Engine.

Таким образом, модуль MPU-6050 является ключевым элементом подсистемы ориентации, обеспечивая синхронизацию физического стенда и виртуальной модели. Применение DMP позволило отказаться от дополнительных фильтров, ограничившись встроенными средствами

стабилизации данных. Подробная реализация программной части, включая полный листинг модуля вычисления ориентации, и передачи телеметрии приведена в Приложении А.

1.2.4 Полетный контроллер на базе микроконтроллера ESP32

Так как алгоритмы стабилизации требуют непрерывного получения текущих углов крена, тангажа и рыскания, а также их угловых скоростей, следующим этапом разработки системы управления является интеграция ИМИ MPU-6050, микроконтроллера ESP32, двигателей квадрокоптера и коммутационной электроники. В данном пункте рассматривается структура аппаратной части, процесс подключения датчиков и исполнительных механизмов, а также процедуры калибровки, необходимые для получения корректных измерений ориентации.

На рисунке 1.11 представлена структура узлов системы управления угловым движением квадрокоптера.

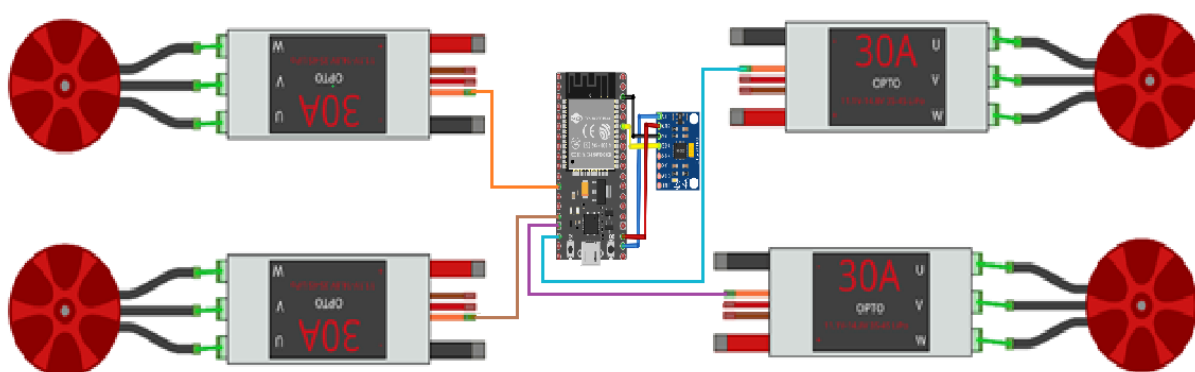


Рисунок 1.11 – Электрическая схема системы управления квадрокоптером

Согласно рис 1.11, схема состоит из:

- ИМИ MPU-6050,
- микроконтроллер ESP32,
- четыре электронных регулятора скорости (ESC),
- четыре бесколлекторных двигателя (BLDC).

MPU-6050 подключается к ESP32 по интерфейсу I²C, где линии SDA и SCL подаются на аппаратные пины ESP32. Данные акселерометра и гироскопа поступают с частотой до 1 кГц, а благодаря встроенному цифровому процессору движения (Digital Motion Processor – DMP) можно использовать аппаратную фильтрацию и получать уже обработанные кватернионы и углы Эйлера, существенно снижая нагрузку на микроконтроллер.

ESC подключаются к ESP32 через соединение к специализированным пинам и принимают оттуда PWM-сигналы, формируемые библиотекой ESP32Servo из аналоговых сигналов управления, полученных в алгоритме автоматического управления угловым движением квадрокоптера. Каждый регулятор получает импульсы шириной от 1000 до 2000 мкс, что соответствует минимальной и максимальной тяге двигателя.

Таким образом, ESP32 является одновременно и вычислительным, и управляющим центром: он принимает данные ориентации и формирует управляющие воздействия на BLDC-двигатели, реализуя замкнутый контур стабилизации.

Подключение ИМИ MPU-6050 включает несколько важнейших этапов, каждый из которых влияет на точность работы системы.

В функции `setup()` конфигурируется высокоскоростной I²C-режим, пример кода отображен на рисунке 1.12.

```
// I2C
Wire.begin(21, 22);
Wire.setClock(400000);
Serial.println("I2C инициализирован (SDA=21, SCL=22)");
```

Рисунок 1.12 – Пример инициализации I²C

Увеличенная частота шины необходима для быстрого обмена данными с MPU-6050, особенно при использовании DMP-режима.

ИМИ MPU-6050 требует загрузки микропрограммы DMP. Пример кода отображен на рисунке 1.13

```
// Инициализация MPU и DMP
Serial.println(F("Инициализация MPU6050..."));
mpu.initialize();
uint8_t devStatus = mpu.dmpInitialize();
Serial.print(F("Статус dmpInitialize: "));
Serial.println(devStatus);
```

Рисунок 1.13 – Пример инициализации MPU6050 и прошивки DMP

На этом этапе загружается алгоритм, который на аппаратном уровне объединяет данные акселерометра и гироскопа, вычисляет кватернионы и формирует прерывания при готовности нового блока данных.

Аппаратная калибровка – критически важный этап, поскольку она определяет корректность дальнейшего вычисления углов. На рисунке 1.14 пример кода.

```
// Калибровка (
Serial.println(F("Калибровка (акселерометр+гироскоп) ... Не двигайте датчик!"));
mpu.CalibrateAccel(6);
mpu.CalibrateGyro(6);
mpu.PrintActiveOffsets();
```

Рисунок 1.14 – Пример калибровки MPU6050

Они выполняют несколько циклов измерений инерциальных датчиков и определяют нулевые смещения в измерениях по всем осям. Убирается систематическая ошибка гироскопа – дрейф, который при интегрировании угловых скоростей приводит к нарастающей ошибке ориентации. Калибровка акселерометра устраняет смещения по измерениям кажущегося ускорения, особенно важные для расчёта тангажа и крена.

После успешной калибровки активируется цифровой процессор движения. Пример кода отображен на рисунке 1.15.


```

if (devStatus == 0) {
    mpu.setDMPEnabled(true);
    packetSize = mpu.dmpGetFIFOPacketSize();
    dmpReady = true;
    Serial.println(F("DMP готов"));
} else {
    Serial.println(F("Ошибка инициализации DMP"));
    while (1) { delay(1000); }
}

```

Maksim Kravtov, 2 недели назад • дописал пункт 1.2.4 +

Рисунок 1.15 – Пример настройки DMP

ИМИ MPU-6050 начинает самостоятельно формировать полноценные кватернионы ориентации и выдавать их по прерыванию. Это позволяет синхронизировать цикл считывания данных с точной частотой (обычно 100–200 Гц).

Параллельно с инициализацией ИМИ MPU-6050 выполняется подключение регуляторов ESC. Каждый регулятор получает сигнал от своего PWM-пина (рисунок 1.16)

```

/* ----- Для моторов -----
      .
      /|\y
  4  |  |  |  1
  |  |  |  |
  |  |  |  |
----->x
  3  |  |  |  2
  |  |  |  |
  |  |  |  |
*/
motor_1.attach(26);
delay(300);
motor_2.attach(25);
delay(300);
motor_3.attach(33);
delay(300);
motor_4.attach(32);
delay(300);

```

Рисунок 1.16 – Пример инициализации двигателей

Перед началом работы необходимо выполнить стандартную процедуру калибровки ESC:

- Подается на вход всех ESC максимальная команда (2000 мкс),
- После паузы – минимальная (1000 мкс).

В проекте это реализовано функцией `calibrateMotors()` (Рисунок 1.17).

```
//Калибровка моторов
Serial.println("SETUP: Calibrate motors");
calibrateMotors();
Serial.println("SETUP: Motors calibrated");
```

Рисунок 1.17 – Пример калибровки моторов

Без калибровки ESC могут неправильно интерпретировать диапазон управляющих импульсов. Это приведет к неожиданным скачкам тяги, асимметрии или даже аварийному запуску одного из двигателей.

После калибровки передается команда `armMotors()`, устанавливающая минимальное значение PWM и переводящая регуляторы в рабочий режим.

Этот шаг обязателен, так как ESC воспринимают минимальную тягу как готовность к управлению, а любое другое значение как попытку немедленного запуска двигателя.

ESP32 постоянно анализирует корректность работы ИМИ MPU-6050:

- проверяет регулярность прихода DMP-прерываний,
- анализирует объем данных в FIFO-буфере ИМИ,
- контролирует превышение угловых ограничений.

Если ориентация выходит за пределы, установленные константами, то выполнение немедленно прекращается. Это обеспечивает безопасную работу на стенде и предотвращает удара рамы о механические ограничители подвеса (Рисунок 1.18).

```
// если углы слишком большие - отключение
if (abs(pitch) > 60.0 || abs(roll) > 60.0 || abs(yaw) > 120.0) {
    emergencyStop = true;
    Serial.println("Аварийная остановка: превышен предельный угол");
}
```

Рисунок 1.18 – Пример аварийной остановки квадрокоптера

Таким образом, процесс интеграции квадрокоптера, ESP32 и ИМИ MPU-6050, включает калибровку сенсоров, настройку DMP-режима, инициализацию двигателей и защиту системы. Однако полученные

измерения ориентации сами по себе ещё не являются готовыми для применения в контуре управления – они требуют корректного понимания структуры данных MPU-6050, принципов получения акселерометрических и гироскопических измерений, особенностей шкал и осей, а также анализа возможных источников ошибок.

1.2.5 Двухсторонняя передача данных ориентации от ESP32 на ПК по протоколу UDP

Как отмечалось в описании стенда полунатурного моделирования (см. п.1.2.1), для визуализации движения квадрокоптера в среде моделирования Airsim требуется информация об угловом положении и угловом движении квадрокоптера. Следовательно, после получения угловой ориентации квадрокоптера с использованием ИМИ MPU-6050 и встроенного цифрового процессора DMP необходимо передать эти данные на персональный компьютер, где выполняется программная визуализация движения модели в среде Unreal Engine. Для полунатурного стенда принципиально важно обеспечить минимальные задержки, устойчивую передачу данных и синхронизацию между физическим датчиком и виртуальной моделью. В данном пункте рассматривается организация передачи данных по Wi-Fi с использованием протокола UDP, структура формируемых пакетов, особенности их обработки и программная реализация данного механизма.

ESP32 поддерживает несколько способов беспроводной передачи данных: Bluetooth, Wi-Fi TCP и Wi-Fi UDP. Для стенда с имитацией углового движения квадрокоптера принципиальным является минимальное время отклика между фактом изменения ориентации квадрокоптера и обновлением положения модели в Unreal Engine. Протоколы с подтверждением доставки TCP обеспечивают высокую надёжность, но вносят задержки 20–70 мс из-за механизмов контроля пакетов, повторной пересылки и управления потоком.

Протокол UDP, напротив, обеспечивает передачу с минимальным накладным временем и отсутствием ожидания подтверждений. Для задач реального времени, где критичнее своевременность, чем гарантированная доставка каждого пакета, протокол UDP является естественным выбором. Даже при потере единичных пакетов общая динамика ориентации квадрокоптера не нарушается, что делает UDP оптимальным для передачи быстроменяющихся данных:

$$t_{delay,UDP} \leq t_{delay,TCP} \quad (1.14)$$

Таким образом, для передачи между полетным контроллером и ПК выбран протокол UDP по следующим причинам:

- минимальная задержка передачи (<5 мс в локальной сети),
- отсутствие необходимости подтверждения пакетов,
- устойчивость к временным потерям данных (поскольку каждый пакет содержит полный набор ориентационных параметров),
- высокая частота передачи (до 100–150 Гц),
- простота реализации на ESP32.

Для визуализации движения в Unreal Engine требуется передать три угла ориентации: крен φ , тангаж θ и рысканья ψ . Передача углов в формате углов Эйлера является удобной для приложения на стороне ПК, однако параметры могут также быть дополнены нормализованными компонентами кватерниона, если это потребуется в следующих версиях проекта.

В результате каждое UDP-сообщение содержит полную ориентацию и угловые скорости, что позволяет системе визуализации корректно отображать состояние квадрокоптера даже при потере отдельных сообщений.

Логическая схема передачи данных представлена на рисунке 1.19.



Рисунок 1.19 – Схема передачи данных от модуля ESP32 к персональному компьютеру

После считывания данных от датчика MPU-6050 и получения углов ориентации в виде Эйлеровых углов и угловых ускорений формируется строковый пакет. Отправка выполняется через стандартные библиотеки Wi-Fi и WiFiUDP для ESP32.

Передача выполняется с частотой около 50–100 Гц, что полностью соответствует динамике полёта квадрокоптера и возможностям программной визуализации.

На стороне ПК используется Python-скрипт, который: слушает указанный порт UDP, принимает поступающие сообщения, разбирает строку на компоненты и передаёт значения в Unreal Engine.

```

print(f" Ожидание данных от ESP32 ({ESP32_IP}:{PORT})...")
sock.sendto(b"START", (ESP32_IP, PORT))

def get_current_sensor_data():
    try:
        data, addr = sock.recvfrom(1024)
        msg = data.decode().strip()
        parts = msg.split(',')
        if len(parts) == 3:
            return float(parts[0]), float(parts[1]), float(parts[2])
    except:
        pass
    return None
  
```

Рисунок 1.20 – Пример кода приема данных с ESP32

В ходе разработки были выявлены несколько ключевых особенностей, влияющих на устойчивость передачи данных:

- Потеря отдельных UDP-пакетов – UDP не гарантирует доставку, однако, поскольку каждый пакет содержит полный набор ориентационных параметров, потеря одного сообщения не влияет на точность;

- Неравномерность интервалов отправки – в экспериментах наблюдались редкие пиковые задержки сети, однако использование фиксированного временного шага позволило избежать накопления ошибок;
- Замедления из-за DMP FIFO – DMP формирует данные с фиксированной частотой, поэтому важно корректно считывать FIFO-буфер, чтобы минимизировать переполнение;
- Сетевые особенности точки доступа – используемая мобильная точка доступа добавляет 1-3 мс задержки.

Кроме визуализации углового движения квадрокоптера, в среде Airsim также проводится моделирование его поступательного движения. Для этого требуется информация о силах, действующих на квадрокоптер. Как написано в п.1.2.3.1 на квадрокоптер действуют три силы:

- T – суммарная тяга от двигателей,
- mg – сила тяжести,
- $F_{\text{аэро}}$ – аэродинамическая сила.

На практике для рассматриваемого в дипломном проекте квадрокоптера можно считать, что сила тяжести постоянна и равна mg , где m – масса квадрокоптера (для используемого в проекте равна 0.5кг), $g=9.81$ м/с².

Т.к. аэродинамическая сила не измеряется бортовыми датчиками, данная сила моделируется в среде Airsim с использованием стандартной математической модели (1.6)

Сила тяги моторов так же не может быть измерена нашим бортовым оборудованием. Однако, в алгоритме управления проводится расчет требуемой тяги моторов, на основе которой формируются ШИМ-сигналы, необходимые для каждого мотора. Поскольку эти ШИМ-сигналы вычисляются с использованием довольно точной модели зависимости тяги моторов от ШИМ-сигнала, можно считать, что реальная тяга моторов приблизительно равна расчетной в полетном контроллере. Таким образом,

для моделирования поступательного движения квадрокоптера в Airsim можем использовать значения суммарной тяги всех моторов, переданные из ESP-32.

Помимо передачи данных от ESP32 на персональный компьютер, система реализует обратный канал связи для управления состоянием квадрокоптера. Этот двусторонний обмен позволяет организовать замкнутый контур управления, где ПК выступает в роли высокоуровневого контроллера, а ESP32 выполняет функции низкоуровневой стабилизации.

Для управления квадрокоптером с персонального компьютера реализован набор текстовых команд, передаваемых по UDP.

Команды управления состоянием системы:

- ARM – взвод моторов (перевод системы в рабочее состояние),
- DISARM – отключение моторов,
- RESET – сброс аварийного состояния.

Команды управления режимами передачи данных:

- START – начало непрерывной передачи телеметрии,
- STOP – остановка передачи телеметрии.

Команды задания параметров управления:

- SET <yaw> <pitch> <roll> – установка целевых углов ориентации.

Пример обработки команд на стороне ESP32 представлен на рисунке 1.21

```

if (cmd.equalsIgnoreCase("START")) {
    streamActive = true;
    Serial.println("СТРИМИНГ ТЕЛЕМЕТРИИ: ВКЛ");
    return;
}

if (cmd.equalsIgnoreCase("STOP")) {
    streamActive = false;
    Serial.println("СТРИМИНГ ТЕЛЕМЕТРИИ: ВЫКЛ");
    return;
}

if (cmd.equalsIgnoreCase("ARM")) {
    flag = false; // сброс аварийного флага
    armMotors();
    return;
}

if (cmd.equalsIgnoreCase("DISARM")) {
    flag = true; // установка флага выключения
    motorsArmed = false;
    Serial.println("МОТОРЫ ОТКЛЮЧЕНЫ");
    return;
}

if (cmd.equalsIgnoreCase("RESET")) {
    flag = false;
    Serial.println("Аварийная остановка сброшена");
    return;
}

// Команда SET от Python-программы: SET yaw pitch roll throttle
if (cmd.startsWith("SET")) {
    if (!motorsArmed) {
        Serial.println("ОШИБКА: Моторы не взведены! Отправьте команду ARM сначала.");
        return;
    }
}

```

Рисунок 1.21 – функция обработки входящих команд на ESP32

Команда SET является основной для управления ориентацией квадрокоптера. При её получении ESP32 выполняет следующую последовательность действий:

- Проверка состояния системы;
- Парсинг параметров – извлечение значений углов из текстовой строки;
- Валидация данных – проверка корректности формата и диапазонов значений;
- Установка целевых значений – присвоение значений переменных yawDesired, pitchDesired, rollDesired;

– Интеграция в контур управления – использование заданных значений в алгоритме ПИД-регулирования.

Двусторонняя связь ESP32-ПК обеспечивает не только управление, но и синхронизацию физического стенда с виртуальной моделью в Unreal Engine. Текущая ориентация, передаваемая от ESP32, используется для обновления положения виртуального квадрокоптера в реальном времени, визуализации реакции на управляющие воздействия, записи траекторий движений для последующего анализа.

1.2.6 Обработка данных ориентации на ПК и синхронизация движения стенда

Передача данных от MPU-6050 через ESP32 – лишь первый этап построения системы полунатурного моделирования. Главная задача – обеспечить корректное, устойчивое и минимально задержанное соответствие между движением реального стенда и виртуального квадрокоптера в среде Unreal Engine/AirSim. Для этого требуется разработать специальный программный модуль, который будет выполнять обработку поступающих данных, согласование координатных систем, сглаживание шумов и передачу изменённой ориентации в AirSim.

Этот этап является критическим для всей системы: именно он обеспечивает соединение между физическим устройством и виртуальной сценой. От качества реализации зависит точность визуализации, реалистичность динамики и возможность проведения экспериментов с алгоритмами управления.

Процесс синхронизации не тривиален по ряду причин:

- Различие координатных систем,
- Различие в частотах передачи и обработки,
- Потери UDP-пакетов,

- Несовпадение динамики реального стенда и виртуального объекта.

Различие координатных систем – система координат MPU-6050 ориентирована на корпус устройства – связанная система координат. Она формирует ориентацию относительно:

- оси X – крен,
- оси Y – тангаж,
- оси Z – курс.

В свою очередь, AirSim использует систему координат NED (North–East–Down), в которой:

- оси X – север,
- оси Y – восток,
- оси Z – вниз вдоль нормали.

для формирования скорости квадрокоптера относительно земли, и его координат.

При этом для удобства в среде виртуального моделирования ориентация задаётся не углами Эйлера, а кватернионами. Поэтому требуется преобразование форматов и согласование пространственных осей. В противном случае при одинаковых углах реальный и виртуальный квадрокоптер будут ориентированы по-разному.

Различие в частотах передачи и обработки – в нашей системе:

- ESP32 передаёт данные с частотой 50–100 Гц,
- AirSim обновляет состояние сцены с частотой кадров 30–120 FPS,
- Python-скрипт может обрабатывать данные с любой частотой, зависящей от загрузки CPU.

Несовпадение частот приводит к дерганию, если данные передаются нерегулярно или слишком быстро. Поэтому требуется механизм сглаживания.

Потери UDP-пакетов – UDP выбран в качестве основного протокола передачи (см. раздел 1.2.5) по двум причинам: минимальная задержка до 1–3

мс на локальной сети и простота реализации и отсутствие накладных расходов.

Однако UDP не гарантирует доставку пакетов и их порядок. Поэтому в принимающей программе на ПК необходимо предусмотреть фильтрацию выбросов, защиту от резких скачков углов и восстановление плавности движения.

Несовпадение динамики реального стенда и виртуального объекта – Airsim обладает собственной физической моделью квадрокоптера. Если накладывать внешние углы ориентации поверх внутренней физики, возникает конфликт: AirSim пытается стабилизировать квадрокоптер самостоятельно; пользовательская программа одновременно насильно меняет ориентацию.

Поэтому для корректной синхронизации квадрокоптер должен удерживаться в фиксированной позиции.

Использование AirSim API для внешнего управления ориентацией – AirSim официально поддерживает несколько режимов управления:

- управление моторами,
- управление положением,
- управление позой.

Наилучший режим – прямое управление позой. Он позволяет полностью отключить внутренний контроллер и использовать ориентацию, приходящую с реального стенда. Это превращает AirSim фактически в визуализатор движения MPU-6050. В таблице 1.6 представлены особенности работы Airsim.

Таблица 1.6 – Особенности использования Airsim API

Критерий	Требование системы	AirSim решение
Реальное полунатурное моделирование	Требуется точная ориентация от стенда	SimSetVehiclePose() полностью перезаписывает ориентацию
Высокая точность	Отсутствия моторной динамики	Работа с кватернионами без ошибки накопления

Продолжение таблицы 1.6

Интеграция с Unreal Engine	Требуется сцена с 3D-визуализацией	AirSim является UE-плагином
Минимальная задержка	Связи ESP32 – ПК – UE	AirSim API работает в реальном времени
Гибкость	Сложность разработки собственной визуализации	AirSim обеспечивает готовый движок

Класс `AirSimController` инкапсулирует всю логику взаимодействия с симулятором. При подключении к AirSim выполняется инициализация API, подтверждение соединения и активация внешнего управления. Для управления движением квадрокоптера используется метод `moveByVelocityZAsync`, который позволяет задавать скорости перемещения по осям X и Y (север и восток в системе NED) при поддержании фиксированной высоты (рисунок 1.22). Этот подход обеспечивает естественную динамику движения, соответствующую реальному поведению квадрокоптера.

```
def move_by_velocity(self, velocity_x: float, velocity_y: float,
                    yaw: float = 0) -> bool:
    """Движение дрона с заданной скоростью"""
    if not self.connected or not self.in_air:
        return False

    try:
        self.client.moveByVelocityZAsync(
            velocity_x,
            velocity_y,
            Config.TARGET_ALTITUDE,
            0.1,
            airsims.DrivetrainType.MaxDegreeOfFreedom,
            airsims.YawMode(False, yaw)
        )
        return True
    except:
        return False
```

Рисунок 1.22 – Пример кода с использованием метода `moveByVelocityZAsync`

Важной особенностью разработанной системы является обработка преобразования координатных систем и формирование линейных скоростей квадрокоптера. Управляющие воздействия, вычисленные на основе относительных углов наклона сенда, интерпретируются как команды на

изменение скоростей в связанной системе координат (ССК), жёстко связанной с корпусом квадрокоптера. В данной системе координат ось OX_b направлена вперёд по продольной оси аппарата, ось OY_b – вправо, а ось OZ_b – вниз.

Формирование линейных скоростей осуществляется на основе уравнений движения центра масс квадрокоптера (1.7).

Полученные локальные скорости далее преобразуются в глобальную навигационную систему координат NED с учётом текущего угловой ориентации. Для этого используется матрица поворота между ССК и NED (рисунок 1.23).

$$\begin{pmatrix} V_{xg} \\ V_{yg} \\ V_{zg} \end{pmatrix} = \begin{pmatrix} \cos \theta \cos \varphi & \sin \theta \cos \varphi & -\sin \psi \cos \theta \\ \sin \psi \sin \varphi - \cos \psi \sin \theta \cos \varphi & \cos \psi \sin \varphi + \sin \psi \sin \theta \cos \varphi & \cos \theta \cos \varphi \\ \sin \psi \cos \varphi + \cos \psi \sin \theta \sin \varphi & -\cos \psi \cos \varphi - \sin \psi \sin \theta \sin \varphi & \cos \theta \sin \varphi \end{pmatrix} \cdot \begin{pmatrix} V_{xb} \\ V_{yb} \\ V_{zb} \end{pmatrix}$$

Рисунок 1.23 – Преобразование скорости между ССК и NED

Данное преобразование обеспечивает корректное направление движения квадрокоптера в глобальной системе координат независимо от его ориентации в пространстве и используется при формировании команд управления в виртуальной среде AirSim.

Для обеспечения плавной визуализации реализована сложная система управления временными характеристиками. Основной цикл обработки данных работает с частотой около 20 Гц, что является компромиссом между скоростью реакции и нагрузкой на процессор. При этом частота обновления телеметрической информации на экране ограничена 5 Гц для предотвращения мелькания и улучшения читаемости.

Механизм интерполяции и прогнозирования используется для компенсации разницы в частотах обновления. При пропуске UDP-пакетов система продолжает использовать последние корректные данные, применяя к

ним экспоненциальное затухание для имитации инерционности. Для поддержания активности соединения с AirSim реализован механизм keep-alive, который периодически отправляет нулевые управляющие воздействия, предотвращая автоматическую посадку квадрокоптера по таймауту.

Для удобства работы с системой разработан интерактивный интерфейс командной строки с поддержкой горячих клавиш. Класс MenuManager реализует управление различными режимами работы, включая калибровку, настройку камеры, управление моторами и полетные режимы (рисунок 1.24). Интерфейс обеспечивает немедленную реакцию на нажатия клавиш без необходимости подтверждения через Enter, что критически важно для оперативного управления.

```
class MenuManager:
    """Класс для управления меню и вводом с клавиатуры"""

    def __init__(self):
        self.current_mode = 1 # 1 - главное меню, 2 - меню камеры
        self.program_running = True
        self.telemetry_enabled = False
        self.print_telemetry_flag = False
        self.menu_shown = False
        self.camera_menu_shown = False
        self.show_menu_after_command = False
        self.telemetry_line_shown = False
```

Рисунок 1.24 – Пример кода реализации меню программы

Реализовано несколько режимов отображения информации: компактная строка телеметрии, показывающая текущие углы и скорости в реальном времени; подробное меню с настройками; и специализированный интерфейс управления камерой. Система логирования записывает все телеметрические данные в файл с временными метками, что позволяет проводить последующий анализ работы алгоритмов.

Проведенные измерения показали, что общая задержка в цепи полунатурный стенд – ESP32 – ПК – AirSim составляет менее 50 мс в нормальных условиях работы. Наибольший вклад в задержку вносит сетевое взаимодействие (10-20 мс) и обработка графики в Unreal Engine (15-30 мс).

Python-обработка добавляет не более 5-10 мс благодаря оптимизированным алгоритмам и эффективному использованию буферизации.

Разработанная система демонстрирует стабильную работу при потерях UDP-пакетов до 20. Частота обновления визуализации поддерживается на уровне не менее 30 FPS даже на среднепроизводительном оборудовании, что обеспечивает плавное и реалистичное отображение движений.

Реализованная система обработки данных ориентации и синхронизации с AirSim представляет собой комплексное решение, учитывающее все аспекты полунатурного моделирования. Архитектура, основанная на разделении ответственности между специализированными классами, обеспечивает гибкость, надежность и возможность дальнейшего расширения. Интеграция с AirSim API через управление скоростью, а не напрямую позой, позволила достичь естественной динамики движений при сохранении точности отображения ориентации.

Система успешно решает задачи преобразования координатных систем, компенсации временных задержек, фильтрации шумов и обеспечения устойчивости к сетевым сбоям. Разработанное решение не только обеспечивает высококачественную визуализацию движений физического стенда, но и создает основу для проведения экспериментов с алгоритмами управления в условиях, максимально приближенных к реальным.

1.3 Разработка программы визуального моделирования движения квадрокоптера в среде Unreal Engine

1.3.1 Подготовка среды моделирования

Unreal Engine версии 4.27 представляет собой современный и высокопроизводительный игровой движок, широко применяемый не только в индустрии компьютерных игр, но и в задачах имитационного моделирования, инженерных симуляций, архитектурной визуализации и разработки интерактивных систем. В рамках данного дипломного проекта Unreal Engine используется в сочетании с симуляционной платформой AirSim для реализации виртуальной среды и визуального моделирования движения квадрокоптера [16].

Для организации рабочего окружения первоначально была выполнена установка среды Unreal Engine 4.27 с использованием приложения Epic Games Launcher, которое является официальным инструментом для управления продуктами Epic Games. Начальный интерфейс среды разработки Unreal Engine представлен на рисунке 1.25.

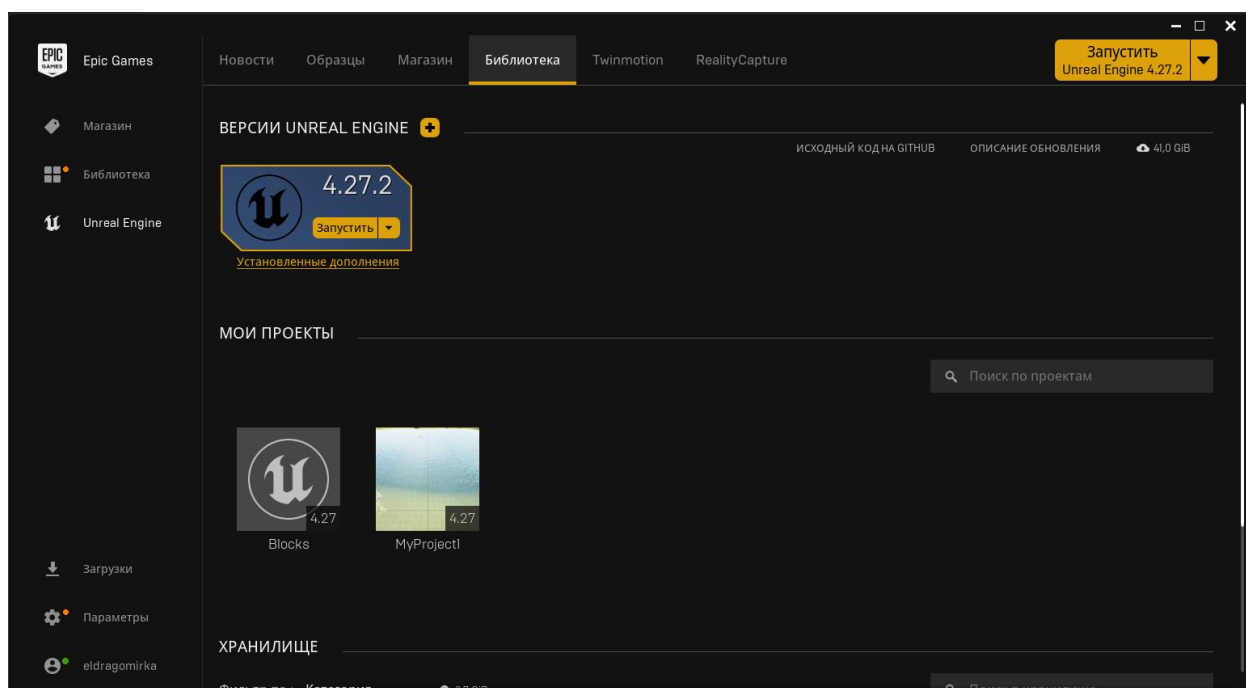


Рисунок 1.25 – Начальная среда работы с Unreal Engine

После завершения установки был создан новый проект в Unreal Engine, ориентированный на задачи моделирования и визуализации. При создании проекта использовался стандартный шаблон, обеспечивающий корректную работу физического движка, системы освещения и компонентов сцены. Процесс создания проекта в Unreal Engine 4.27 представлен на рисунке 1.26.

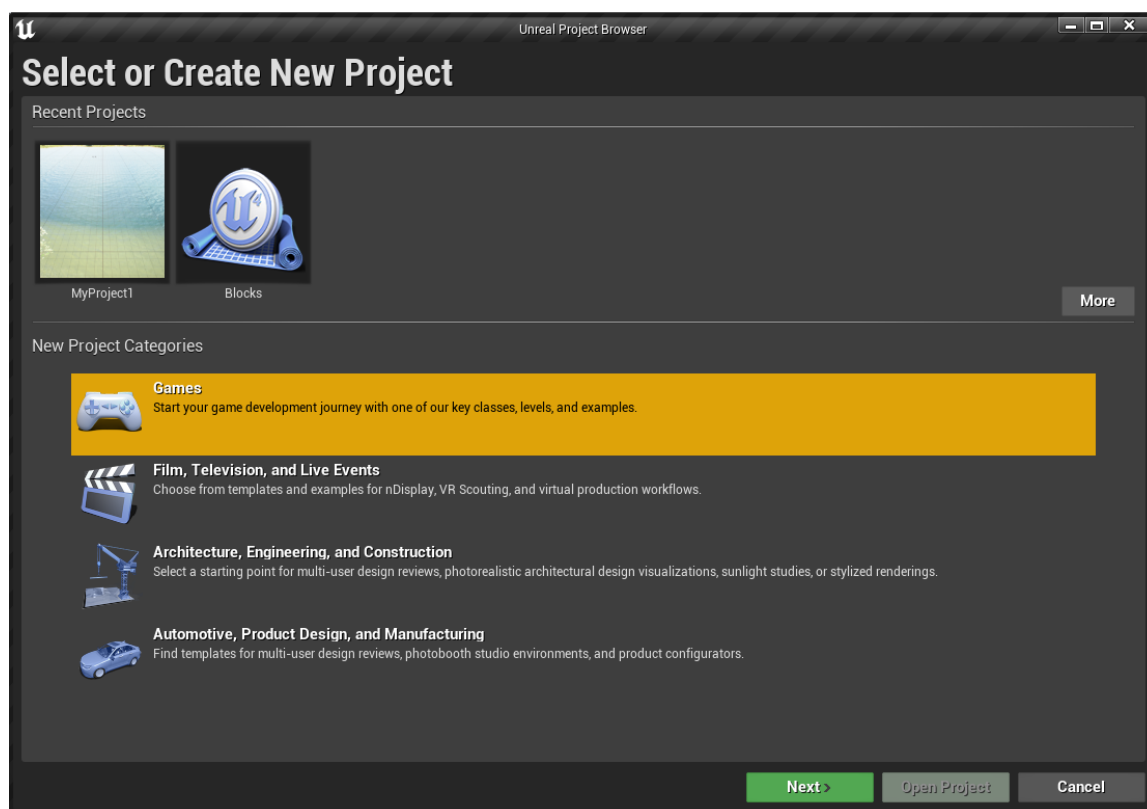


Рисунок 1.26 – Создание проекта на Unreal Engine 4.27

Сформированный проект в среде Unreal Engine 4.27 показан на рисунке 1.27 и представляет собой базовую сцену, служащую основой для последующей интеграции симуляционной платформы AirSim.

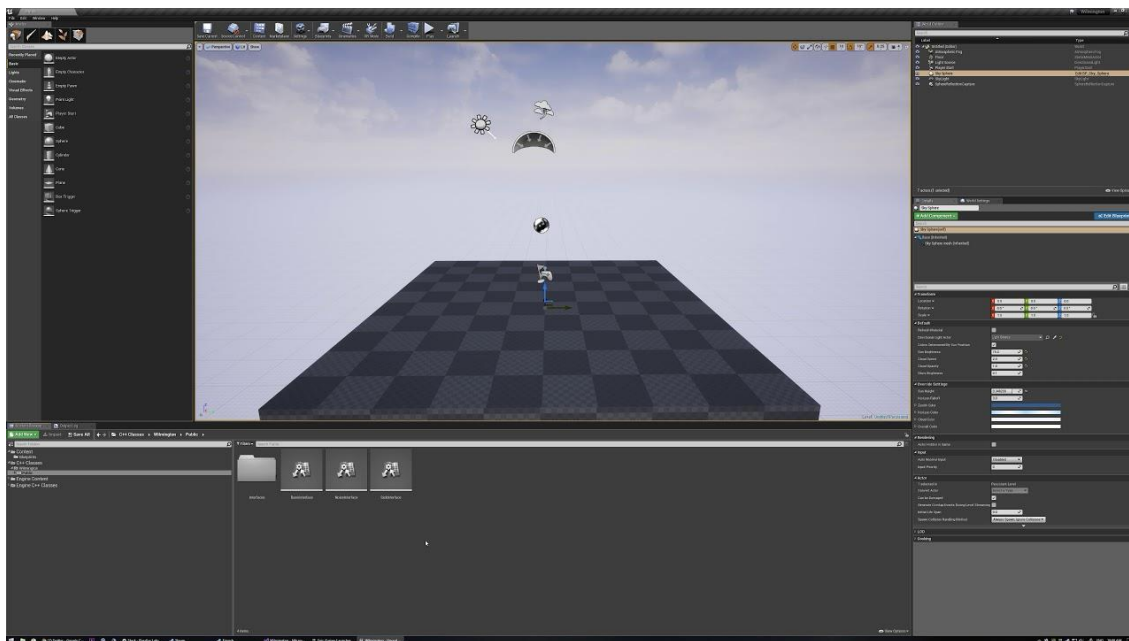


Рисунок 1.27 – Базовый проект в Unreal Engine 4.27

Для реализации симуляции полёта квадрокоптера была использована платформа AirSim – открытая среда моделирования автономных транспортных средств, разработанная корпорацией Microsoft. AirSim предоставляет готовую физическую модель летательных аппаратов, реалистичную симуляцию датчиков и развитый программный интерфейс управления. Установка AirSim осуществлялась на основе официальной документации проекта, после чего был выполнен перенос компонентов AirSim в структуру проекта Unreal Engine путём подключения соответствующего плагина [9].

Интеграция AirSim в проект Unreal Engine включает модификацию конфигурационных файлов проекта и использование исходного кода на языке C++. При сборке проекта использовалась среда разработки Microsoft Visual Studio, поддерживаемая выбранной версией AirSim. Корректность подключения платформы подтверждается наличием зависимостей AirSim в конфигурации проекта, пример которой приведён на рисунке 1.28.



Рисунок 1.28 – Конфигурация проекта Unreal Engine с подключённым AirSim

После успешной интеграции AirSim была выполнена настройка параметров симуляции, определяющих характеристики виртуального квадрокоптера и его сенсорных систем. Основные параметры задаются в конфигурационном файле `settings.json`, в котором описываются тип летательного аппарата, параметры инерциальных датчиков, камеры, а также сетевые настройки взаимодействия с внешними приложениями. Пример конфигурационного файла приведён на рисунке 1.29.

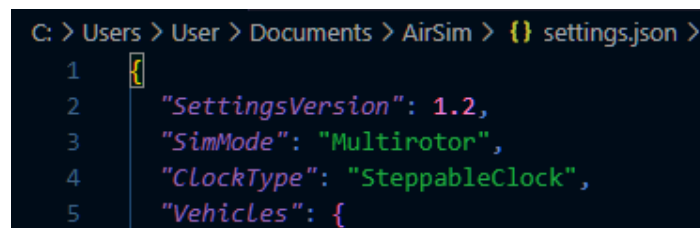


Рисунок 1.29 – Конфигурационный файл `settings.json`

Завершающим этапом настройки среды стало тестирование корректности работы симуляции. В ходе тестирования была проверена загрузка сцены, инициализация квадрокоптера, корректность физической модели и возможность управления летательным аппаратом. Пример тестирования симуляции AirSim в среде Unreal Engine представлен на рисунке 1.30.

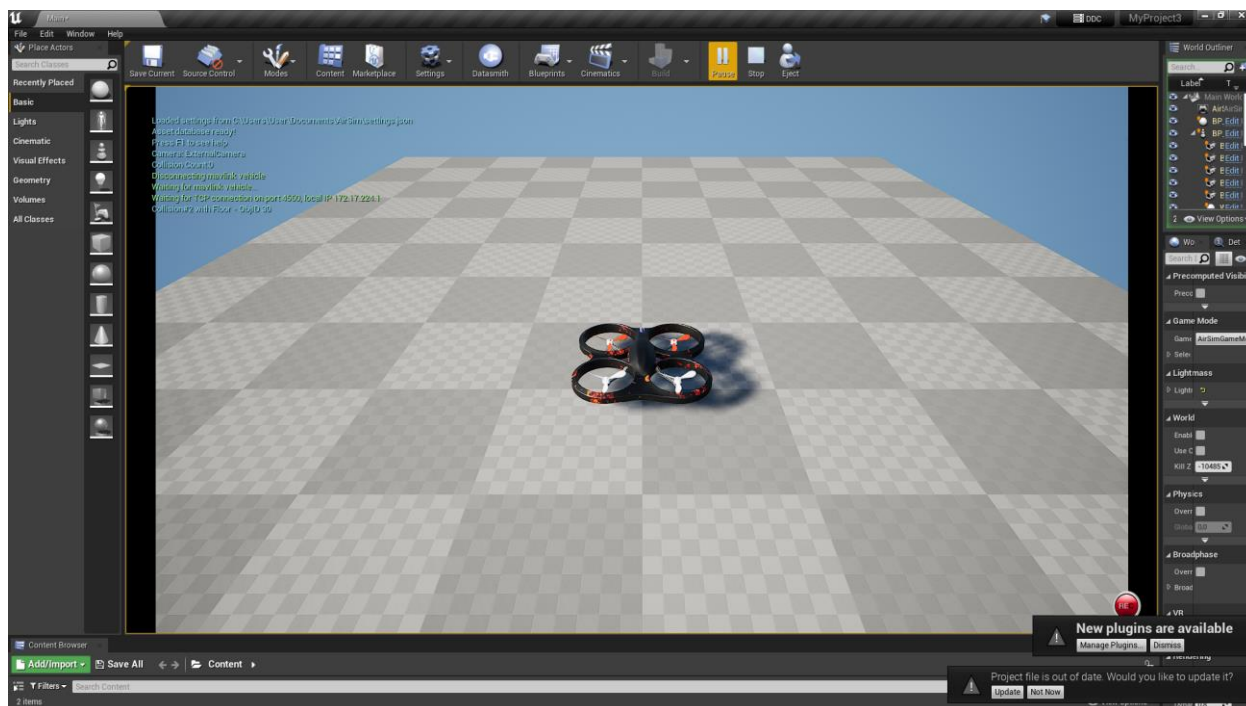


Рисунок 1.30 – Тестирование AirSim

После проверки корректности запуска симуляции и инициализации виртуального квадрокоптера основное тестирование системы управления проводилось на основе разработанного полунатурного стенда моделирования, оснащённого инерциальным измерительным модулем MPU-6050.

В рамках данного подхода управление виртуальным квадрокоптером в среде AirSim осуществляется непосредственно за счёт физических угловых перемещений стенда, измеряемых датчиком MPU-6050 и передаваемых через микроконтроллер ESP32 на персональный компьютер. Таким образом, виртуальный квадрокоптер в Unreal Engine повторяет ориентацию и

движение реального объекта, закреплённого на кардановом подвесе, что обеспечивает эффект полунатурного моделирования.

Программное взаимодействие с симулятором AirSim реализовано посредством пользовательского программного модуля управления, обеспечивающего подключение к API AirSim, инициализацию виртуального квадрокоптера и передачу управляющих воздействий, сформированных на основе данных ИМИ MPU-6050. Общая структура данного модуля представлена классом AirSimController, отвечающим за установление соединения, взлёт, посадку, управление скоростями и ориентацией камеры.

Фрагмент программной реализации подключения к AirSim и инициализации управления представлен на рисунке 1.31.

```
def connect(self) -> bool:
    """Подключение к AirSim"""
    try:
        print("🔌 Подключение к AirSim...")
        self.client = airsims.MultirotorClient()
        self.client.confirmConnection()
        self.client.enableApiControl(True)
        self.client.armDisarm(False)
        self.connected = True
        print("✅ AirSim подключен")
        return True
```

Рисунок 1.31 – Пример кода подключения Airsim

Данный код обеспечивает установление соединения с симулятором, активацию программного управления и подготовку виртуального квадрокоптера к выполнению команд, поступающих извне.

Движение квадрокоптера в симуляции осуществляется как сочетание углового движения по данным, переданным от бортового оборудования, и поступательного движения, моделируемого в Airsim по информации о суммарной тяге моторов, с использованием принятой математической модели аэродинамики пропеллеров квадрокоптера. Таким образом, для визуализации движения квадрокоптера требуются:

- значения угловых скоростей,
- значения углов ориентации,
- значения линейных скоростей,
- значения координатах в системе NED.

Передача таких данных реализуется с использованием функции `Airsim.moveByVelocityZAsync`.

Такой подход позволяет связать углы крена, тангажа и курса, измеряемые на стенде, с поступательным движением виртуального квадрокоптера, обеспечивая интуитивное и устойчивое управление.

Обработка данных, поступающих с инерциального измерительного модуля MPU-6050, осуществляется в два этапа. На бортовом микроконтроллере выполняется предварительная обработка измерений, включающая калибровку и подавление шумов, в результате чего формируются оценки углов ориентации квадрокоптера.

На стороне ПК данные используются исключительно в целях взаимодействия с симуляционной средой AirSim. Данный этап включает согласование формата представления данных, приведение единиц измерения и масштабирование значений углов ориентации в соответствии с требованиями программного интерфейса AirSim и движка Unreal Engine, что иллюстрируется следующим фрагментом на рисунке 1.32.

```

def process_sensor_data(self, raw_yaw: float, raw_pitch: float,
                        raw_roll: float) -> SensorData:
    """Обработка сырых данных с датчиков"""
    # Сглаживание данных
    yaw = 0.8 * self.prev_yaw + 0.2 * raw_yaw
    pitch = 0.8 * self.prev_pitch + 0.2 * raw_pitch
    roll = 0.8 * self.prev_roll + 0.2 * raw_roll

    self.prev_yaw, self.prev_pitch, self.prev_roll = yaw, pitch, roll

    return SensorData(
        yaw=yaw,
        pitch=pitch,
        roll=roll,
        timestamp=time.time()
    )

```

Рисунок 1.32 – Пример обработки данных с датчика

После масштабирования и учёта «мёртвой зоны» углы ориентации преобразуются в локальные скорости движения, которые затем пересчитываются в глобальную систему координат с учётом текущего угла курса виртуального квадрокоптера. Это обеспечивает корректное соответствие между физическим положением стенда и движением модели в виртуальной среде.

Дополнительно реализована возможность управления ориентацией виртуальных камер, закреплённых на квадрокоптере. Поворот камер по углам тангажа и курса осуществляется программно через API AirSim, что позволяет имитировать работу бортовой системы наблюдения и использовать получаемые изображения для дальнейших задач анализа и распознавания. Описание такой возможности приведено в п.1.3.3.

Применение описанного подхода позволило реализовать замкнутую систему «реальный стенд – датчик – виртуальная модель», в которой все движения в среде Unreal Engine формируются на основе реальных измерений ориентации. Это обеспечивает более высокую достоверность моделирования по сравнению с классическим управлением с клавиатуры или геймпада и

создаёт основу для дальнейшего расширения системы, включая задачи автономного управления и обработки визуальной информации.

1.3.2 Настройка виртуального окружения и карты в Unreal Engine

Для обеспечения достоверного и наглядного моделирования движения квадрокоптера в виртуальной среде важнейшим этапом является корректная настройка окружения, в котором осуществляется симуляция. Среда моделирования в Unreal Engine должна не только визуально соответствовать реальным условиям эксплуатации квадрокоптера, но и обеспечивать корректную работу физической модели, сенсоров и алгоритмов управления, реализованных в AirSim.

Unreal Engine предоставляет развитые средства для создания трёхмерных сцен, включающих сложные ландшафты, архитектурные объекты, источники освещения и динамические элементы. В сочетании с платформой AirSim это позволяет формировать высокореалистичную симуляционную среду, предназначенную для тестирования алгоритмов навигации, управления и детекции препятствий квадрокоптеров.

В рамках данной работы для построения виртуального окружения была использована готовая карта CityParkEnvironmentCollection, полученная через встроенный маркетплейс Epic Games Launcher. Использование готовых карт обусловлено рядом причин:

- значительное сокращение времени разработки сцены,
- наличие проработанных городских и природных элементов,
- корректная оптимизация геометрии и материалов,
- совместимость с версией Unreal Engine 4.27,
- возможность интеграции с AirSim без необходимости глубокой модификации исходных ассетов.

Выбранная карта представляет собой городскую парковую среду, включающую здания, дорожную инфраструктуру, зелёные насаждения и элементы городской среды. Такое окружение хорошо соответствует условиям маловысотных полётов квадрокоптеров в городской и пригородной застройке, что делает его подходящим для задач, решаемых в данной работе. Используемая карта проекта представлена на рисунке 1.33.



Рисунок 1.33 – Карта проекта Airsim

После добавления карты в проект Unreal Engine она была назначена в качестве основной сцены моделирования и интегрирована с AirSim. Для корректной работы симуляции было необходимо обеспечить соответствие координатной системы карты системе координат, используемой AirSim, а также корректное позиционирование стартовой точки квадрокоптера.

AirSim использует Unreal Engine в качестве визуального и физического ядра, однако накладывает определённые требования на структуру сцены. В частности, необходимо наличие корректно настроенного объекта PlayerStart, который определяет начальное положение виртуального квадрокоптера, а

также отсутствие конфликтов между встроенной физикой Unreal Engine и физической моделью AirSim.

В процессе интеграции карты CityParkEnvironmentCollection были выполнены следующие действия:

- размещение стартовой позиции квадрокоптера в безопасной зоне, свободной от препятствий,
- настройка масштаба сцены таким образом, чтобы один метр в Unreal Engine соответствовал одному метру в модели AirSim,
- проверка коллизий объектов карты с целью исключения некорректных столкновений,
- настройка освещения сцены для обеспечения стабильной визуализации данных с виртуальных камер.

Корректная настройка этих параметров позволила обеспечить стабильную работу симуляции и точное соответствие движения виртуального квадрокоптера управляющим воздействиям, формируемым на основе данных с полунатурного стенда.

Моделирование сценариев полёта для квадрокоптера является важным этапом в разработке и тестировании алгоритмов автономного управления и детекции препятствий. В рамках данной работы виртуальная среда использовалась не только для визуализации движения квадрокоптера, но и для формирования условий, приближенных к реальным эксплуатационным ситуациям.

Для моделирования реалистичных сценариев полёта в симуляционной среде были определены и размещены различные объекты препятствий. В программе использовались следующие типы препятствий:

- Крыши зданий, опоры линий электропередач и фонари были размещены на различных высотах и расстояниях друг от друга для создания разнообразных полетных маршрутов. Крыши находились на старте нашего полета и в конце траектории;

- Верхушки деревьев были распределены случайным образом по всей территории симуляции, создавая как редкие, так и густые лесные участки;
- Птицы были запрограммированы на случайное движение в пределах заданной области, что добавляет элементы неожиданности и сложности в процессе навигации;
- Линии электропередач были натянуты между опорами, создавая трудные для обнаружения препятствия, которые требуют высокой точности детекции;
- Мосты были смоделированы с различными уровнями сложности, включая как низкие, так и высокие структуры. Это позволило протестировать квадрокоптер в условиях, где требуются маневры под мостом или облет конструкции.

На рисунках 1.34-1.39 представлены объекты препятствий, размещенных в среде моделирования.

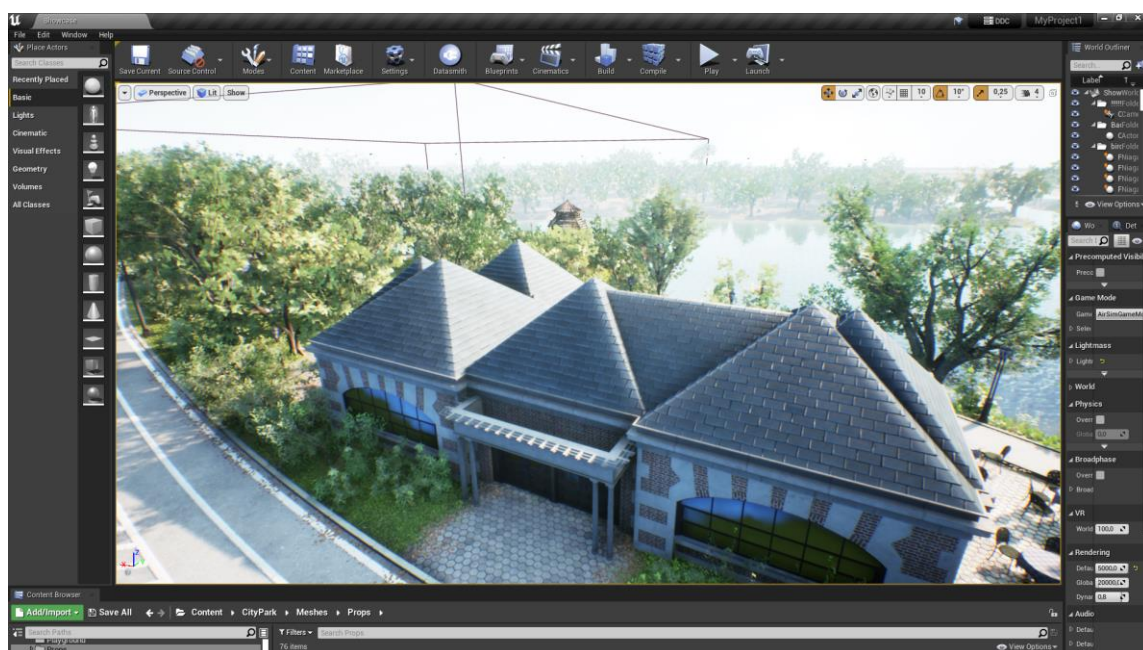


Рисунок 1.34 – Объект препятствия – крыша

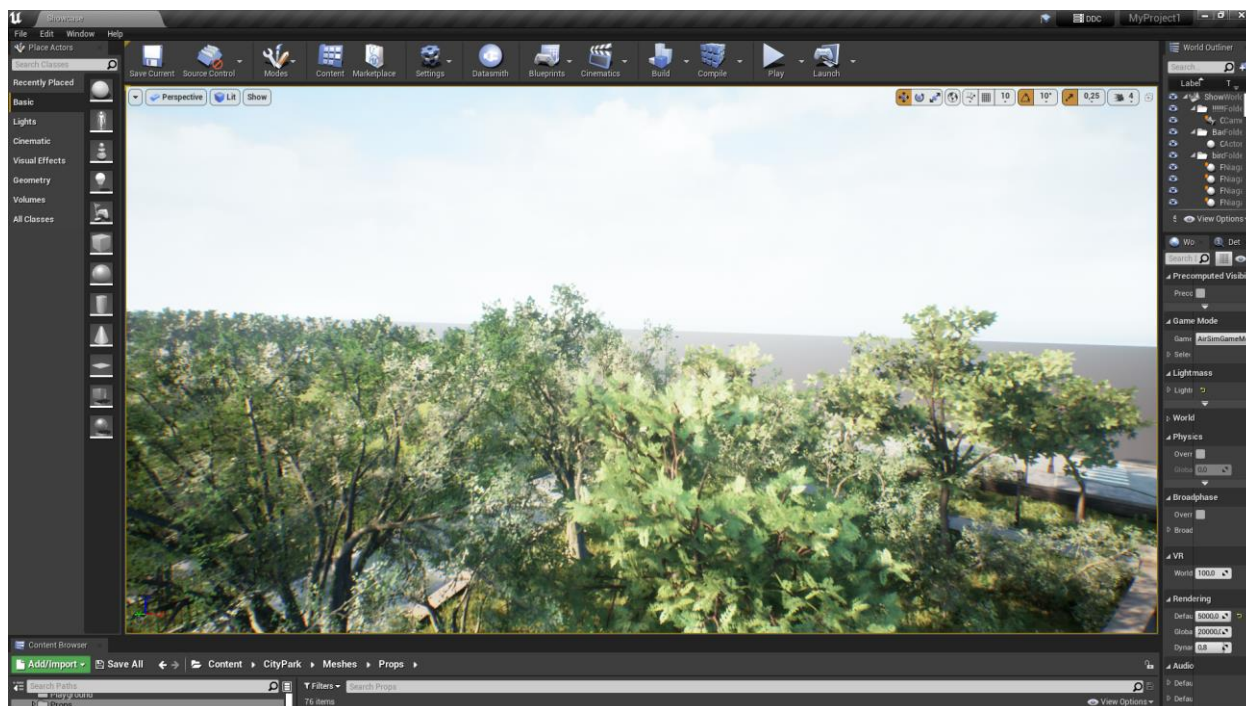


Рисунок 1.35 – Объект препятствия – верхушки деревьев



Рисунок 1.36 – Объекты препятствия – линии электропередач и опоры линий электропередач



Рисунок 1.37 – Объект препятствия – фонарный столб



Рисунок 1.38 – Объект препятствия – птица



Рисунок 1.39 – Объект препятствия – мост

Благодаря использованию Unreal Engine и AirSim достигается высокая степень реализма моделирования, включающая корректную геометрию сцены, освещение и взаимодействие объектов. Это позволяет рассматривать созданную среду как полноценный цифровой полигон для исследований и экспериментов.

После завершения настройки карты и размещения всех объектов были проведены тестовые полёты виртуального квадрокоптера. В ходе испытаний отрабатывались следующие сценарии:

- взлёт и стабилизация на заданной высоте,
- полёт на малой высоте над рельефом,
- маневрирование вблизи статических препятствий,
- прохождение сложных участков с высокой плотностью объектов,
- посадка в ограниченном пространстве.

Результаты тестирования показали, что выбранная карта и выполненная настройка среды обеспечивают устойчивую работу симуляции и адекватное отображение движения квадрокоптера, синхронизированного с полунатурным стендом. Это позволило перейти к дальнейшим этапам

работы, связанным с управлением виртуальными камерами и реализацией алгоритмов анализа визуальной информации.

1.3.3 Управление бортовыми камерами и формирование визуальных данных

1.3.3.1 Конфигурация камер AirSim

Одним из ключевых элементов системы автономного управления квадрокоптером является формирование визуальной информации, используемой для анализа окружающей среды и детекции препятствий. В условиях городской застройки и полетов на малых высотах визуальные данные играют решающую роль, так как позволяют получать информацию о пространственном расположении объектов, их форме, размерах и относительном движении.

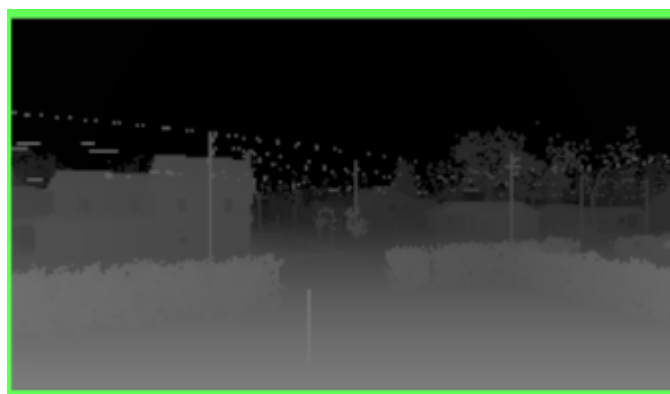
В рамках данной работы формирование визуальных данных реализовано с использованием бортовых камер квадрокоптера, моделируемых в среде Unreal Engine с применением симулятора AirSim. AirSim предоставляет развитый инструментарий для работы с виртуальными камерами, включая поддержку различных режимов съемки, настройку параметров изображения и программное управление ориентацией камер.

Для решения задач навигации и последующей детекции препятствий в AirSim были использованы несколько типов камер, каждая из которых выполняет собственную функциональную роль:

- камера глубины – используется для получения информации о расстоянии до объектов в поле зрения, что особенно важно при полетах на малых высотах и при маневрировании в ограниченном пространстве;
- камера сегментации – формирует изображение, в котором каждому объекту сцены присваивается уникальная цветовая метка, что значительно упрощает процесс формирования обучающего датасета;

– FPV-камера – обеспечивает изображение, близкое к реальному видеопотоку с бортовой камеры квадрокоптера, и используется для записи видеоматериалов и визуального анализа полета.

На рисунке 1.40 представлены примеры изображений, получаемых с указанных камер.



а)



б)



в)

Рисунок 1.40 – Виды с камеры: а – вид с камеры глубины; б – вид с камеры сегментации; в – вид с камеры FPV

Последняя из перечисленных камер применялась для записи видеопоследовательностей, используемых на этапах анализа и обучения нейросетевых моделей. Переключение между режимами камер в процессе моделирования осуществлялось с помощью стандартных сочетаний клавиш F1–F3, что позволяло оперативно изменять тип визуализации без перезапуска симуляции.

Дополнительно AirSim предоставляет несколько режимов наблюдения за сценой, таких как FPV, Fly with me и Ground View, что позволяет контролировать полет как с точки зрения бортовой камеры, так и со стороны внешнего наблюдателя.

Параметры работы камер, включая разрешение, угол обзора, частоту обновления и тип визуализации, задаются в конфигурационном файле settings.json. В рамках данной работы использовались базовые параметры, рекомендованные разработчиками AirSim, что позволило обеспечить баланс между качеством визуальных данных и производительностью вычислительной системы.

Проведенный анализ показал, что включение даже одной камеры оказывает заметное влияние на быстродействие симуляции. На рисунке 1.41 представлена зависимость количества кадров в секунду (FPS) от числа активных камер. Как видно из графика, активация одной камеры приводит к снижению производительности примерно на 15 FPS.

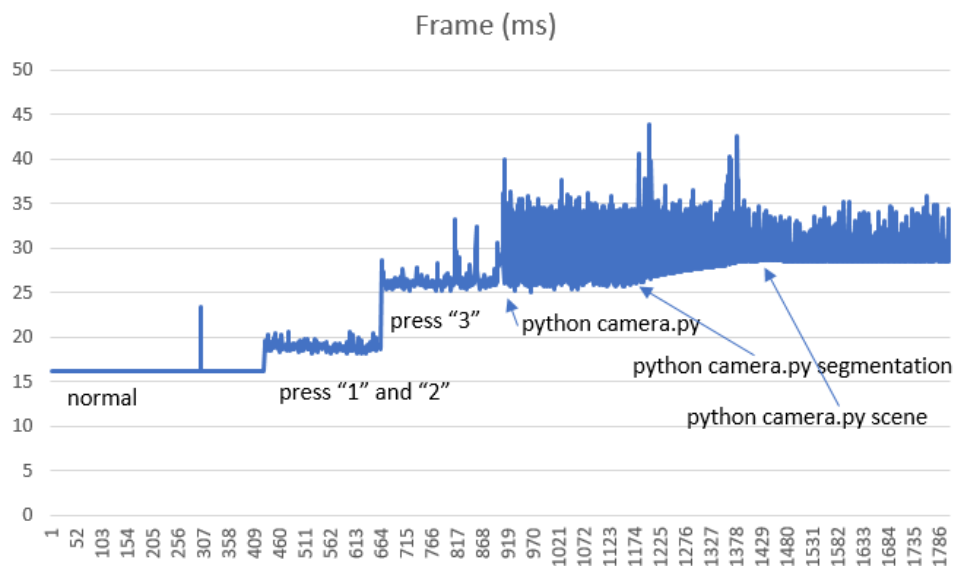


Рисунок 1.41 – График влияния включенных камер на FPS

Дополнительно была исследована нагрузка на центральный процессор, возникающая при работе камер. Соответствующая зависимость представлена на рисунке 1.42. Рост нагрузки CPU обусловлен необходимостью параллельной обработки графических данных, симуляции сенсоров и передачи изображений через API AirSim.

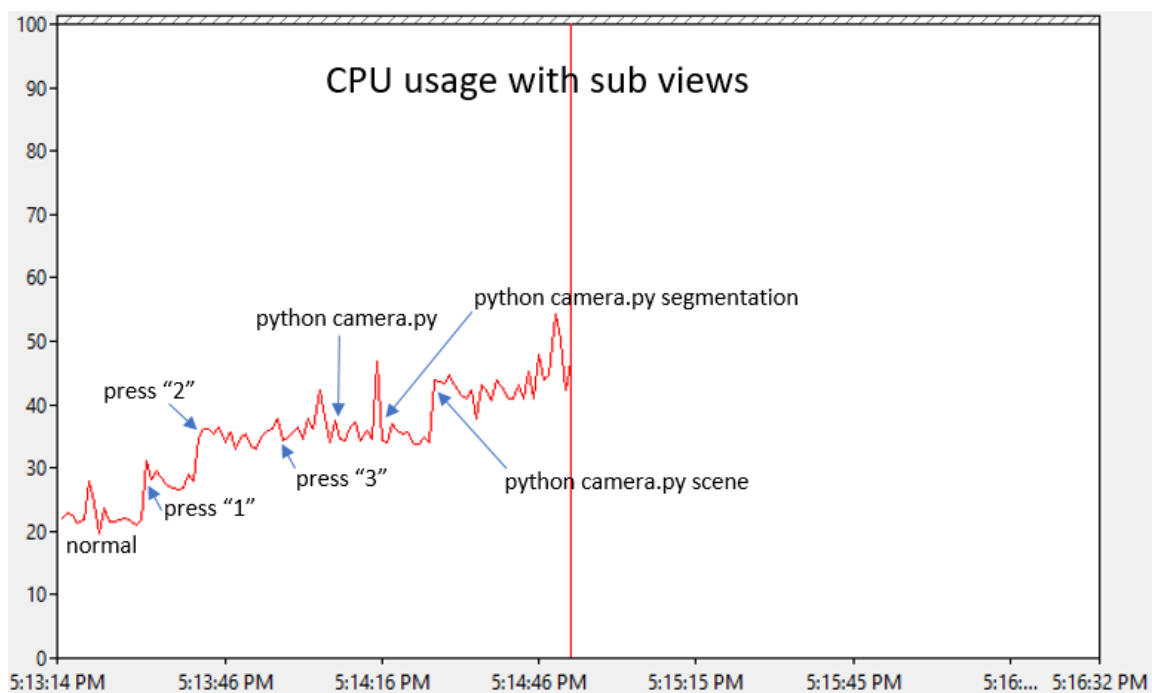


Рисунок 1.42 – График зависимости нагрузки CPU от включенных камер

С учетом характеристик вычислительной системы, на которой проводилось моделирование, для обеспечения стабильной работы симуляции было принято решение использовать одну активную камеру, с которой осуществлялась запись визуальных данных. Такой подход позволил избежать падения производительности и обеспечить воспроизводимость экспериментов.

Для повышения гибкости системы и расширения возможностей формирования датасета в работе было реализовано программное управление ориентацией бортовой камеры. Изменение углов визирования камеры позволяет получать изображения одной и той же сцены под различными ракурсами, что положительно сказывается на обобщающей способности нейросетевых моделей.

Управление положением камеры осуществлялось путем изменения углов тангажа и рыскания в реальном времени с использованием клавиатурного ввода. Фрагмент программного кода, реализующего обработку команд управления камерой, приведен ниже на рисунке 1.43.

```
def _handle_camera_menu_key(self, key: str):
    """Обработка клавиш меню камеры"""
    step = 5
    changed = False

    if key == 'up':
        self.airsim.camera_pitch += step
        changed = True
    elif key == 'down':
        self.airsim.camera_pitch -= step
        changed = True
    elif key == 'left':
        self.airsim.camera_yaw -= step
        changed = True
    elif key == 'right':
        self.airsim.camera_yaw += step
        changed = True
    elif key == 'b':
        self.menu.current_mode = 1
        self.menu.menu_shown = False
        self.menu.camera_menu_shown = False
        self.menu.print_telemetry_flag = self.menu.telemetry_enabled
        print("\n Возврат в главное меню")
        return

    if changed:
        if self.airsim.set_camera_orientation(self.airsim.camera_pitch,
                                              self.airsim.camera_yaw):
            print(f"\n Камера: Pitch={self.airsim.camera_pitch:3}°, "
                  f"Yaw={self.airsim.camera_yaw:3}°", end='', flush=True)
```

Рисунок 1.43 – Пример обработки команд управления камерой

Также был реализован отдельный режим настройки камеры, обеспечивающий интуитивно понятное взаимодействие оператора с системой. Реализация показана на рисунке 1.44.

```
def _handle_camera_mode(self):
    self.menu.current_mode = 2
    self.menu.menu_shown = False
    self.menu.camera_menu_shown = False
    self.menu.clear_telemetry_line()
    print("\n 📷 РЕЖИМ НАСТРОЙКИ КАМЕРЫ")
    print(" ↑/↓ – наклон камеры (вверх/вниз)")
    print(" ←/→ – поворот камеры (влево/вправо)")
    print(" [b] – возврат в меню")
    print(f"\r 📷 Камера: Pitch={self.airsim.camera_pitch:3}°, "
          f"Yaw={self.airsim.camera_yaw:3}°", end='', flush=True)
```

Рисунок 1.44 – Пример меню режима настройки камеры

Данный механизм позволил оперативно изменять ориентацию камеры непосредственно в процессе полета, что особенно важно при моделировании различных сценариев обнаружения препятствий.

Использование различных типов камер и возможность изменения их ориентации позволили сформировать разнообразный набор визуальных данных, включающий сцены с различными препятствиями, углами обзора и условиями полета. Полученные изображения использовались для последующего анализа и обучения нейросетевых алгоритмов детекции препятствий.

Таким образом, реализованная система управления бортовыми камерами в среде Unreal Engine и AirSim обеспечивает гибкое формирование визуальных данных, необходимых для разработки и тестирования алгоритмов автономной навигации и компьютерного зрения, что создает основу для дальнейшего рассмотрения методов детекции и обучения нейросетевых моделей в следующих подразделах.

1.3.3.2 Формирование датасета и обучение нейронной сети

Для реализации системы визуальной детекции препятствий квадрокоптера требуется формирование специализированного набора данных, отражающего реальные условия эксплуатации. В рамках данной работы датасет формировался в виртуальной среде Unreal Engine с использованием симулятора AirSim, что позволило получить контролируемые, воспроизводимые и разнообразные визуальные данные без риска повреждения реального оборудования.

Сбор данных осуществлялся путем записи видеопоследовательностей с бортовой камеры виртуального квадрокоптера в процессе выполнения полетных сценариев. Полеты моделировались в заранее подготовленной среде с размещенными объектами-препятствиями, характерными для городской и пригородной застройки. Камера фиксировала сцены при различных высотах, углах визирования и скоростях движения, что обеспечило вариативность визуального материала.

В рамках работы были записаны видеопоследовательности следующих типов:

- тестовые видеозаписи для первичной отладки системы,
- обучающие видеозаписи различной продолжительности,
- специализированные видеозаписи для каждого типа препятствий.

Всего было сформировано 16 видеозаписей, часть из которых сохранялась в виде непрерывных видеопотоков, а часть – сразу в виде последовательностей кадров. Такой подход позволил обеспечить баланс между объемом данных и удобством их последующей обработки. На рисунке 1.45 изображен полученный набор данных для обучения нейронной сети.

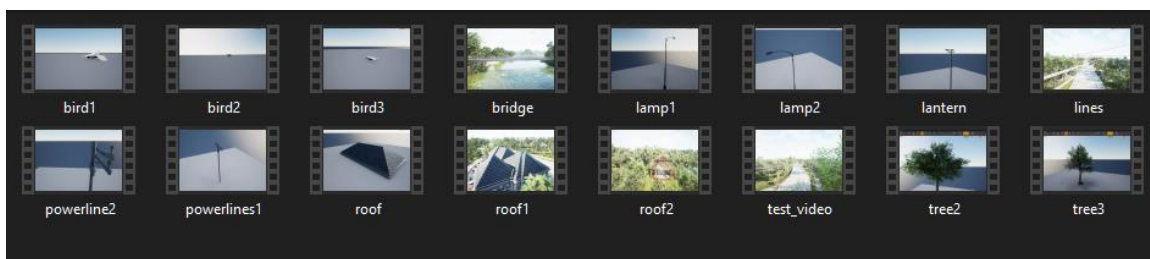


Рисунок 1.45 – Получившийся набор данных для датасета

Для последующей разметки видеоданные были преобразованы в отдельные кадры. Данный этап является необходимым, поскольку большинство современных алгоритмов компьютерного зрения работают с наборами статических изображений.

Преобразование видеопоследовательностей в наборы кадров выполнялось с использованием программных средств на языке Python с применением библиотеки OpenCV, а также с использованием утилиты FFmpeg. Полученные изображения автоматически нумеровались и структурировались по каталогам, что обеспечивало удобство загрузки данных в систему аннотирования.

В результате обработки были получены тысячи кадров, содержащих объекты-препятствия в различных конфигурациях, ракурсах и условиях наблюдения. Получившийся результат датасета представлен на рисунке 1.46.

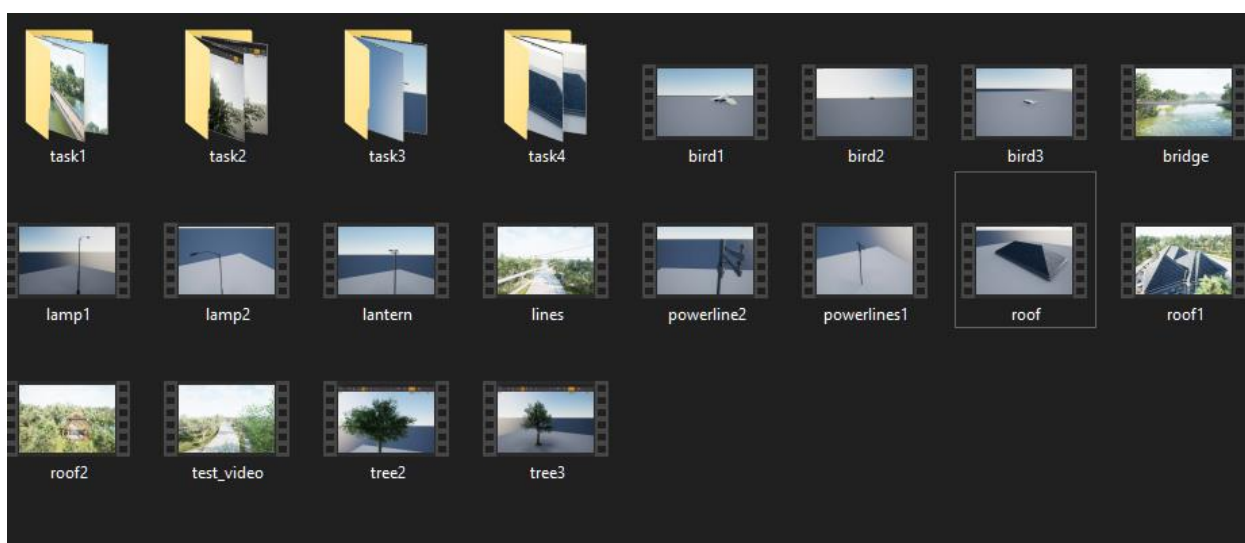


Рисунок 1.46 – Получившийся набор данных для датасета

Аннотирование изображений выполнялось с использованием инструмента CVAT, который представляет собой веб-ориентированную платформу для разметки данных компьютерного зрения. Для каждого изображения вручную задавались ограничивающие прямоугольники и соответствующие метки классов.

В рамках работы были выделены следующие классы объектов:

- крыши зданий,
- верхушки деревьев,
- линии электропередач,
- опоры линий электропередач,
- мосты,
- птицы,
- фонарные столбы.

Аннотирование выполнялось как для тренировочного, так и для валидационного наборов данных. Особое внимание уделялось консистентности разметки и точности выделения объектов, поскольку ошибки на данном этапе могут существенно снизить качество обучения нейронной сети.

После завершения разметки датасет был экспортирован в формате YOLO, а затем дополнительно преобразован в структуру, совместимую с фреймворком YOLOv8.

Перед началом обучения датасет был разделен на три части:

- тренировочный набор,
- валидационный набор,
- тестовый набор.

Такое разделение позволило корректно оценивать способность модели к обобщению и избежать переобучения. Дополнительно в процессе обучения применялись методы аугментации данных, включая масштабирование,

повороты и изменения яркости изображений, что повышало устойчивость модели к вариативности условий съемки.

Обучение модели выполнялось с использованием предварительно обученных весов, что позволило сократить время обучения и повысить итоговую точность.

Основные параметры обучения включали:

- количество эпох,
- размер пакета данных,
- размер входных изображений,
- скорость обучения.

Процесс обучения проводился в среде Jupyter Notebook, что обеспечило удобство контроля за ходом эксперимента, визуализации метрик и анализа промежуточных результатов. В процессе обучения осуществлялся мониторинг функции потерь и показателей точности на валидационном наборе данных. На рисунке 1.47 представлено обучение модели YOLOv8.

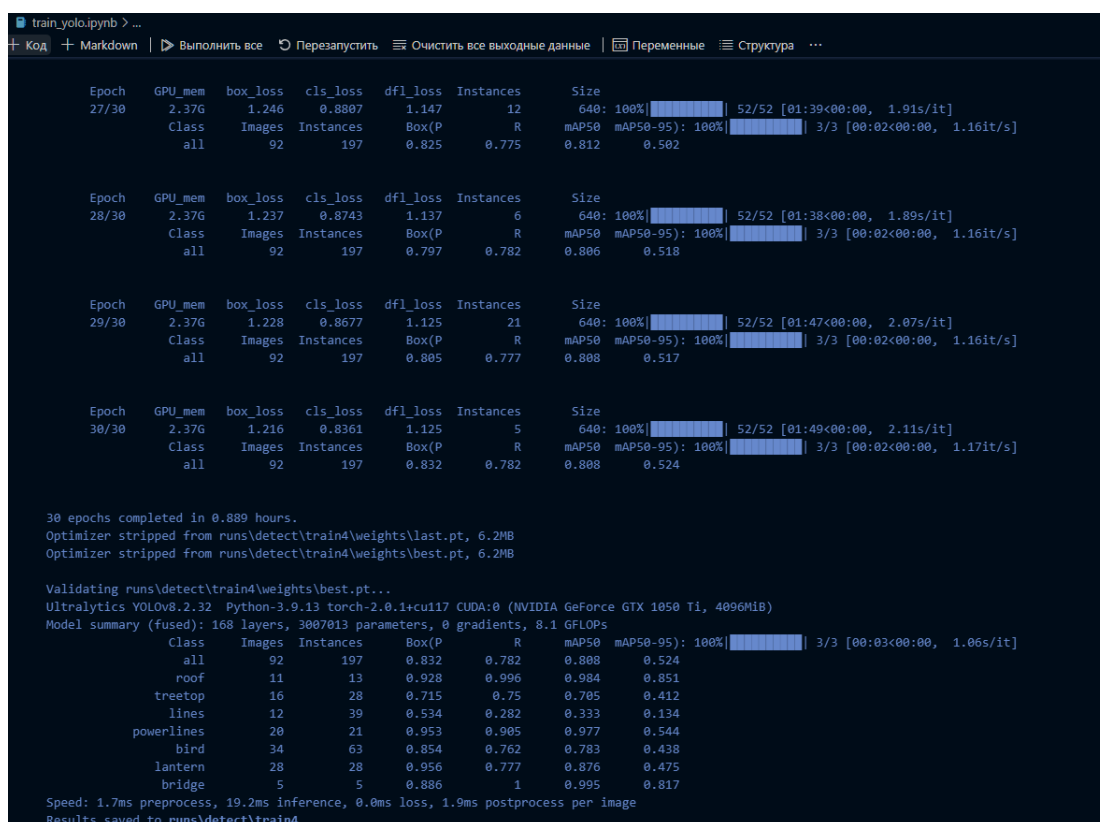


Рисунок 1.47 – Обучение модели YOLOv8

1.3.3.3 Анализ результатов обучения и обнаружения объектов

По завершении обучения нейронной сети была выполнена оценка ее эффективности на валидационном и тестовом наборах данных. Основными метриками оценки качества модели являлись точность и полнота, которые позволяют комплексно оценить способность модели обнаруживать объекты и минимизировать количество ложных срабатываний.

Анализ результатов показал, что модель уверенно распознает крупные и хорошо различимые объекты, такие как мосты, крыши зданий и опоры линий электропередач. Более сложными для детекции оказались тонкие и малоконтрастные объекты, в частности линии электропередач и удаленные птицы, что обусловлено их малыми геометрическими размерами и особенностями визуального восприятия.

Рассмотрим получившиеся результаты обучения нейросети YOLOv8. Общая сводка обучения указана в таблице 1.7.

Таблица 1.7 – Общая сводка обучения

Количество изображений	92
Количество экземпляров	197
Точность	0.832
Полнота	0.782
Средняя точность при mAP50	0.808
Средняя точность при mAP50-95	0.524

Из таблицы 1.7 видно, что модель достигла достаточно высокой точности и полноты, что означает, что большинство объектов были правильно распознаны и классифицированы.

Теперь рассмотрим каждый класс отдельно:

– Крыша:

Таблица 1.8 – Обучение объекта препятствия «Крыша»

Количество изображений	11
Количество экземпляров	13
Точность	0.928
Полнота	0.996
Средняя точность при mAP@50	0.984
Средняя точность при mAP@50-95	0.851

Модель показала отличные результаты для класса «Крыша», с высокой точностью и полнотой. Высокие значения mAP указывают на хорошее распознавание этого класса.

– Верхушки деревьев:

Таблица 1.9 – Обучение объекта препятствия «Верхушки деревьев»

Количество изображений	16
Количество экземпляров	28
Точность	0.715
Полнота	0.75
Средняя точность при mAP50	0.705
Средняя точность при mAP50-95	0.412

Результаты для класса «Верхушки деревьев» ниже по сравнению с классом «крыша». Точность и полнота здесь ниже, что может указывать на сложности в распознавании этого класса, возможно из-за схожести с другими объектами или вариаций в форме.

– Линии электропередач:

Таблица 1.10 – Обучение объекта препятствия «Линии электропередач»

Количество изображений	12
Количество экземпляров	39
Точность	0.534
Полнота	0.282
Средняя точность при mAP50	0.333

Продолжение таблицы 1.10

Средняя точность при mAP50	0.333
Средняя точность при mAP50-95	0.134

Класс «Линии электропередач» показал самые низкие результаты. Низкие значения точности и полноты указывают на значительные трудности модели в распознавании и классификации этих объектов. Это может быть связано с тонкостью и сложностью линий, что требует улучшений в данных или настройках модели.

– Опоры линий электропередач:

Таблица 1.11 – Обучение объекта препятствия «Опоры линий электропередач»

Количество изображений	20
Количество экземпляров	21
Точность	0.953
Полнота	0.905
Средняя точность при mAP _{ч50}	0.977
Средняя точность при mAP50-95	0.544

Модель показала отличные результаты для класса «Опоры линии электропередач», с высокими значениями точности, полноты и mAP. Это указывает на хорошее распознавание этого класса.

– Птицы:

Таблица 1.12 – Обучение объекта препятствия «Птицы»

Количество изображений	34
Количество экземпляров	63
Точность	0.854
Полнота	0.762
Средняя точность при mAP50	0.783
Средняя точность при mAP50-95	0.438

Класс «Птицы» также показал хорошие результаты, хотя $mAP@50-95$ ниже, что может указывать на сложности в распознавании мелких деталей или различий между птицами.

– Фонарный столб:

Таблица 1.13 – Обучение объекта препятствия «Фонарный столб»

Количество изображений	28
Количество экземпляров	28
Точность	0.956
Полнота	0.777
Средняя точность при $mAP50$	0.876
Средняя точность при $mAP50-95$	0.475

Модель хорошо распознала класс «Фонари», с высокой точностью и средними значениями полноты и mAP .

– Мост:

Таблица 1.14 – Обучение объекта препятствия «Мост»

Количество изображений	5
Количество экземпляров	5
Точность	0.886
Полнота	1.0
Средняя точность при $mAP50$	0.995
Средняя точность при $mAP50-95$	0.817

Класс «Мост» показал отличные результаты, с высокой точностью, полнотой и mAP . Это указывает на легкость распознавания мостов для модели.

Модель показала высокие результаты для таких классов, как «крыша», «опоры линий электропередач», «птицы», «фонарные столбы» и «мосты». Это свидетельствует о хорошем распознавании и классификации этих объектов.

Классы «верхушки деревьев» и «линии электропередач» показали более низкие результаты, что указывает на необходимость улучшения данных или параметров модели для этих объектов. Возможно, требуется больше данных для обучения или использование более сложных методов для улучшения распознавания этих объектов.

Полученные результаты свидетельствуют о корректности выбранного подхода к формированию датасета и обучению модели. Использование симуляционной среды Unreal Engine совместно с AirSim позволило создать воспроизводимую и масштабируемую систему подготовки данных, а применение архитектуры YOLOv8 обеспечило необходимый баланс между точностью обнаружения и вычислительной эффективностью.

В совокупности реализованный подход подтверждает возможность применения методов компьютерного зрения и нейронных сетей для решения задач обнаружения препятствий в системах автономного управления квадрокоптерами.

1.3.4 Результаты визуального моделирования и тестирование системы

В результате выполнения работ, была разработана и протестирована система визуального моделирования движения квадрокоптера в среде Unreal Engine с использованием симулятора AirSim. Данная система обеспечивает имитацию полета беспилотного летательного аппарата, формирование визуальных данных с бортовых камер, а также обнаружение препятствий в условиях, приближенных к реальным сценариям эксплуатации квадрокоптера.

На этапе тестирования выполнялись полеты виртуального квадрокоптера в подготовленной симуляционной среде, включающей городскую застройку, элементы инфраструктуры и природные объекты.

Полеты осуществлялись по различным траекториям, включая прямолинейное движение, маневры на малой высоте, облет препятствий и посадку.

На рисунке 1.48 представлен общий вид симуляционной сцены во время полета квадрокоптера. Можно отметить реалистичность визуального окружения, корректную ориентацию летательного аппарата в пространстве и соответствие его движения заданным управляющим воздействиям.

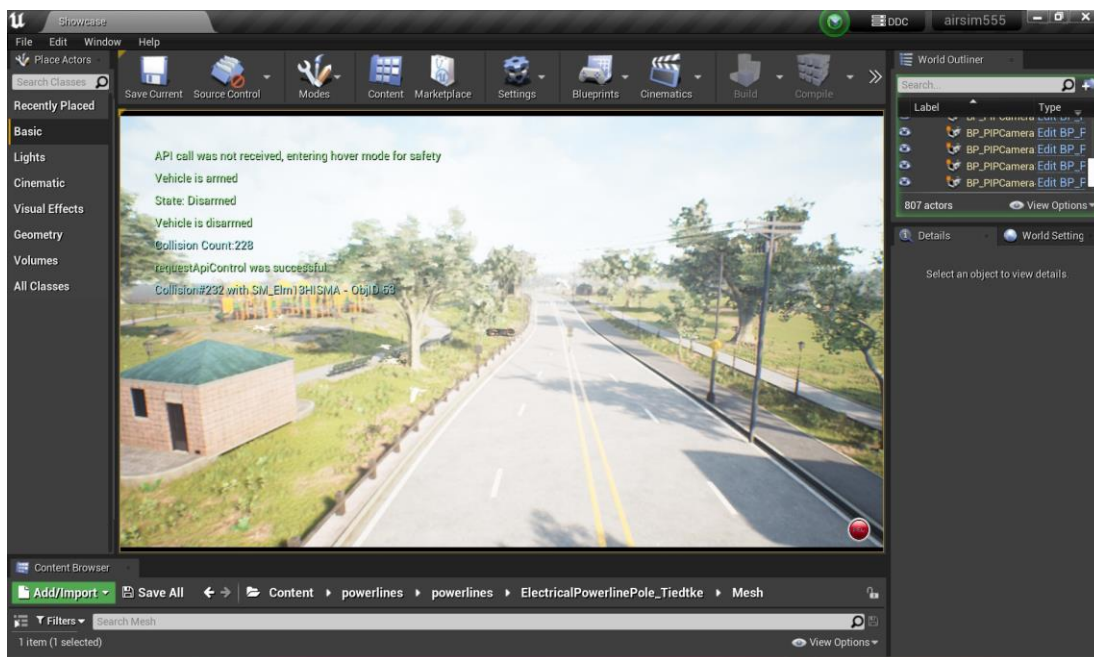


Рисунок 1.48 – Полет виртуального квадрокоптера в среде Unreal Engine

В процессе моделирования подтверждена корректная работа физической модели движения, реализованной в AirSim, а также стабильное взаимодействие симулятора с программной частью, отвечающей за управление и обработку данных.

Для управления виртуальным квадрокоптером использовалась разработанная программная система, принимающая данные ориентации от полунатурного стенда моделирования. Управление осуществлялось посредством текстового меню оператора, реализующего функции взлета, посадки, отображения телеметрии, управления камерами и контроля состояния системы.

На рисунке 1.49 представлен пример интерфейса управления квадрокоптером в процессе моделирования. В интерфейсе отображаются текущие углы ориентации, состояние калибровки, а также активные режимы работы системы.

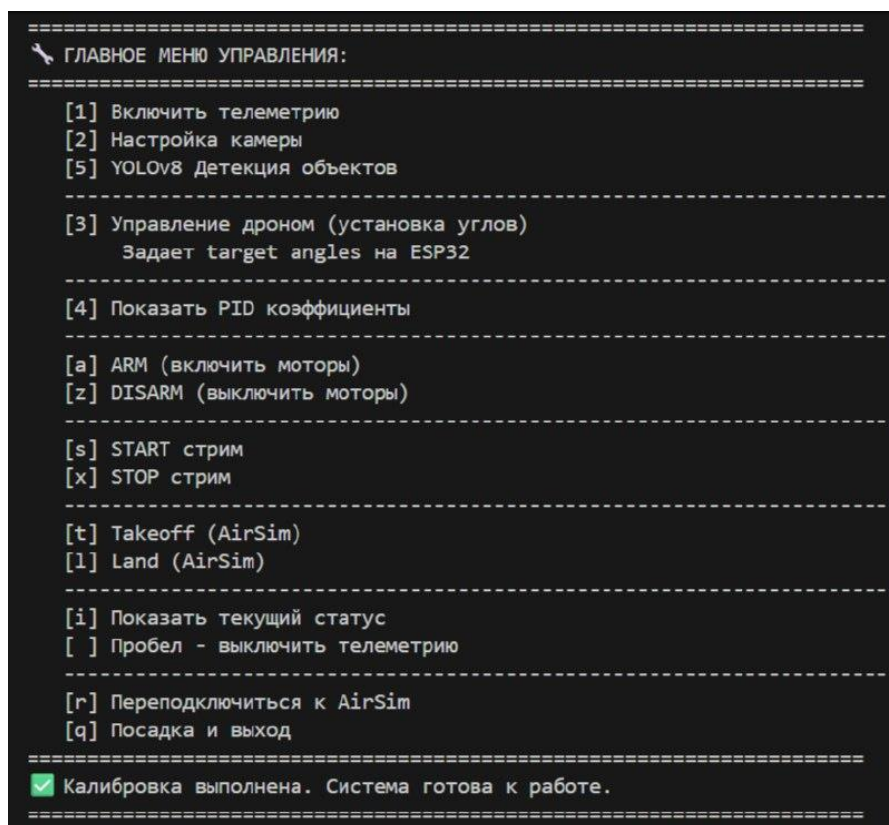


Рисунок 1.49 – Интерфейс управления квадрокоптером в процессе моделирования

Проведенные испытания показали, что система управления обеспечивает устойчивую работу в режиме реального времени, а задержки при передаче и обработке данных не оказывают критического влияния на качество управления и визуализации движения квадрокоптера.

Одним из ключевых результатов визуального моделирования является работа подсистемы обнаружения препятствий на основе нейронной сети. В процессе тестирования обученная модель применялась к видеопотоку, формируемому бортовой камерой квадрокоптера в AirSim.

На рисунке 1.50 приведен пример обнаружения объектов в симуляционной среде. На изображении отображены ограничивающие

прямоугольники и метки классов для различных типов препятствий, включая элементы городской инфраструктуры и природные объекты.



Рисунок 1.50 – Пример обнаружения препятствий в процессе моделирования полета

Результаты тестирования показали, что система уверенно распознает большинство объектов, представленных в обучающем датасете, и корректно отображает их положение в поле зрения камеры. Это подтверждает применимость выбранного подхода к использованию симуляционной среды для отладки и тестирования алгоритмов визуального восприятия.

Таким образом, в ходе выполнения раздела была разработана комплексная система визуального моделирования движения квадрокоптера, включающая:

- симуляционную среду на базе Unreal Engine,
- физически корректную модель квадрокоптера в AirSim,
- систему формирования визуальных данных с бортовых камер,
- подсистему обнаружения препятствий,
- программный интерфейс управления и мониторинга.

Полученные результаты подтверждают, что использование виртуальной среды позволяет эффективно тестировать алгоритмы управления и визуального восприятия без необходимости проведения полевых испытаний на ранних этапах разработки.

Несмотря на широкие возможности виртуального моделирования, оно не позволяет в полной мере учесть все особенности реальных датчиков и механических элементов системы управления. В частности, влияние шумов, погрешностей измерений и особенностей аппаратной реализации требует использования физических компонентов в контуре управления.

В связи с этим следующим этапом работы является описание и разработка стенда полунатурного моделирования системы управления движением квадрокоптера, в котором реальные датчики ориентации используются совместно с виртуальной средой моделирования.

1.4 Анализ результатов

1.4.1 Режимы функционирования и сценарии использования полунатурного стенда

Разработанный полунатурный стенд и программно-аппаратный комплекс предназначены для проведения экспериментальных исследований процессов ориентации и визуального моделирования движения квадрокоптера в виртуальной среде AirSim. Архитектура системы обеспечивает возможность работы в нескольких режимах, отличающихся уровнем динамики, степенью вовлечённости оператора и характером анализируемых параметров. Это позволяет поэтапно проверять корректность функционирования отдельных подсистем и всей системы в целом.

Общий вид стенда полунатурного моделирования с закреплённым квадрокоптером представлен на рисунке 1.51.

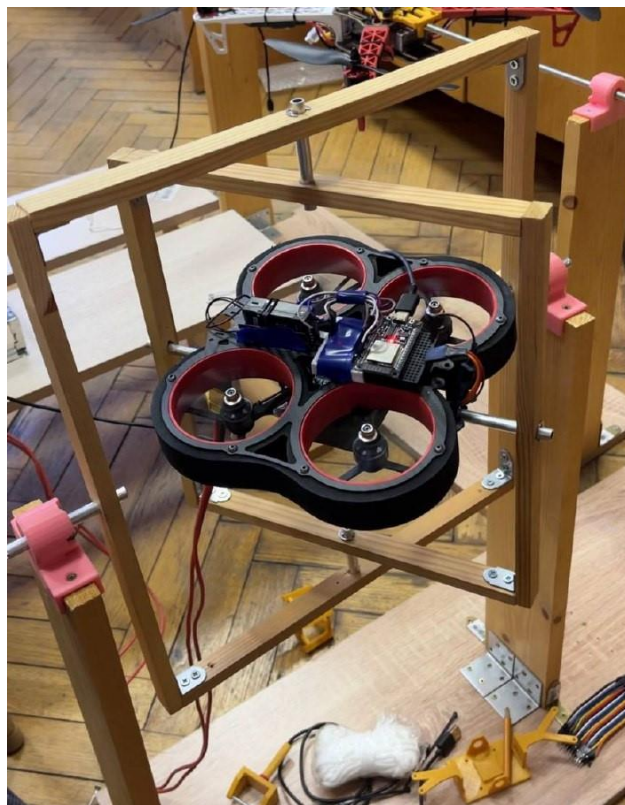


Рисунок 1.51 – Общий вид стенда полунатурного моделирования

Перед началом экспериментальных исследований полунатурного стенда выполняется обязательная процедура статической проверки и калибровки системы. Данная процедура предназначена для подготовки аппаратно-программного комплекса к дальнейшей работе и не относится к самостоятельным режимам функционирования стенда.

В ходе статической проверки и калибровки квадрокоптер, закреплённый на кардановом подвесе, находится в неподвижном состоянии. На данном этапе осуществляется начальная инициализация системы, включающая:

- запуск микроконтроллера ESP32 и инициализацию инерциального модуля MPU6050,
- выполнение процедуры калибровки гироскопов и акселерометров,
- установка нулевых значений ориентации стенда,
- проверку корректности передачи данных по каналу связи между ESP32 и персональным компьютером.

Основной задачей данного режима является устранение систематических ошибок измерения, компенсация смещений датчиков и проверка устойчивости работы программных модулей при отсутствии внешних воздействий. Полученные в статическом режиме данные позволяют убедиться в отсутствии дрейфа углов ориентации и корректной работе алгоритмов обработки измерений. На рисунке 1.52 представлена работа калибровки системы.

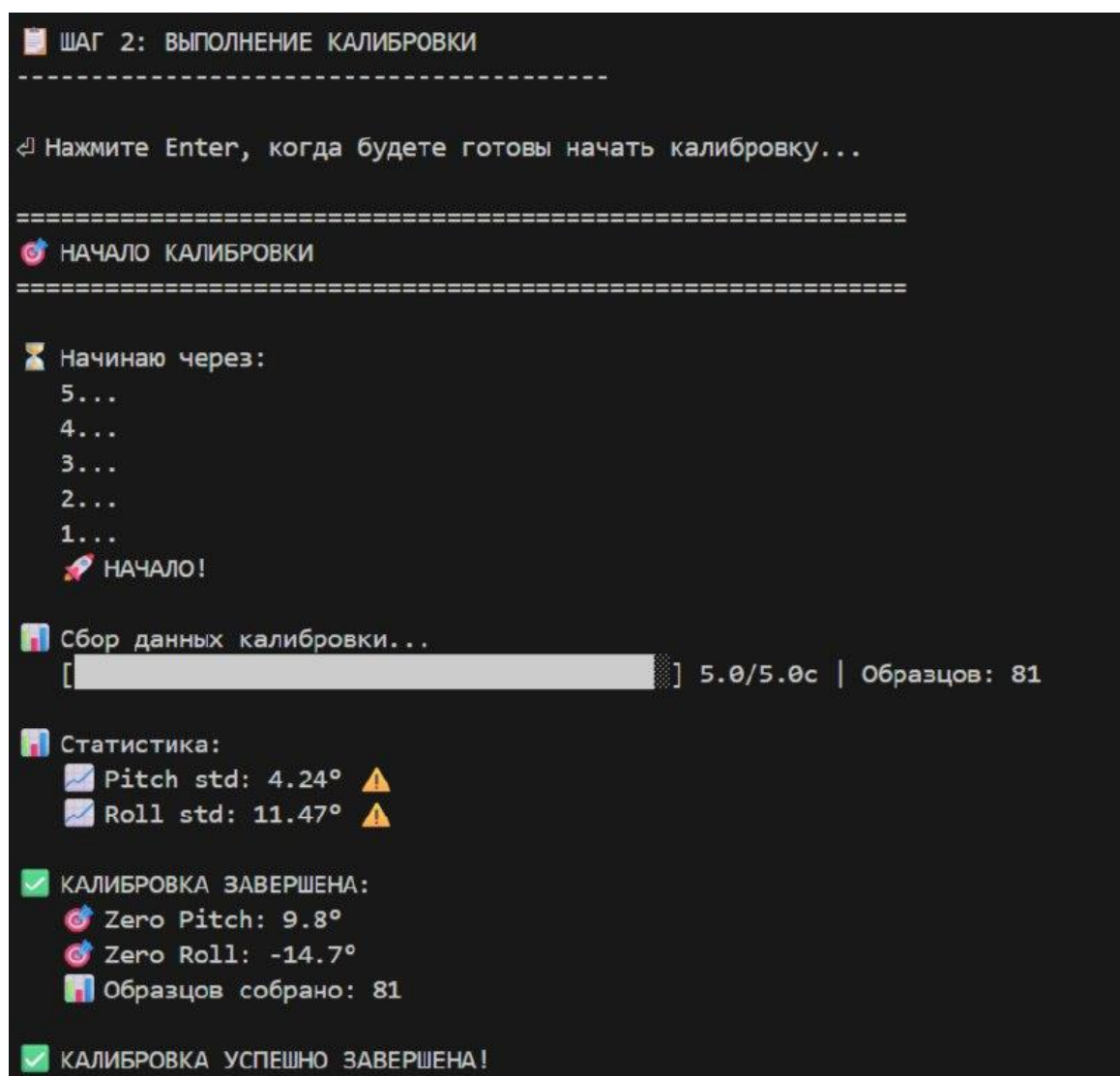


Рисунок 1.52 – Пример калибровки квадрокоптера

После завершения процедуры статической проверки и калибровки полунатурный стенд может быть переведён в один из рабочих режимов функционирования. В рамках данной работы были реализованы и использованы следующие режимы функционирования полунатурного стенда: режим ручного управления ориентацией стенда; экспериментальный режим визуального моделирования.

Режим ручного управления ориентацией стенда – после завершения статической проверки система переводится в режим ручного управления ориентацией стенда. В этом режиме оператор вручную изменяет положение карданового подвеса, создавая реальные угловые перемещения по осям крена, тангажа и курса.

При этом:

- MPU6050 регистрирует угловые скорости и углы ориентации;
- данные передаются на персональный компьютер;
- программный модуль обработки вычисляет текущие значения ориентации;
- виртуальный квадрокоптер в среде AirSim синхронно повторяет движение реального стенда.

Данный режим используется для оценки следующих характеристик системы:

точности передачи углов ориентации;

- задержки между физическим перемещением стенда и реакцией виртуальной модели;
- устойчивости визуальной синхронизации при плавных и резких изменениях ориентации;
- корректности работы замкнутого контура «стенд – AirSim».

Режим ручного управления является ключевым для выявления накопленных ошибок и временных задержек, а также для первичной оценки адекватности математической модели и программной реализации.

Экспериментальный режим визуального моделирования – основным режимом использования разработанной системы является экспериментальный режим визуального моделирования. В данном режиме полунатурный стенд используется совместно с виртуальной средой Unreal Engine и симулятором AirSim для воспроизведения сценариев полёта в условиях, приближенных к реальным.

В экспериментальном режиме:

- физическое вращение стенда формирует управляющие воздействия;
- виртуальный квадрокоптер перемещается в заранее настроенной трёхмерной среде;

- бортовые камеры AirSim формируют визуальные данные;
- осуществляется наблюдение за поведением системы при наличии виртуальных препятствий.

Особенностью данного режима является возможность одновременного анализа:

- ориентации квадрокоптера;
- визуальной информации с бортовых камер;
- реакции системы на изменения ориентации стенда;
- устойчивости работы при длительном моделировании.

Экспериментальный режим позволяет проводить серию повторяемых испытаний без риска повреждения реального летательного аппарата, что особенно важно на этапе отладки алгоритмов управления и визуального восприятия. На рисунке 1.53 изображено тестирование экспериментального режима.

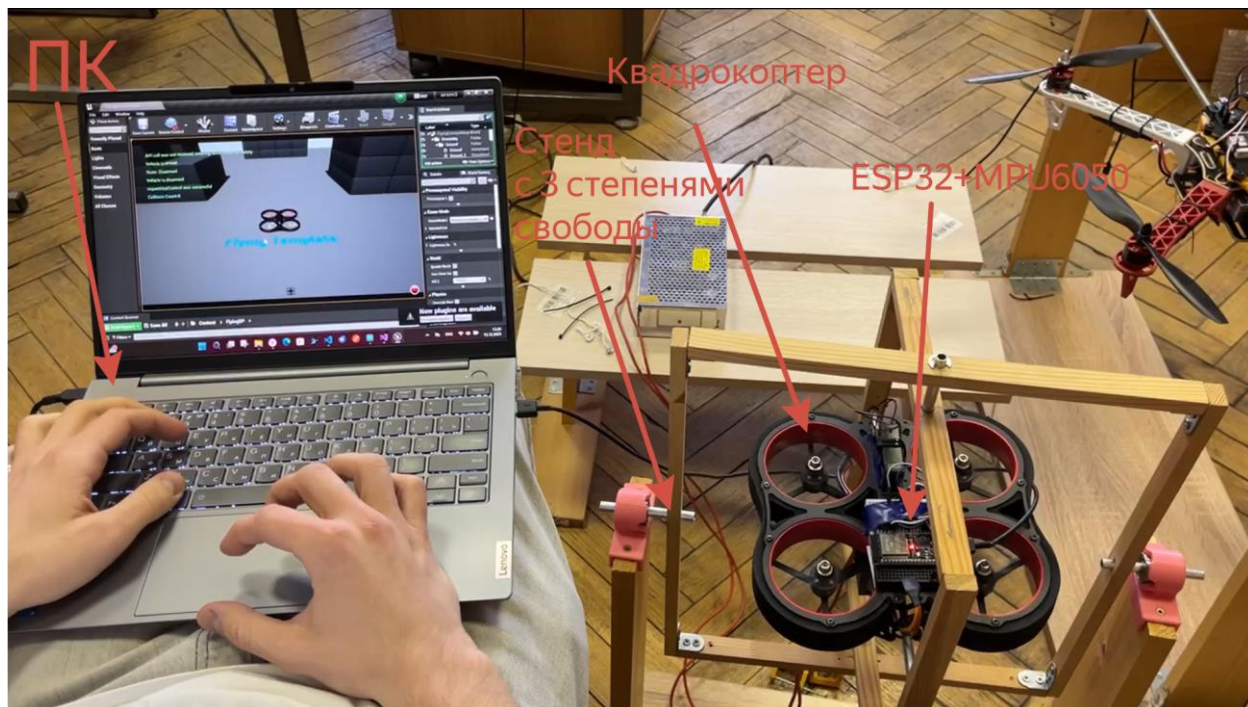


Рисунок 1.53 – Тестирование использования системы в экспериментальном режиме

Для обеспечения корректности и безопасности экспериментальных исследований был разработан регламент запуска и остановки системы. Перед началом каждого эксперимента выполняется обязательная инициализация датчиков и проверка связи между подсистемами. В процессе работы используются программные ограничения на допустимые углы ориентации, а также механизмы аварийной остановки, предотвращающие выход системы за безопасные пределы.

Такая организация экспериментов позволяет снизить вероятность возникновения ошибок, связанных с аппаратными сбоями или некорректной инициализацией, и обеспечивает воспроизводимость результатов.

В процессе работы в различных режимах регистрируются следующие параметры:

- углы ориентации квадрокоптера,
- временные задержки передачи данных,
- визуальная синхронизация движения,
- устойчивость работы системы при длительной нагрузке.

Таким образом, разработанный полунатурный стенд и программно-аппаратный комплекс обеспечивают возможность проведения экспериментальных исследований в различных режимах, включая статические проверки, динамическое управление ориентацией и визуальное моделирование полёта в среде AirSim.

Реализованная архитектура позволяет регистрировать параметры ориентации, анализировать поведение виртуальной модели, а также оценивать устойчивость и точность работы системы в условиях, приближенных к реальным.

Полученные в ходе экспериментального моделирования данные используются для дальнейшего анализа характеристик системы, оценки её точности и эффективности, что рассматривается в следующем разделе работы.

1.4.2 Результаты моделирования

Результаты моделирования, полученные в ходе выполнения данного проекта, представляют собой воспроизведение полётного движения квадрокоптера в виртуальной среде AirSim. Моделирование проводилось в трёхмерной сцене, содержащей виртуальные препятствия, и позволило оценить корректность работы программно-аппаратного комплекса в динамических условиях.

Общая схема виртуальной полётной сцены, используемой при моделировании, представлена на рисунке 1.53. В рамках данной сцены реализовано свободное пространство для движения квадрокоптера и размещены препятствия, предназначенные для проверки функционирования системы визуального восприятия и навигации.

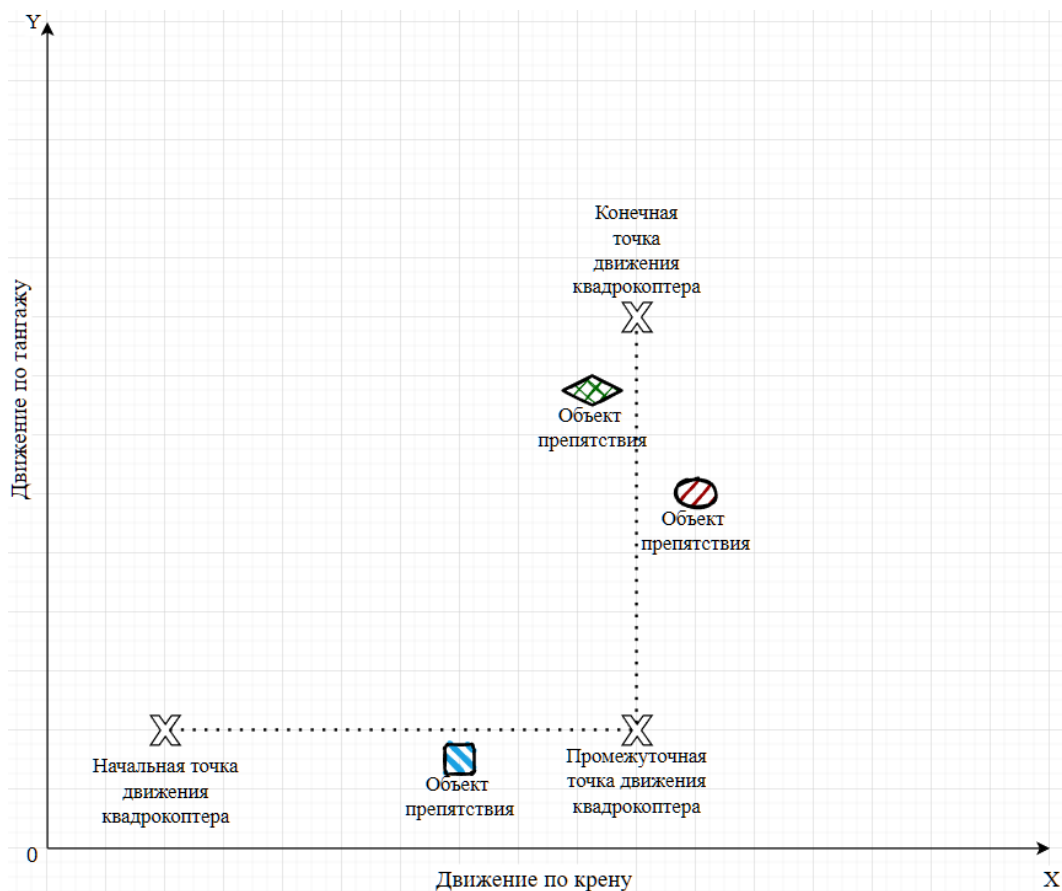


Рисунок 1.53 – Виртуальная полётная сцена в среде AirSim

На начальном этапе моделирования виртуальный квадрокоптер находился в неподвижном состоянии. Углы крена, тангажа и рыскания принимали нулевые значения, что соответствовало исходному положению физического стенда. Данный этап использовался для проверки корректности инициализации системы и устойчивости работы программных модулей при отсутствии управляющих воздействий.

Далее в систему была подана команда на изменение угла крена квадрокоптера. Значение крена изменялось до заданного уровня и сохранялось в течение заданного временного интервала $\Delta t_1 = 50\text{с}$, после чего угол крена вновь устанавливался равным нулю. На следующем этапе, спустя временной интервал Δt_2 , формировалось управляющее воздействие на изменение угла тангажа квадрокоптера, который также изменялся до заданного значения и затем возвращался в исходное положение.

Изменение углов крена и тангажа во времени в ходе моделирования представлено на рисунке 1.54. График наглядно демонстрирует моменты подачи управляющих воздействий, длительность их действия, а также процесс возврата ориентации квадрокоптера к исходному состоянию.

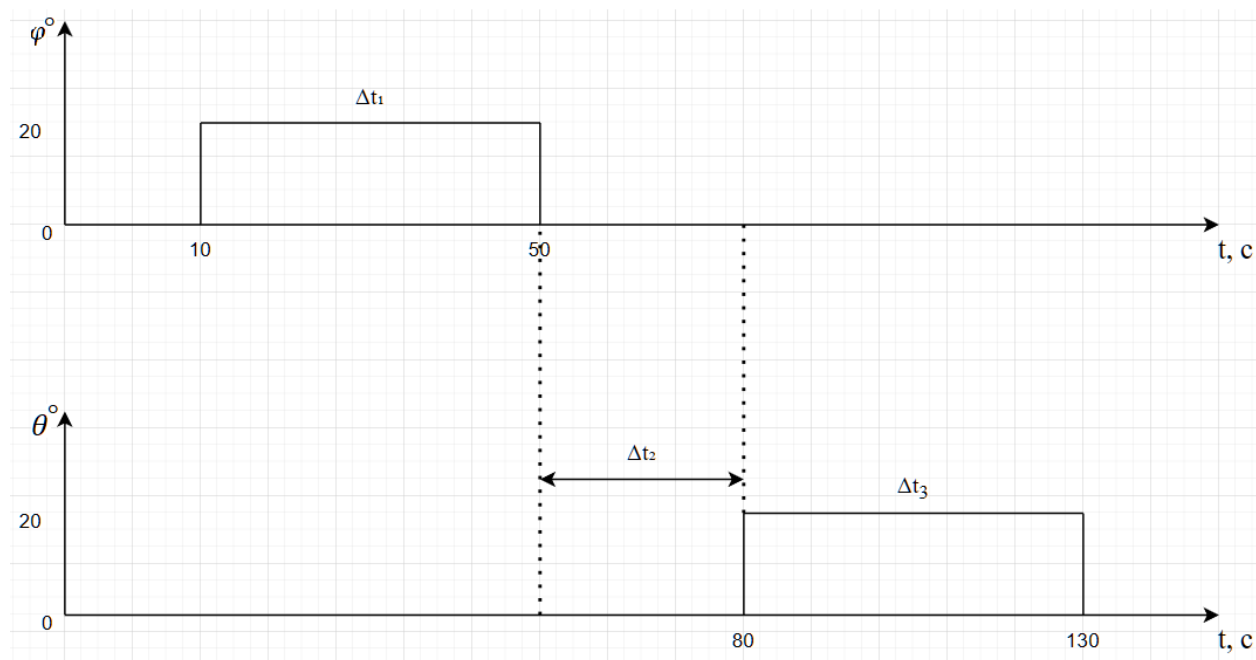


Рисунок 1.54 – Изменение углов крена и тангажа во времени

В результате изменения ориентации квадрокоптера формировалось поступательное движение в виртуальной среде. На рисунках 1.55–1.57 представлены графики изменения угловых скоростей квадрокоптера по осям, связанным с земной системой координат.

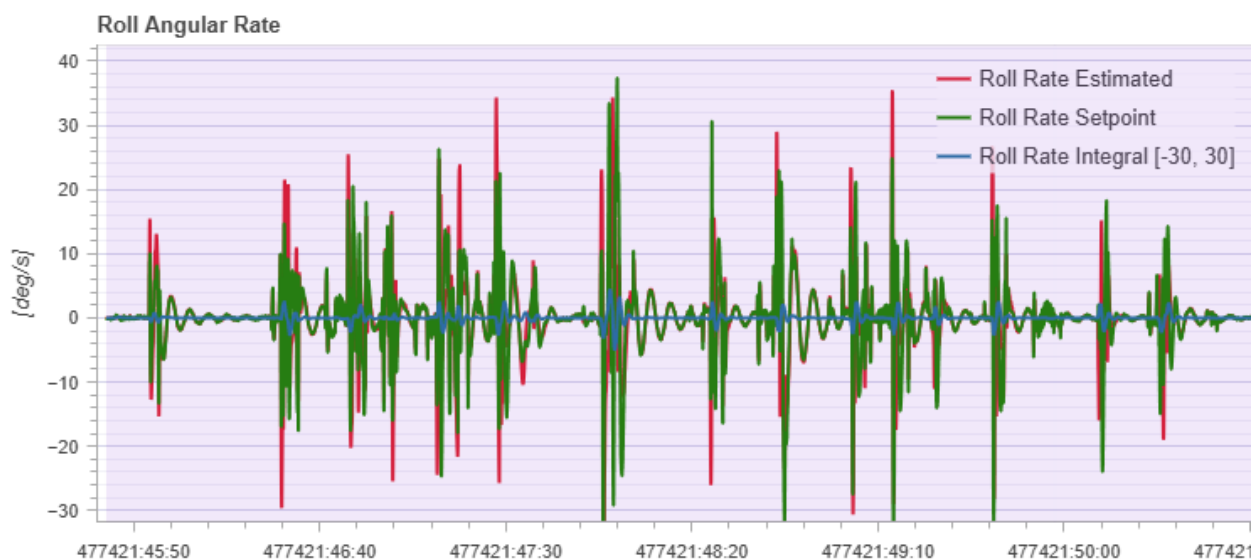


Рисунок 1.55 – График угловой скорости по крену



Рисунок 1.56 – График угловой скорости по тангажу

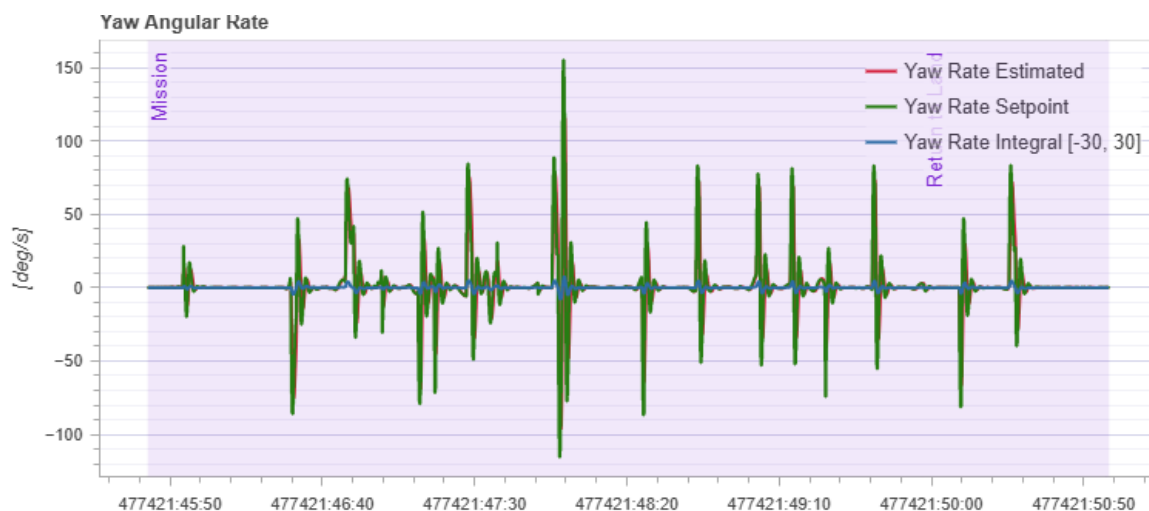


Рисунок 1.57 – График угловой скорости по курсу

На следующем этапе были проанализированы траектории движения квадрокоптера в пространстве. На рисунках 1.58–1.60 представлены графики изменения координат квадрокоптера по осям X, Y и Z. Полученные зависимости подтверждают корректное формирование поступательного движения в результате изменения углов ориентации и демонстрируют согласованность расчётной модели движения с визуальной динамикой виртуального объекта.

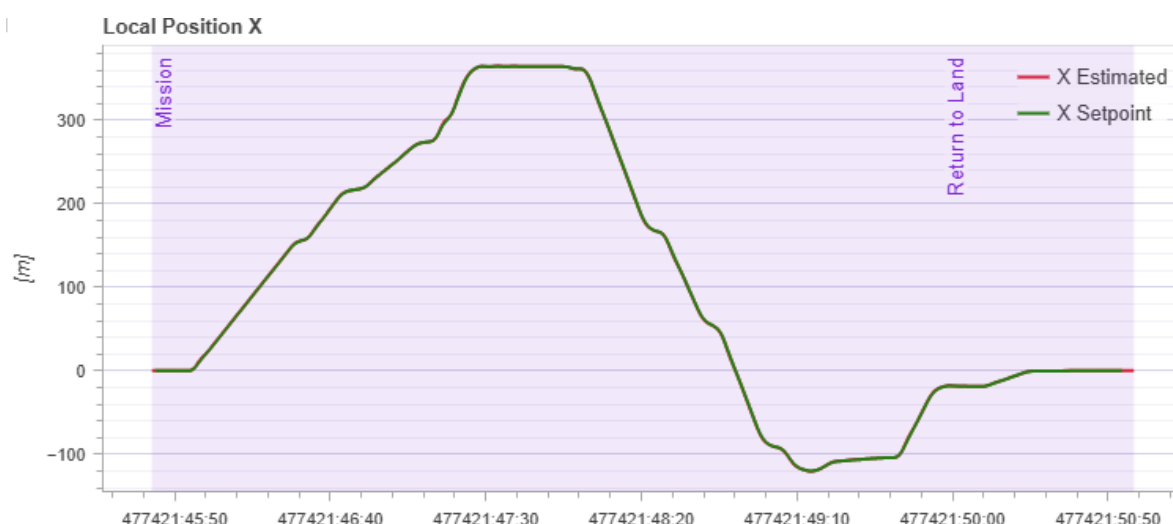


Рисунок 1.58 – Изменение координаты x во времени

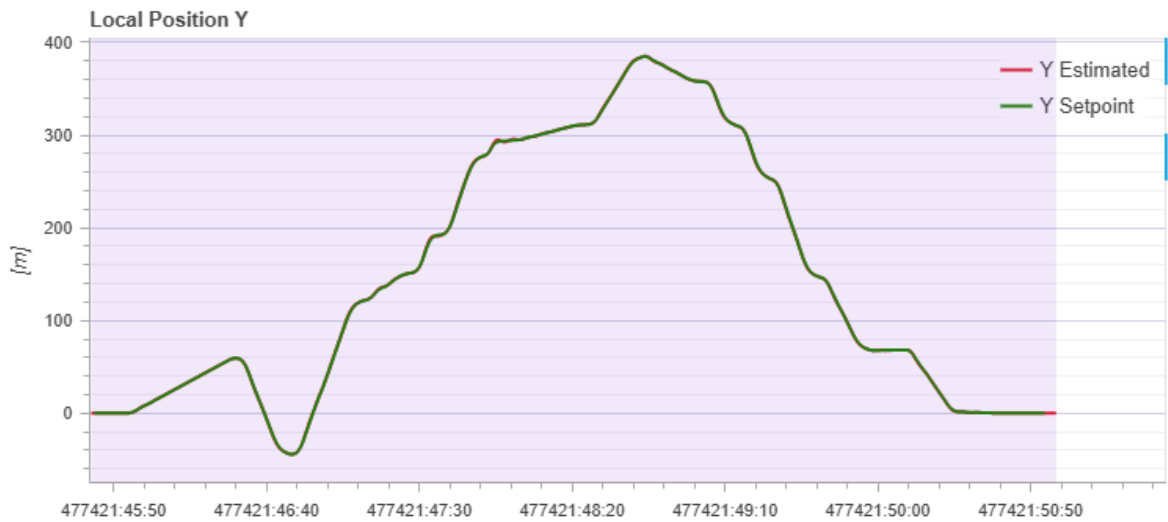


Рисунок 1.59 – Изменение координаты у во времени

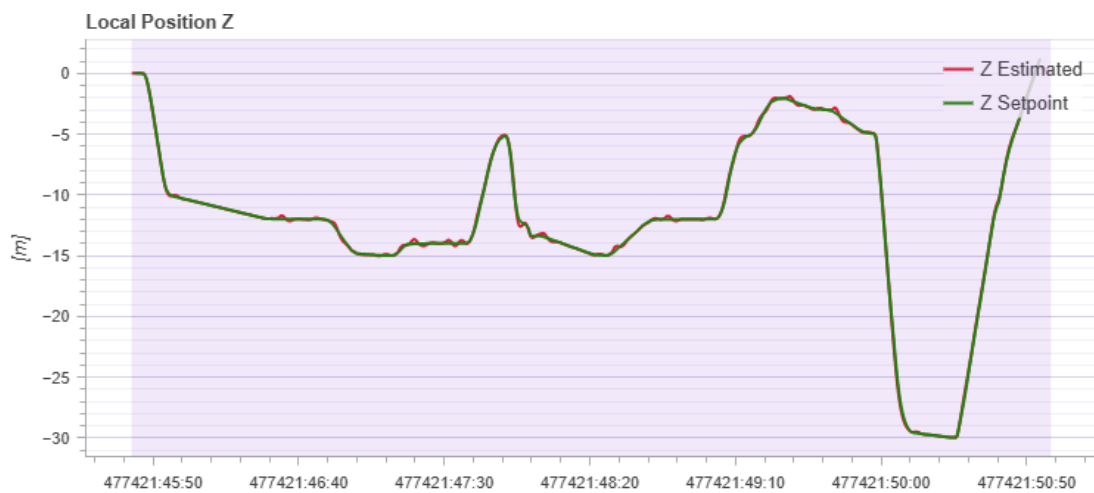


Рисунок 1.60 – Изменение координаты z во времени

В процессе моделирования также проводилась проверка работы модуля визуального восприятия и детекции препятствий. Виртуальные камеры квадрокоптера формировали изображения окружающей среды, на основе которых осуществлялось обнаружение объектов, находящихся на траектории движения. Примеры результатов детекции препятствий в ходе моделирования представлены на рисунках 1.61–1.62.



Рисунок 1.61 – Детекция объекта препятствия «крыша»



Рисунок 1.62 – Детекция объекта препятствия «Линии электропередач»

Проведённое моделирование показало, что разработанная система полунатурного моделирования обеспечивает корректное воспроизведение движения квадрокоптера в виртуальной среде.

1.4.3 Вывод

В ходе выполнения данной работы была разработана и исследована система полунатурного моделирования движения квадрокоптера,

объединяющая физический стенд с виртуальной средой моделирования на базе Unreal Engine и AirSim. Проведённые исследования позволили оценить эффективность выбранных технических и программных решений, а также степень достижения поставленных целей.

В рамках теоретического анализа были рассмотрены современные методы моделирования квадрокоптеров, включая численные методы, программные симуляторы и системы полунатурного моделирования. Отдельное внимание было уделено подходам к детекции препятствий, основанным на использовании компьютерного зрения, сенсорных систем и виртуальных датчиков.

Результаты анализа подтвердили целесообразность использования гибридного подхода, при котором физические измерения ориентации сочетаются с высокодетализированной виртуальной средой. Применение AirSim в связке с Unreal Engine позволило создать условия, приближенные к реальным, без необходимости проведения дорогостоящих и потенциально опасных натурных испытаний.

Разработанная система детекции препятствий основана на использовании визуальных данных, формируемых бортовыми камерами виртуального квадрокоптера. В ходе экспериментов были использованы различные типы препятствий, характерные для маловысотных полётов в городских и пригородных условиях, включая здания, мосты, линии электропередач, элементы городской инфраструктуры и динамические объекты.

Полученные результаты показали, что использование сегментационных и глубинных камер AirSim обеспечивает формирование информативных данных, пригодных для последующего анализа и обучения моделей компьютерного зрения. Экспериментальные запуски подтвердили стабильность получения визуальной информации при изменении ориентации квадрокоптера и различных сценариях полёта.

Созданный полунатурный стенд позволил воспроизвести реальные угловые движения квадрокоптера с использованием карданового подвеса и инерциального модуля MPU6050. В процессе экспериментальных исследований было установлено, что стенд обеспечивает устойчивое измерение углов крена, тангажа и курса с достаточной для задач моделирования точностью.

Использование физического стенда позволило:

- учитывать реальные погрешности датчиков,
- анализировать влияние шумов и задержек передачи данных,
- проверить устойчивость программных алгоритмов в условиях, приближенных к реальной эксплуатации.

Таким образом, полунатурный стенд подтвердил свою эффективность как инструмент промежуточного звена между чисто программным моделированием и натурными испытаниями.

Одним из ключевых результатов работы является успешная интеграция физической и виртуальной частей системы. Реализованный замкнутый контур связи обеспечил синхронизацию ориентации между полунатурным стендом и виртуальным квадрокоптером в AirSim.

В ходе тестирования было установлено, что:

- виртуальная модель адекватно реагирует на изменения ориентации физического стенда,
- задержки передачи данных не оказывают критического влияния на визуальную синхронизацию,
- система сохраняет устойчивость при длительном моделировании.

Интеграция позволила объединить преимущества реальных измерений и виртуального моделирования, что является одним из ключевых достоинств разработанного подхода.

Проведённые экспериментальные испытания в различных режимах работы показали, что разработанная система функционирует стабильно и

соответствует поставленным требованиям. В ходе тестов были подтверждены:

- корректность работы программных модулей,
- устойчивость визуального моделирования,
- воспроизводимость экспериментальных сценариев,
- возможность безопасного тестирования алгоритмов управления и детекции.

Полученные результаты демонстрируют, что система может быть использована как платформа для дальнейших исследований в области автономного управления беспилотными летательными аппаратами, а также для отладки алгоритмов компьютерного зрения и навигации.

Разработанная система полунатурного моделирования обеспечивает эффективное взаимодействие между физическим стендом и виртуальной средой, позволяет формировать качественные визуальные данные и проводить комплексные экспериментальные исследования.

2 Технологическая часть

2.1 Введение

Целью технологической части является разработка и описание полунатурного стенда для исследования пространственного положения квадрокоптера с использованием микроконтроллера ESP32 и датчика MPU-6050. Созданный стенд предназначен для полунатурного моделирования – он позволяет воспроизводить угловые движения квадрокоптера в трёх степенях свободы (тангаж, крен и курс), обеспечивая возможность анализа данных ориентации в реальном времени без выполнения реальных полётов.

Особенностью конструкции является механическая простота и модульность. Стенд выполнен из древесных элементов и металлических стержней, что делает его лёгким и удобным для сборки. Электронная часть смонтирована без пайки – соединения между микроконтроллером ESP32 и датчиком MPU-6050 выполнены с помощью Dupont-проводов, что позволяет легко заменять или калибровать компоненты. Передача данных осуществляется по беспроводному каналу Wi-Fi между микроконтроллером ESP32 и персональным компьютером, где данные визуализируются в среде Unreal Engine.

Таким образом, разработанный стенд представляет собой безопасную, компактную и универсальную платформу для отработки алгоритмов ориентации, стабилизации и передачи данных квадрокоптера в полунатурной среде.

2.2 Разработка и конструкция полунатурного стенда

2.2.1 Технологический процесс сборки стенда

Технологический процесс сборки стенда включает последовательность операций по подготовке элементов, сборке рам, установке осей и монтажу

опорной системы. Все этапы выполнялись вручную с использованием стандартного набора инструментов.

Этапы сборки:

- Подготовка материалов – нарезка элементов рам из ДСтП толщиной 10 мм и 15 мм по заданным размерам, шлифовка кромок и сверление отверстий под стержни и крепёжные уголки.

- Сборка внешней рамы – детали соединяются при помощи лёгких металлических крепёжных уголков и саморезов. Контролируется перпендикулярность углов и симметрия конструкции.

- Сборка внутренней рамы – аналогично внешней, собирается квадратная рамка из ДСтП, которая затем монтируется внутри внешней при помощи алюминиевых стержней, образующих ось курса.

- Установка осей и фиксация стопорными кольцами – внутренний полый металлический стержень устанавливается через внутреннюю рамку и фиксируется стопорными кольцами типа e-clip диаметром 6–7 мм. Проверяется лёгкость вращения без люфта.

- Монтаж опорной системы – 3D-печатные крепления с подшипниками закрепляются на деревянных стойках из ДСтП толщиной 15 мм. Через подшипники пропускаются алюминиевые оси, соединяющие опоры с внешней рамой.

- Крепление квадрокоптера – к центральному внутреннему стержню устанавливается рама квадрокоптера, оснащённая микроконтроллером ESP32 и датчиком MPU-6050. Электронные компоненты подключаются с помощью Dupont-проводов.

- Финальная сборка и проверка – после сборки проводится проверка механической подвижности всех осей, корректность фиксации стержней и устойчивость конструкции на опорах.

2.2.2 Конструктивно-технологическое описание стенда

Полунатурный стенд состоит из двух деревянных рам: внешней и внутренней. Рамы соединены между собой таким образом, чтобы обеспечить вращение по трём осям.

Внешняя рама отвечает за вращение по оси тангажа.

Внутренняя рама, установленная внутри внешней, обеспечивает вращение по оси курса.

Через внутреннюю раму проходит металлический полый стержень, который позволяет воспроизводить вращение по оси крена.

Вся конструкция опирается на систему опор с 3D-печатными креплениями, внутри которых установлены подшипники, обеспечивающие плавное вращение стенда в пространстве. Общая схема стенда представлена на рисунке 2.1.

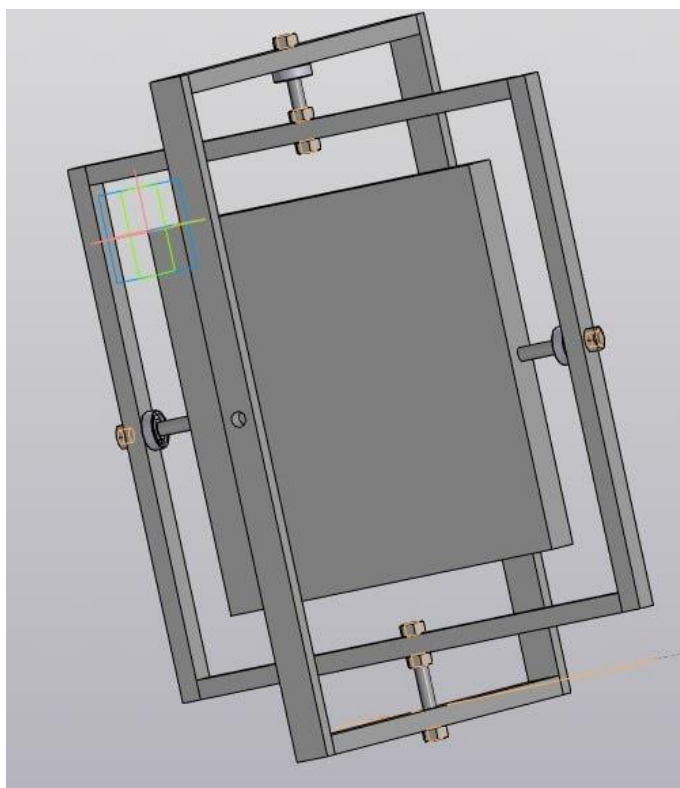


Рисунок 2.1 – Схема стенда

Внешняя рама – выполнена из ДСтП (древесно-стружечная плита) толщиной 10 мм, соединённых по периметру лёгкими металлическими уголками (Рисунок 2.2). Рама служит несущим элементом стенда и обеспечивает вращение по оси тангажа. К внешней раме крепятся опорные стержни, соединяющие конструкцию с опорными балками.



Рисунок 2.2 – Крепежный уголок

Внутренняя рама – также выполнена из ДСтП и закреплена внутри внешней рамы. Она отвечает за вращение по оси курса и соединена с внутренней рамой металлическими стержнями, которые проходят через неё вдоль оси курса. Внутри рамы проходит стержень, на котором крепится квадрокоптер.

Стержни – полые металлические (алюминиевые), что обеспечивает лёгкость и жёсткость конструкции. Стержни фиксируются с помощью стопорных колец диаметром 6-7мм вида e-clip (эллиптические стопорные кольца) (Рисунок 2.3), предотвращающих смещение при вращении и обеспечивающих надёжность механических соединений. Вращение рам относительно стержней происходит без использования подшипников, что упрощает конструкцию и исключает люфты при небольших углах наклона.



Рисунок 2.3 – Стопорное кольцо e-clip

Опорная система – для удержания стенда в пространстве используются опоры с 3D-печатными креплениями, внутри которых установлены подшипники (Рисунок 2.4). Через эти подшипники проходят внешние алюминиевые стержни, соединяющие раму с опорами и обеспечивающие вращение всей конструкции. Опоры зафиксированы на прочной горизонтальной поверхности, обеспечивающей устойчивость системы.



Рисунок 2.4 – Подшипник качения

Общий вид собранной конструкции показан на рисунке 2.5.

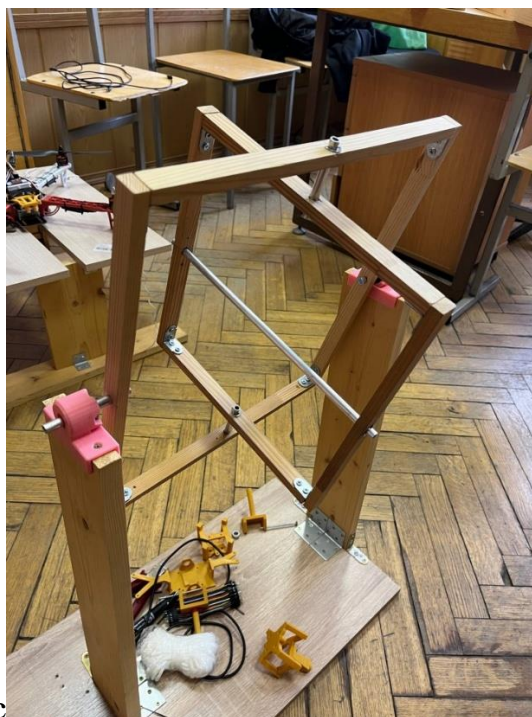


Рисунок 2.5 – Собранный вид стенда

Таким образом, в рамках технологической части была выполнена сборка полунатурного стенда, включающая подготовку деревянных и металлических элементов, установку осей вращения и монтаж опорной системы с подшипниками. Конструкция получилась жёсткой, устойчивой и в то же время лёгкой, что позволяет проводить эксперименты по исследованию углового положения квадрокоптера в контролируемых условиях.

2.3 Установка и описание квадрокоптера-рамы

Квадрокоптер, который закреплен на внутреннем стержне стенда, служит основной частью экспериментальной системы. Его конструкция выполнена на базе готового комплекта рамы Badboy 146 мм из углеродного волокна, предназначенной для лёгких и манёвренных квадрокоптеров класса «Cinewhoop». Данный тип рамы отличается высокой жёсткостью при минимальной массе, что позволяет обеспечить устойчивость к вибрациям при вращении на стенде.

[illegible]

A top-down view of a black and red quadcopter frame. The frame has four large, circular motor mounts with red inner rings. Each mount has a black propeller attached. The central area of the frame is black with some silver-colored structural elements. A red and black cable is connected to the bottom right of the frame.

116

Для обеспечения симметричной тяги и реалистичной имитации динамики квадрокоптера на стенде применяются четыре бесщёточных двигателя 1507 3800 KV. Двигатели установлены на концах лучей рамы с помощью винтов M2. Тип 1507 означает, что статор имеет диаметр 15 мм и высоту 7 мм, а значение 3800 KV характеризует число оборотов на 1 вольт.

Каждый двигатель способен развивать до 550 г тяги на 3-дюймовом пропеллере при напряжении 11.1 В, обеспечивая достаточный запас по мощности.

Для управления скоростью вращения моторов используется 4-в-1 регулятор скорости ESC на 35 А, установленный в центральной части рамы. Данная плата объединяет четыре независимых регулятора в одном корпусе, что уменьшает массу и упрощает компоновку проводки.

Используются четыре трёхлопастных пропеллера 3045 (диаметр 3 дюйма, шаг 4.5 дюйма) из ударопрочного пластика ABS. Такая форма обеспечивает высокий объёмный поток воздуха и плавную динамику при высоких оборотах, а также низкий уровень шума при вращении на стенде. Пропеллеры имеют конфигурацию CW и CCW (правая и левая резьба), что компенсирует реактивные моменты и устойчивость при вращении.

В качестве источника энергии используется литий-полимерная аккумуляторная батарея HRB 1500 mAh 11.1 V (3 S) 65C FPV Pro. Батарея отличается высокой токоотдачей (до 97,5 А в пиковом режиме), что гарантирует стабильную работу всей системы.

Центральный элемент системы – микроконтроллер ESP32, установленный на верхней платформе рамы. Он осуществляет обмен данными с датчиком MPU-6050 по интерфейсу I²C и передаёт результаты по Wi-Fi на ПК. Питание контроллера и датчика осуществляется через встроенный ВЕС регулятора ESC. Все соединения выполнены при помощи Dupont-проводов без пайки, что облегчает модернизацию и обслуживание системы.

При работе стенда квадрокоптер не совершает полёта – он закреплён на внутреннем металлическом стержне. Квадрокоптер вращается в зависимости от оси (тангаж, крен или курс). Датчик MPU-6050 фиксирует изменения ускорений и угловых скоростей, а ESP32 в режиме реального времени передаёт данные на ПК через Wi-Fi в среду Unreal Engine. Таким образом, виртуальная модель квадрокоптера Unreal Engine повторяет физическое вращение стенда, что позволяет проводить испытания и отладку алгоритмов ориентации и стабилизации без реальных полетов.

2.4 Подбор микроэлектромеханических датчиков и микроконтроллера

2.4.1 Микроконтроллер

Микроконтроллер представляет собой компактное электронное устройство, объединяющее в одном корпусе все необходимые элементы для выполнения вычислительных операций и управления различными системами. В его архитектуру входят центральный процессор, оперативная и энергонезависимая память RAM и Flash/ROM, а также набор периферийных блоков – таймеров, аналогово-цифровых преобразователей, интерфейсов обмена данными (UART, SPI, I²C) и портов ввода-вывода. Благодаря сочетанию компактности, энергоэффективности и широких функциональных возможностей, микроконтроллеры нашли применение в робототехнике, автоматизированных установках, бытовой электронике и квадрокоптерах.

Основное достоинство микроконтроллеров заключается в способности выполнять заданные программы в режиме реального времени, обеспечивая обработку информации от датчиков и формирование управляющих сигналов для исполнительных механизмов – электродвигателей, светодиодов и других элементов. Интеграция всех вычислительных и управляющих компонентов в единую систему-на-кристалле обеспечивает высокую надёжность, быстродействие и

компактность, что делает микроконтроллеры ключевым элементом современных интеллектуальных устройств.

Кроме того, современные микроконтроллеры поддерживают подключение широкого спектра периферийных модулей, что позволяет адаптировать их под конкретные задачи и условия эксплуатации. Такая универсальность делает их эффективным инструментом для реализации инновационных технических решений в самых разных областях – от промышленной автоматизации и медицины до аэрокосмических систем.

В системах управления полётом квадрокоптера микроконтроллер выполняет роль центрального вычислительного узла, координирующего работу всех подсистем. Он обрабатывает сигналы от инерциальных датчиков и других сенсоров, определяя текущее положение аппарата в пространстве. На основе этих данных формируются управляющие импульсы, регулирующие скорость вращения двигателей, что обеспечивает стабилизацию, корректировку траектории и удержание квадрокоптера в заданной ориентации. Такая архитектура позволяет достичь высокой точности, устойчивости и адаптивности полёта.

Наиболее распространёнными микроконтроллерами, применяемыми в полётных контроллерах квадрокоптеров, являются серии STM32 от компании STMicroelectronics, отличающиеся высокой производительностью и поддержкой вычислений с плавающей точкой (FPU), а также ATmega (например, ATmega328P) от Microchip Technology, используемые в простых и учебных системах, включая платформы Arduino.

2.4.2 Микроконтроллер ESP32

Микроконтроллер ESP32 (рисунок 2.8) – мощный микроконтроллер, разработанный компанией Espressif Systems, который представляет предыдущую версию своего предшественника ESP8266.



Рисунок 2.8 – Микроконтроллер ESP32

Он сочетает в себе высокую производительность, низкое электропотребление, широкий набор встроенных функций и небольшую цену, что делает его идеальным выбором для современных проектов. Подробные характеристики ESP32 приведены в таблице 2.1

Таблица 2.1 – Технические характеристики ESP32 из документации к микроконтроллеру

Характеристика	Значение
Процессор	Xtensa dual-core 32-bit LX6, до 240 МГц (ESP32-D0WDQ6)
Память ROM	448 КБ
Память SRAM	520 КБ
Встроенная Flash	4 МБ
GPIO	До 32 пинов
UART	Поддерживается (2 контроллера, flow control)
SPI	4 контроллера (SPI0 как буфер, SPI1 как master, SPI2/SPI3 как master/slave)
I^2C	2 интерфейса (master/slave), до 5 Мбит/с
ADC	2 × 12-бит SAR, 18 каналов, до 2 Мвыб/с

Продолжение таблицы 2.1

ДАС	2 × 8-бит
Питание	3.0–3.6 В
Размеры	18 × 25.5 × 3.1 мм
Рабочая температура	–40...+85 °С
Потребление тока	В зависимости от режима: от 5 мкА (Deep Sleep) до 1.1 А (максимальная нагрузка с CPU, Wi-Fi и Bluetooth)

2.4.3 Микроэлектромеханические системы

Микроэлектромеханические системы представляют собой миниатюрные устройства, совмещающие в себе механические и электронные элементы, функционирующие на микроуровне. Они создаются с использованием технологий полупроводникового производства, что позволяет изготавливать датчики и исполнительные элементы размером от нескольких микрометров до миллиметров. Основной принцип работы МЭМС-устройств заключается в преобразовании физических параметров окружающей среды (ускорений, угловых скоростей, давления, магнитного поля и т.д.) в электрические сигналы, которые могут быть оцифрованы и обработаны микроконтроллером.

МЭМС-датчики включают микромеханические элементы (мембраны, балки, подвижные массы), которые реагируют на внешние воздействия, и интегрированные электронные схемы, обеспечивающие измерение и фильтрацию сигналов. Ключевые преимущества МЭМС-технологии заключаются в следующем:

- Компактность – малые размеры позволяют интегрировать сенсоры в мобильные и автономные устройства;
- Низкое энергопотребление – работа с малыми токами делает их оптимальными для встроенных систем;

- Высокая чувствительность и надёжность – современные технологии обеспечивают точное измерение при минимальных шумовых искажениях;

- Доступная стоимость – массовое производство и высокая интеграция снижают себестоимость компонентов.

Принцип функционирования МЭМС-датчиков основан на регистрации изменений физических свойств микромеханических структур под действием внешних факторов. Возникающие сигналы преобразуются в электрические с помощью встроенных аналогово-цифровых преобразователей (АЦП) и передаются для дальнейшей цифровой обработки.

В данном проекте применяется датчик MPU-6050, представляющий собой типичное МЭМС-устройство, объединяющее в одном корпусе трёхосевой акселерометр и трёхосевой гироскоп. Он регистрирует ускорения и угловые скорости вдоль трёх взаимно перпендикулярных осей, что позволяет точно определять пространственное положение и ориентацию квадрокоптера. Полученные данные используются микроконтроллером ESP32 для вычисления углов ориентации и последующей визуализации движения в среде Unreal Engine.

2.4.4 Инерциальный измерительный модуль MPU-6050

MPU-6050 – это шестикомпонентный датчик, объединяющий в себе трехосевой гироскоп и трехосевой акселерометр. Он разработан компанией InvenSense и широко используется в различных приложениях. MPU-6050 способен измерять линейные ускорения и угловые скорости по трём осям, что позволяет вычислять углы тангажа, крена и курса. Датчик изображен на рисунке 2.9.

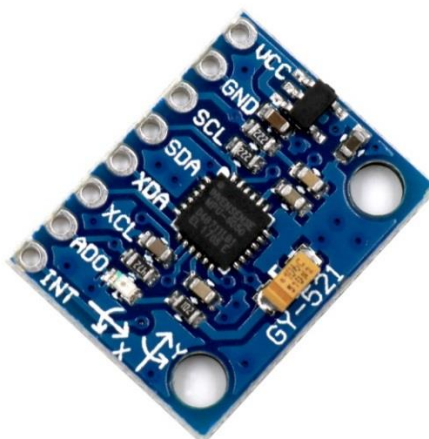


Рисунок 2.9 – Модуль MPU-6050

MPU-6050 поддерживает функцию интеграции данных, которая позволяет объединять данные с акселерометров и гироскопов для получения более точных данных. А также имеет модуль обработки данных, имеющий название DMP (digital motion processor), который обладает встроенными алгоритмами фильтрации. Подробные технические характеристик MPU-6050 приведены в таблице 2.2.

Таблица 2.2 – Технические характеристики MPU-6050 из документации к модулю

Характеристика	Значение
Процессор	Digital Motion Processor (DMP) для 3D motion fusion и алгоритмов
Память	1024-байтный FIFO буфер
Акселерометр	3-осевой MEMS, 16-бит ADC, диапазоны $\pm 2/\pm 4/\pm 8/\pm 16$ g
Гироскоп	3-осевой MEMS, 16-бит ADC, диапазоны $\pm 250/\pm 500/\pm 1000^\circ/\text{с}$
Гироскоп	3-осевой MEMS, 16-бит ADC, диапазоны $\pm 250/\pm 500/\pm 1000/\pm 2000^\circ/\text{с}$
Температурный сенсор	Диапазон $-40...+85^\circ\text{C}$, чувствительность ~ 340 LSB/ $^\circ\text{C}$
Интерфейсы	I ² C (до 400 кГц, primary и auxiliary)
Питание	VDD: 2.375–3.46 В; VLOGIC: 1.71–VDD В

Продолжение таблицы 2.2

Потребление тока	От ~5 мкА (idle/low power) до ~3.9 мА (полная работа: gyro + accel + DMP)
Размеры	4 × 4 × 0.9 мм (QFN-24)
Рабочая температура	–40...+85 °C

После калибровки модуль передаёт значения ускорений и угловых скоростей на ESP32, где они преобразуются в углы Эйлера с последующей передачей на компьютер.

2.4.5 Схема подключения

На рисунке 2.10 представлена схема подключения датчика MPU-6050 к микроконтроллеру ESP32.

Соединение осуществляется четырьмя проводами:

- VCC (3.3 В) – питание датчика,
- GND – общий провод,
- SDA – линия данных интерфейса I²C,
- SCL – линия синхронизации.

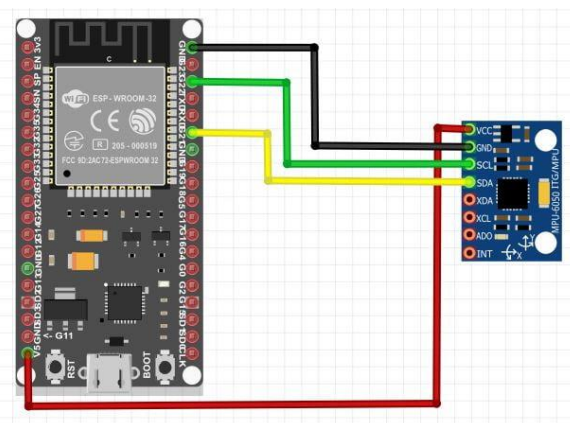


Рисунок 2.10 – Схема подключения MPU-6050 к ESP32

При вращении станда MPU-6050 регистрирует изменения ускорений и угловых скоростей вдоль каждой оси. Данные передаются на ESP32, где

проходят фильтрацию (например, через комплементарный фильтр) и преобразование в ориентационные углы. Далее результаты в формате UDP-пакетов отправляются по Wi-Fi-каналу на персональный компьютер. На стороне ПК реализуется визуализация ориентации в среде Unreal Engine, где движение физического квадрокоптера синхронно отображается в виртуальной модели.

2.5 Вывод

В технологической части была разработана конструкция полунатурного стенда для исследования углового положения квадрокоптера. Стенд выполнен из древесно-стружечных плит и металлических стержней, обеспечивающих три степени свободы вращения – тангаж, крен и курс. Его конструкция отличается простотой, модульностью и устойчивостью за счёт использования 3D-печатных креплений и подшипников.

В состав системы входят квадрокоптер на базе рамы Badboy 146 мм, микроконтроллер ESP32 и инерциальный датчик MPU-6050, обеспечивающие измерение ориентации и передачу данных по Wi-Fi для последующей визуализации в среде Unreal Engine. Разработанный стенд представляет собой компактную и функциональную платформу для исследования и отладки алгоритмов стабилизации квадрокоптера в полунатурных условиях.

3 Охрана труда и окружающей среды

3.1 Введение

Настоящий раздел посвящён вопросам охраны труда и обеспечения безопасности при разработке и эксплуатации полунатурного стенда для моделирования движения квадрокоптера. Основная идея работы заключается в создании аппаратно-программного комплекса, обеспечивающего визуализацию поведения квадрокоптера в виртуальной среде на основе данных, поступающих с реального стенда, что позволяет проводить исследование систем управления и отработку алгоритмов без необходимости проведения полевых испытаний.

В рамках реализации проекта все работы выполняются в лабораторном помещении площадью $19,44 \text{ м}^2$, в котором расположено рабочее место инженера-программиста и установлен полунатурный стенд. Лаборатория оснащена двумя окнами шириной 111 см, обеспечивающими естественное освещение и вентиляцию, а также одной потолочной лампой для искусственного освещения. В помещении имеется рабочий стол, персональный компьютер и оборудование стенда. Планировка помещения представлена на рисунке 3.1.

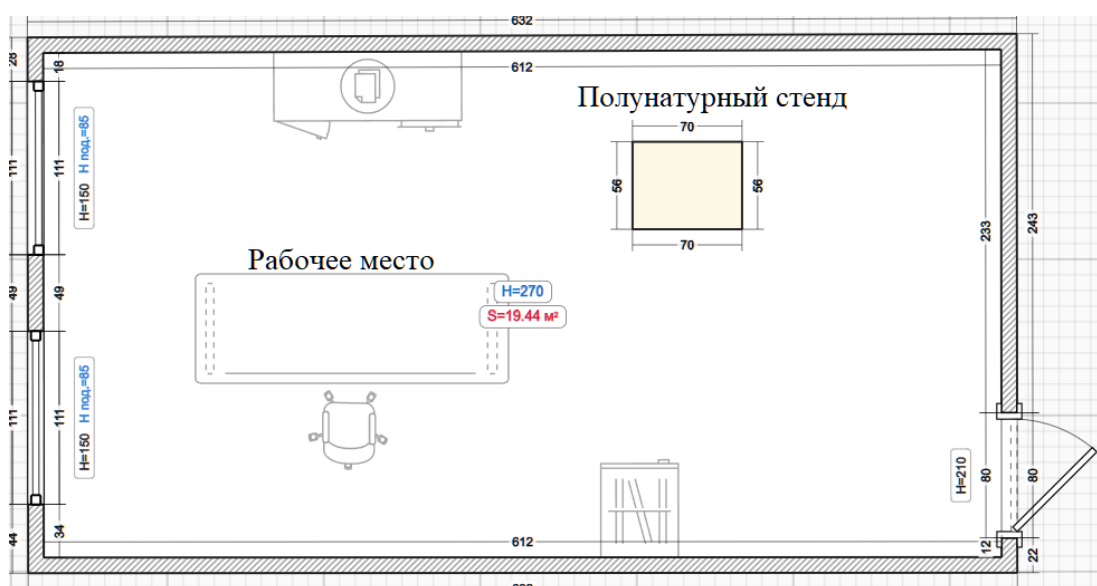


Рисунок 3.1 – Схема рабочего помещения

Полунатурный стенд, установленный в лаборатории, представляет собой механическую установку с тремя степенями свободы, имитирующую движения квадрокоптера по углам тангажа, крена и курса. В состав стенда входят электроприводы, контроллер, блок питания, а также датчики ориентации. Отказ элементов стенда, особенно при вращении, может привести к травмам или повреждению оборудования, что требует соблюдения норм электробезопасности, ограничения зоны вращения подвижных частей и применения защитных кожухов и ограждений. Инженер-программист перед началом работы обязан пройти вводный инструктаж по технике безопасности и раз в полгода проходить повторные инструктажи. Конструкция стенда показана на рисунке 3.2.

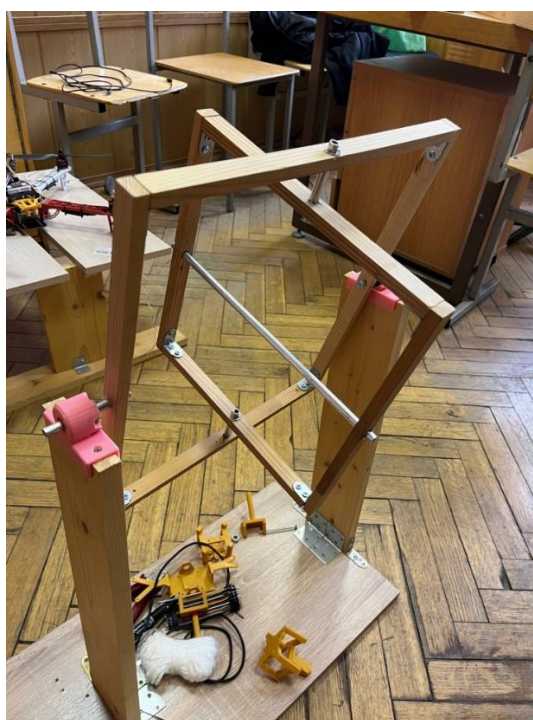


Рисунок 3.2 – Полунатурный стенд

3.2 Анализ условий труда

Оценка условий труда проводится в соответствии с Приказом Минтруда России от 21.11.2023 №817н [15], а также с использованием положений и норм:

- СанПиН 1.2.3685-21 «Гигиенические нормативы и требования к обеспечению безопасности труда» [10];
- СП 2.2.3670-20 «Санитарно-эпидемиологические требования к условиям труда» [11];
- СП 52.13330.2016 «Естественное и искусственное освещение» [12];
- ГОСТ 12.1.005-88 «ССБТ. Общие санитарно-гигиенические требования к воздуху рабочей зоны» [13];
- ГОСТ 12.0.003-2015 «Система стандартов безопасности труда. Опасные и вредные производственные факторы. Классификация» [14].

Безопасность труда является требованием при реализации любого инженерного и технологического процесса, особенно при проведении работ в лабораторной среде, где осуществляется сборка, наладка и испытание экспериментальных установок. В ходе создания полунатурного стенда для исследования динамики квадрокоптера, включающего операции по подготовке компонентов, монтажу электронных элементов, их пайке, подключению источников питания и проведению тестовых экспериментов, могут возникать определённые вредные и опасные производственные факторы. Их воздействие способно отрицательно сказываться на состоянии здоровья исполнителя, вызывать переутомление, профессиональные заболевания или травмы при нарушении правил безопасности.

Определение и оценка таких факторов выполняется в соответствии с требованиями ГОСТ 12.0.003-2015 «Система стандартов безопасности труда. Опасные и вредные производственные факторы. Классификация» [14]. Указанный стандарт регламентирует классификацию факторов по характеру их действия: физические, химические, биологические и психофизиологические, и подчёркивает необходимость их своевременного выявления для последующего анализа и разработки мер защиты.

Согласно СанПиН 1.2.3685-21 «Гигиенические нормативы и требования к обеспечению безопасности труда» [10], выполняемые в процессе разработки и эксплуатации полунатурного стенда для моделирования движения квадрокоптера, операции относятся к категории Ib

(таблица 3.1), то есть к лёгким работам, требующим незначительного физического напряжения и выполняемым преимущественно в сидячем положении. К данной категории относятся сборка и настройка электронных компонентов, пайка, тестирование и калибровка датчиков.

Таблица 3.1 – Классификация категорий работ по уровню энерготрат

Категория работ	Энерготраты, ккал/ч	Характер работ (примеры)
Ia	до 139	Сидячие работы, не требующие физического усилия (офис, анализ данных)
Iб	140–174	Лёгкие работы с незначительным физическим напряжением (пайка, сборка схем, настройка приборов)
IIa	175–232	Работы средней тяжести, включающие ходьбу и перенос мелких предметов (до 1 кг)
IIб	233–290	Работы средней тяжести с большей нагрузкой (перенос деталей до 10 кг)
III	свыше 290	Тяжёлые работы, требующие значительных усилий (перенос тяжёлых грузов, физический труд)

Для категории Iб общий класс условий труда оценивается как допустимый (класс 2), поскольку физические нагрузки минимальны, а воздействие вредных факторов имеет эпизодический характер. Биологические факторы в лабораторной деятельности отсутствуют.

Согласно требованиям СП 2.2.3670-20 «Санитарно-эпидемиологические требования к условиям труда» [11], производственные помещения, в которых выполняются работы категории Ia–Iб, должны удовлетворять следующим нормам:

- объём помещения на одного работника – не менее 15м³ при лёгких видах труда (Ia–Iб);
- площадь помещения – не менее 4,5 м² на одного человека;
- полы должны быть выполнены из материалов, устойчивых к воздействию химически активных веществ (флюс, припой и др.);

- входные зоны должны быть оборудованы приспособлениями для очистки обуви;
- системы вентиляции – обеспечивать удаление вредных веществ и предотвращать поступление холодного воздуха снаружи.

В лабораторном помещении, где осуществляется сборка и отладка стенда, указанные требования соблюдены: объём помещения составляет около 30 м³ на одного работника, площадь – порядка 10 м²; поверхность пола устойчива к воздействию паяльных материалов и легко очищается; вход оборудован ковриками для очистки обуви; естественная вентиляция через два окна обеспечивает воздухообмен и снижение концентрации вредных паров.

Такие условия способствуют снижению профессиональных рисков, обеспечивают комфортный микроклимат и безопасность при выполнении операций категории Ib, где физическая нагрузка невелика, но требуется надёжная вентиляция и соблюдение санитарно-гигиенических норм при работе с электронными компонентами и паяльным оборудованием.

Рассмотрим основные профессиональные риски на рабочем месте: какие вредные факторы присутствуют, как они влияют на здоровье, какие установлены нормативы и что можно сделать для защиты работников.

3.3 Микроклимат

В соответствии с требованиями СанПиН 1.2.3685-21 «Гигиенические нормативы и требования к обеспечению безопасности и (или) безвредности для человека факторов среды обитания» [10], микроклимат производственных помещений определяется такими параметрами, как температура воздуха, относительная влажность и скорость его движения. Данные показатели оказывают непосредственное влияние на самочувствие и

работоспособность человека, особенно при длительном пребывании в помещении.

Работа инженера-программиста, выполняемая в рамках данного проекта, относится к категории Іб – лёгкие физические работы, характеризующиеся умеренными энергозатратами (до 174 ккал/ч). Согласно санитарным нормам, для данной категории установлены оптимальные и допустимые значения микроклиматических параметров, приведённые в таблице 3.2.

Таблица 3.2 – Оптимальные и фактические параметры микроклимата для категории Іб (по СанПиН 1.2.3685-21 [10])

Категория работ	Показатель	Период года	СанПиН 1.2.3685-21	Фактические значения
Лёгкая – Іб	Температура воздуха, °С	Холодный	21–23	21–22
		Тёплый	22–24	20–22
	Относительная влажность, %	Холодный	40–60	40–55
		Тёплый	40–60	45–55
	Скорость движения воздуха, м/с	Холодный	≤ 0,1	0,1
		Тёплый	≤ 0,2	0,1

Как видно из таблицы, в тёплый период года температура воздуха в лаборатории находится в границах нормы. В то же время в холодный период температурные условия находятся ниже нижней границе допустимого диапазона, что может вызывать ощущение холода и снижение концентрации внимания оказывая отрицательного влияния на работоспособность персонала.

Таким образом, по показателям микроклимата в холодный сезон условия труда можно отнести к классу 3.1 (вредные) согласно классификации санитарных норм. Это означает наличие отклонения от оптимальных условий, которое при длительном воздействии способно вызвать дискомфорт, утомляемость и временное ухудшение самочувствия.

Для устранения выявленного несоответствия рекомендуется предусмотреть меры по нормализации микроклимата: установку системы кондиционирования с автоматическим регулированием температуры, либо

использование приточно-вытяжной вентиляции для обеспечения равномерного воздухообмена.

Реализация указанных мероприятий позволит поддерживать параметры микроклимата в пределах оптимальных санитарно-гигиенических норм, обеспечивая комфорт и безопасность труда инженера-программиста при работе со стендом и компьютерным оборудованием.

3.4 Шум

В помещении площадью 19,44 м² источниками шума являются системный блок ПК, двигатели квадрокоптера, а также внешний шум, проникающий через окна со стороны улицы.

К основным источникам относятся:

- Системный блок ПК – 35 дБА,
- Работающие двигатели квадрокоптера на стенде – до 50 дБА при испытаниях.
- Шум от улицы (через окна) – 40 дБА.

Для расчета суммарного уровня шума воспользуемся формулой:

$$L_{\text{сум}} = L_6 + \delta L, \quad (3.1)$$

где L_6 – больший уровень шума, дБА;

δL – добавка к большему уровню, определяемая по таблице в зависимости от разности уровней (рисунок 3.3).

Сложение уровней звукового давления													
Показатели	Значения показателей												
Разность двух складываемых уровней, дБ	0	1	2	3	4	5	6	7	8	9	10	15	20
Добавка к более высокому уровню для получения суммарного уровня, дБ	3	2,5	2	1,8	1,5	1,2	1	0,8	0,6	0,5	0,4	0,2	0

Рисунок 3.3 – Таблица сложение уровней звукового давления

Поскольку наибольший источник шума – двигатели квадрокоптера, а остальные источники создают меньший уровень, принимаем:

$$L_{\text{сум}} = 50 + 0.4 = 50.4 \text{ дБА}$$

Таким образом, фактический уровень шума в помещении составляет 50.4 дБА.

Согласно Приказу Минтруда №817н от 21.11.2023 [15] и СанПиН 1.2.3685-21 [10], данный уровень шума соответствует допустимому классу условий труда (2). Однако при длительных испытаниях стенда рекомендуется кратковременное ограничение работы стенда или использование звукоизолирующих материалов в зоне установки, чтобы минимизировать акустическую нагрузку на работников.

3.5 Освещенность

Рабочее место инженера-программиста, на котором осуществляется разработка ПО для управления стендом и подключением загрузчика данных к квадрокоптеру, расположено в лаборатории площадью 19,44 м². Работа выполняется за персональным компьютером с монитором разрешением full hd 1920×1080p, что позволяет отнести данный процесс к высокоточному

виду зрительной деятельности. Согласно требованиям СП 52.13330.2016 «Естественное и искусственное освещение» [12], минимальный размер различаемого объекта составляет 0,3–0,5 мм, контраст высокий, фон светлый. Разряд зрительной работы – IIIв.

Помещение оборудовано двумя оконными проёмами размером 111 см, однако из-за старых рам и частично закрытых ставней естественное освещение является недостаточным. В солнечные дни поток света ограничен загрязнением стекол и износом уплотнителей, что дополнительно снижает уровень освещенности. Искусственное освещение осуществляется одной потолочной лампой, световой поток которой невелик и не обеспечивает равномерного распределения света по рабочей зоне.

Для расчета фактической освещенности воспользуемся формулой:

$$E = \frac{(\Phi \times N \times \eta)}{(S \times K \times Z)}, \quad (3.2)$$

где Φ – световой поток одной лампы (4800 лм);

N – количество светильников (1 шт.);

η – коэффициент использования (0,52);

S – площадь помещения (19,44 м²);

K – коэффициент запаса (1,5);

Z – коэффициент неравномерности освещения (1,1).

$$E = \frac{(4800 \times 1 \times 0.52)}{(19.44 \times 1.5 \times 1.1)} = 76 \text{ лк.}$$

Таким образом, фактическая освещенность рабочего места составляет всего 76 лк, что значительно ниже нормативного значения 750 лк, установленного Приказом Минтруда России №817н от 21.11.2023 [15] для разряда зрительных работ IIIв.

Недостаточная освещенность приводит к повышенной зрительной утомляемости, снижению производительности и повышению вероятности ошибок при работе с программным кодом. По данному фактору условия труда относятся к вредному классу 3.1.

3.6 Тяжесть трудового процесса

Оценка тяжести трудового процесса проводится в соответствии с Приказом Министерства труда и социальной защиты Российской Федерации от 21.11.2023 г. №817н [15], приложение №13, где устанавливаются критерии отнесения условий труда к определённому классу по степени тяжести.

Работа инженера-программиста в данной лаборатории носит в основном умственный и статичный характер, однако периодически включает элементы физической активности, связанные с обслуживанием и настройкой полунатурного стенда. Эти действия требуют наклонов корпуса вперёд и работы в неудобной позе в течение коротких промежутков времени.

В течение 8-часового рабочего дня специалист выполняет около 21 000 стереотипных движений кистями и пальцами рук, что характерно для локальной динамической нагрузки при работе с клавиатурой и компьютерным оборудованием. Согласно Приложению 13 Приказа №817н, такие нагрузки относятся к допустимому классу 2.

Для определения влияния положения тела и наклонов корпуса учтены следующие наблюдения:

- продолжительность свободной позы сидя – до 85 % рабочего времени,
- количество наклонов корпуса при работе со стендом – до 40 раз за смену при угле наклона до 30°,
- продолжительность статического напряжения при обслуживании стенда – около 15 минут в час.

Сводные показатели тяжести трудового процесса приведены в таблице 3.3.

Таблица 3.3 – Оценка тяжести трудового процесса инженера-программиста

Показатель	Фактическое значение	Класс условий труда
Количество стереотипных движений кистей и пальцев рук	~21 000 движений за смену	2 (допустимый)
Свободное положение тела (сидя)	до 85 % рабочего времени	2 (допустимый)
Количество наклонов корпуса	до 40 наклонов/смену	2 (допустимый)
Продолжительность статического напряжения при работе со стендом	до 15 минут в час	2 (допустимый)

Учитывая совокупность факторов, тяжесть трудового процесса инженера-программиста относится к допустимому классу 2. Работа не вызывает выраженных физиологических отклонений и не требует специальных мероприятий по снижению физической нагрузки, однако рекомендуется проводить активные перерывы и выполнять разминку для рук и спины каждые 2 часа.

3.7 Напряженность трудового процесса

Оценка напряженности трудового процесса проводится в соответствии с Приказом Министерства труда и социальной защиты Российской Федерации от 21.11.2023 г. №817н [15]. Классификация условий труда по данному фактору осуществляется на основании показателей, характеризующих интеллектуальные, сенсорные и эмоциональные нагрузки, а также степень монотонности выполняемых операций.

Работа инженера-программиста с полунатурным стендом требует постоянного визуального контроля параметров, анализа поступающих данных и корректировки алгоритмов управления квадрокоптером. Несмотря на высокую степень ответственности и концентрации, деятельность не

связана с эмоциональным напряжением или воздействием внешних раздражителей (шум, сигналы, устные команды), что снижает общий уровень напряженности. Показатели напряженности трудового процесса приведены в таблице 3.4

Таблица 3.4 – Основные показатели напряженности трудового процесса

Показатель напряженности	Оптимальный показатель	Фактические значения	Класс условий труда
Плотность сигналов (звуковых, визуальных) в среднем за 1 час	до 75	72	1
Количество одновременно контролируемых параметров и объектов	до 5	4	1
Работа с оптическими приборами или монитором высокой точности	до 25	18	1
Нагрузка на голосовой аппарат	до 16	3	1
Количество элементов, необходимых для выполнения одной операции	более 10	11	1
Продолжительность однообразных действий, ч	менее 75	70	1

Показатели указывают, что напряженность труда находится в пределах оптимальных значений, а деятельность не сопровождается выраженными психоэмоциональными нагрузками.

С учётом результатов оценки можно заключить, что класс условий труда по фактору напряженности соответствует 1 (оптимальный). Работа не вызывает утомления психофизиологических функций и не требует специальных мер по снижению нагрузки, однако рекомендуется делать короткие перерывы для профилактики зрительного утомления и поддержания концентрации внимания.

3.8 Итоговый класс условий труда

На основании проведённого анализа факторов производственной среды и трудового процесса определены классы условий труда по каждому из них. Итоговые данные представлены в таблице 3.5.

Таблица 3.5 – Сводная оценка условий труда по отдельным факторам

Фактор	Класс / подкласс условий труда
Микроклимат	3.1
Шум	2
Освещённость	3.1
Тяжесть труда	2
Напряжённость труда	1
Итоговый класс условий труда	3.1

Согласно Приказу Минтруда России от 21.11.2023 №817н [15], общий класс условий труда определяется по наивысшему классу (подклассу), установленному хотя бы по одному фактору.

Поскольку по факторам микроклимата и освещённости установлены значения 3.1 (вредный класс), итоговая оценка условий труда на рабочем месте инженера-программиста также относится к вредному классу 3.1.

Такие условия труда допускаются при выполнении работ ограниченное время и требуют принятия мер по их нормализации – в частности, улучшения системы освещения и оптимизации микроклиматических параметров помещения с целью доведения их до допустимого уровня (класс 2).

3.9 Проектирование системы освещения

Для обеспечения нормативной освещённости рабочей поверхности в лаборатории площадью $S = 19,44 \text{ м}^2$ (разряд зрительной работы – IIIв, требуемая освещённость $E_{\text{норм}} = 700 \text{ лк}$) применим классический метод расчёта по коэффициенту использования светового потока.

Основная формула метода:

$$E = \frac{(\Phi \times N \times \eta)}{(S \times K \times Z)}$$

Переставим формулу для нахождения требуемого суммарного светового потока:

$$\Phi = \frac{(E \times S \times K \times Z)}{\eta}$$

Подставляем проектные параметры:

- $E=700$ лк;
- $S=19,44\text{м}^2$;
- $K=1,5$;
- $Z=1,1$;
- $\eta=0,60$.

Вычисление:

$$\Phi = \frac{(700 \times 19,44 \times 1,5 \times 1,1)}{0,6} = 37422\text{лм}$$

Итак, для обеспечения освещения на рабочем месте 700 лк по указанным допущениям требуется суммарный световой поток порядка ≈ 37400 лм.

Рекомендуется установить LED-панели 600×600, световой поток ≈ 6000 лм (мощность $\sim 40\text{--}50$ Вт)

- Необходимое количество: $N = 37400/6000 \approx 6,2 \Rightarrow 6$ шт;
- Рекомендуется установить 6 панелей и распределить равномерно по потолку.

На рисунке 3.4 представлено расположение панелей над рабочим местом инженера-программиста

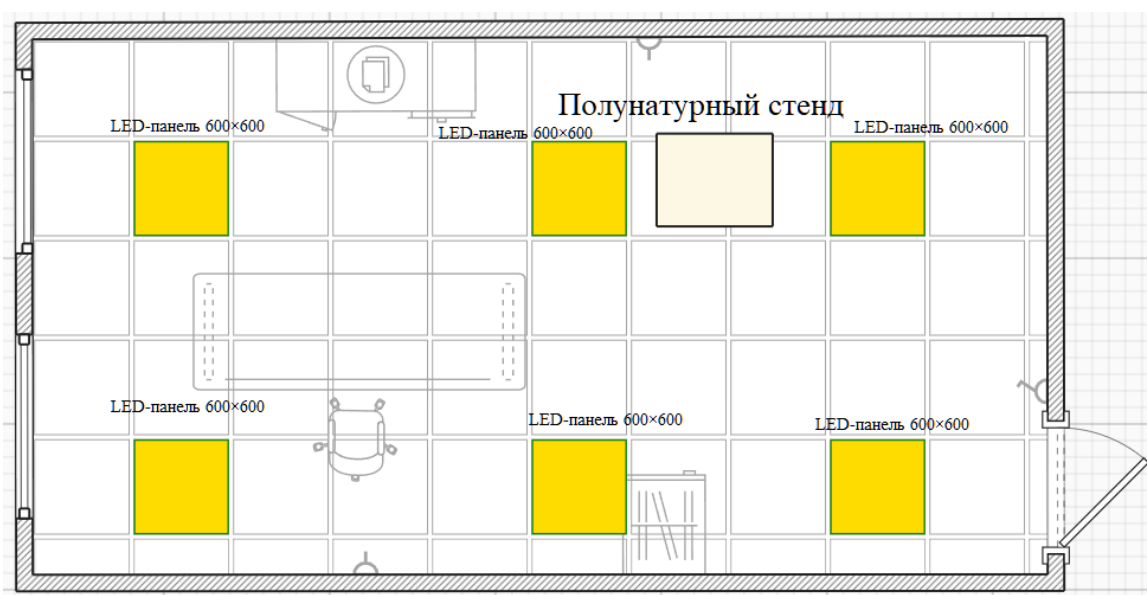


Рисунок 3.4 – Расположение ламп на рабочем месте

3.10 Вывод

В разделе «Охрана труда и окружающей среды» была проведена оценка условий труда инженера-программиста, работающего с полунатурным стендом для отладки и испытаний алгоритмов управления квадрокоптером. Анализ включал изучение микроклимата, уровня шума, освещённости, тяжести и напряжённости трудового процесса.

Для улучшения условий труда был проведен расчет системы освещения, подобраны современные светодиодные светильники, обеспечивающие равномерное распределение яркости в зоне стенда и на рабочих поверхностях.

Введение предложенных мероприятий позволит повысить освещённость на рабочем месте до нормативных значений, снизить зрительное напряжение и повысить комфорт при выполнении программно-алгоритмических и экспериментальных работ. Реализация предложенных мер способствует снижению количества ошибок и повышению качества проводимых работ.

4. Экономическая часть

4.1 Введение

Цель экономической части дипломной работы – выполнение технико-экономического обоснования разработанной системы полунатурного моделирования и программно-алгоритмического обеспечения для автономного управления квадрокоптера с использованием стенда с тремя степенями свободы и виртуальной среды AirSim. В рамках данного раздела необходимо определить эффективность внедрения создаваемой НТПр по сравнению с существующими аналогами, оценить затраты на её разработку и прогнозируемую окупаемость.

В данном разделе решаются следующие задачи:

- Оценка целесообразности разработки и её научно-технической прогрессивности;
- Оценка экономической эффективности разрабатываемой системы;
- Планирование работ по созданию стенда и программной части, а также распределение затрат по этапам;
- Обоснование преимуществ применения полунатурного моделирования вместо полевых испытаний.

Чтобы оценить научно-техническую продукцию (НТПр), необходимо рассчитать показатели научно-технической прогрессивности и новизну.

4.2 Оценка целесообразности выполнения разработки на основе определения ее технической прогрессивности

4.2.1 Оценка научно-технической прогрессивности

Технико-экономическое обоснование целесообразности выполненной работы представляет из себя расчет оценки целесообразности разработки. Чтобы провести оценку целесообразности необходимо выполнить анализ

результатов работы с целью определения и оценки интегрального показателя научно-технической прогрессивности.

Для обоснования целесообразности разработки необходимо:

- выбрать аналог разрабатываемого продукта для того, чтобы использовать его для сравнения с НТПр,
- сформулировать перечень характеристик качества разработки по предлагаемому программному продукту (ПП),
- определить конкретные значения характеристик и их значимость.

Оценка научно-технической прогрессивности разрабатываемой НТПр осуществляется на основе сравнения её характеристик с характеристиками существующего аналога. Для проведения оценки используется метод балльных оценок и нормативные таблицы показателей научно-технического уровня НТПр (таблица 4.1), перспективности НТПр (таблица 4.1) и возможности применения результатов НТПр (таблица 4.2).

Таблица 4.1 – Ориентировочная шкала оценки признаков научно-технического эффекта НТПр в баллах

Признак научно-технического эффекта НТПр	Возможное определение научно-технического эффекта НТПр	Оценка в баллах
Научно-технический уровень (новизна)	Превышает мировые достижения	9–10
	Находится на уровне мировых достижений	6–8
	Приближается к мировым достижениям	3–5
	Тривиальный	1–2
Перспективность	Первостепенная важность	8–10
	Важная	5–7
	Полезная	1–4

Таблица 4.2 – Ориентировочная шкала оценки возможности применения результатов НТПр в баллах

Период возможного применения результатов НТПр	Область применения НТПр		
	Народное хозяйство	Отрасль	Предприятие
До 3 лет	10	8	4
3–5 лет	8	7	3
6–10 лет	6	5	2
Более 10 лет	4	3	1

Таблицы 4.1 и 4.2 используются в качестве базы для сравнения, на основе которых заполняется таблица 4.3.

В качестве аналога для оценки научно-технической прогрессивности выбрана Gazebo – типовая виртуальная система моделирования квадрокоптера, использующая только программный симулятор без взаимодействия с реальными датчиками и без физического полунатурного стенда. Такие решения обеспечивают тестирование исключительно в моделируемой среде, не учитывают реальных динамических свойств квадрокоптера и не позволяют сочетать физическое движение с виртуальной моделью.

Предлагаемая в дипломе НТПр включает:

- полунатурный стенд с тремя степенями свободы,
- получение реальных данных MPU-датчиков и передача их через Bluetooth,
- интеграцию с AirSim,
- возможность безопасного тестирования автономных систем управления.

Таблица 4.3 – Научно-технический эффект НТПр

Признак научно-технического эффекта	Значимость признаков	Уровень свойств НТПр (аналог)	Уровень свойств разрабатываемой НТПр
Научно-технический уровень (новизна)	0,4	6	7
Перспективность	0,4	7	9
Возможность применения результатов	0,2	4	7

Расчёт обобщённого показателя научно-технического уровня разработки:

$$H_{НТПН} = \frac{\sum_i Q_{Hi} R_i}{\sum_i R_i}, \quad (4.1)$$

где Q_{Hi} – значения показателей НТПр в баллах;
 R_i – весовые коэффициенты показателей.

$$H_{НТПН} = 7 \times 0,4 + 9 \times 0,4 + 7 \times 0,2 = 7,8$$

Для базового варианта:

$$H_{НТПБ} = \frac{\sum_i Q_{Bi} R_i}{\sum_i R_i} = 6 \times 0,4 + 7 \times 0,4 + 4 \times 0,2 = 6$$

где Q_{Bi} – показатели НТПр-аналога.

Индекс научно-технической прогрессивности:

$$J_{НТП} = \frac{H_{НТПН}}{H_{НТПБ}}, \quad (4.2)$$

$$J_{НТП} = \frac{7,8}{6} = 1,3.$$

Полученное значение $J_{НТП} > 1$ свидетельствует о высокой научно-технической прогрессивности и целесообразности разработки. Разработка превосходит аналогичные решения по уровню новизны, возможностям применения и перспективности использования, что подтверждает целесообразность её реализации и внедрения.

4.2.2 Оценка новизны НТПр и уровня научно-технического прогресса (НТП)

Оценка новизны научно-технического предложения проводится на основе системы балльных критериев. Каждый критерий характеризует

степень отличия предлагаемого решения от существующих аналогов. Баллы выставляются согласно таблице 4.4.

Таблица 4.4 – Оценка новизны предложения

Критерий новизны	Характер новизны	Баллы
Функциональный	Принципиально новое	5
Технологический	Новое	3
Производственный	Улучшенное	2
Социальный	Улучшенное	2
Юридический	Улучшенное	2

Суммарная оценка новизны определяется как:

$$B_k = 5 + 3 + 2 + 2 + 2 = 17 \text{ баллов (сумма набранных баллов),}$$

$B_m = 5 \times 5 = 25$ баллов (максимально возможная сумма по всем критериям).

Новизна определяется соотношением:

$$H = \frac{B_k}{B_m}, \quad (4.3)$$

$$H = \frac{17}{25} = 0,68.$$

Полученное значение свидетельствует о среднем уровне новизны, что характерно для прикладных проектов, ориентированных на внедрение улучшенных методов и технологий в рамках существующей технической базы.

Уровень научно-технического прогресса определяется с учётом темпов накопления новых знаний и рассчитывается по выражению:

$$I_{НТП} = e^{at+b}, \quad (4.4)$$

где $a = \frac{(\ln 2)}{5} = 0,139$ – постоянный коэффициент для промышленности;

$t = 1$ – период оценки, лет;

$b = \ln J_{\text{НТП}} = \ln 1,3 = 0,262$ – логарифм базового уровня НТП.

$$I_{\text{НТП}} = e^{0,139 \times 1 + 0,262} = e^{0,401} = 1,49$$

Снижение эффективности технического решения во времени определяется по формуле:

$$K_T = \frac{K_0}{(1 + J_{\Delta \text{ВГ}})^T}, \quad (4.5)$$

где $K_0 = 1$ – первоначальная стоимость;

$J_{\Delta \text{ВГ}} = 0,15$ – средний годовой прирост производительности труда;

$T = 2$ – срок эксплуатации, лет.

$$K_T = \frac{1}{(1 + 0,15)^2} = 0,756$$

Период эффективности для новых алгоритмов и программного обеспечения составляет 1-2 года, что соответствует полученным расчетам.

4.3 Планирование разработки

Планирование разработки – это определение трудоемкости работ, необходимых для создания программного обеспечения (ПО). Планирование выполнения разработки нужно для определения трудоемкости разработки, выявлении состава работ, необходимых для выполнения конкретной

разработки, в расчете параметров, необходимых для построения сетевого графика разработки

4.3.1 Определение трудоемкости разработки

Данный дипломный проект включает в себя традиционный метод программирования, когда каждый ПП содержит все этапы решения задач (комплексов задач), начиная с ввода исходных данных, и кончая выводом результатов, поэтому общая трудоемкость реализации ПП, выраженная через параметры объектов, алгоритмов ПП и базовые трудоемкости программирования типовых процедур, рассчитывается по формуле:

$$t_{\text{пп}} = t_0 + t_{\text{и}} + t_{\text{А}} + t_{\text{К}} + t_{\text{отл}} + t_{\text{Д}}, \quad (4.6)$$

где t_0 – затраты труда на подготовку описания задачи, комплекса задач;
 $t_{\text{и}}$ – затраты труда на изучение и постановку задачи, комплекса задач;

$$t_{\text{и}} = \frac{Q * B}{75 * K}$$

$t_{\text{А}}$ – затраты труда на разработку алгоритма решения задачи, комплекса задач;

$$t_{\text{А}} = \frac{Q}{20 * K}$$

$t_{\text{К}}$ – затраты труда на написание кода программы;

$$t_{\text{к-}} = \frac{Q}{10 * K}$$

$t_{\text{отл}}$ – затраты труда на отладку программы;

$$t_{\text{отл}} = \frac{Q}{5 * K}$$

$t_{\text{д}}$ – затраты труда на подготовку документации по ПП;

$$t_{\text{д}} = \frac{1,75 * Q}{15 * K}$$

Q – условное количество операторов(строк) в машинной программе;

$$Q = q * K_c * (1 + K_{k1} + \dots + K_{kn})$$

где $q = 400$ – количество этапов и элементарных процедур преобразования информации (вычислительных, логических и других процедур);

$K_c = 1,5$ – коэффициент сложности программы;

$K_k = 0,5$ – коэффициент коррекции, при разработке $K_k = 0,05 \dots 0,5$;

$n = 5$ – количество коррекций;

$K = 1$ – коэффициент квалификации разработчика программиста;

$B = 1,2 \dots 3$ – увеличение затрат на изучение и постановку задачи вследствие ее сложности и новизны.

Базовые значения трудоемкости зависят от квалификации программиста и среды программирования.

Исходя из выше приведенных формул можно рассчитать трудовые затраты, подставив значения в формулу 4.3, тогда получим следующие значения:

$$Q = 400 * 1,5 * (1 + 0,1 * 5) = 900 \text{ строк}$$

$$t_{\text{И}} = \frac{900 * 1,5}{75 * 1} = 18 \text{ чел./час}$$

$$t_{\text{А}} = \frac{900}{20 * 1} = 45 \text{ чел./час}$$

$$t_{\text{К}} = \frac{900}{10 * 1} = 90 \text{ чел./час}$$

$$t_{\text{отл}} = \frac{900}{5 * 1} = 180 \text{ чел./час}$$

$$t_{\text{Д}} = \frac{900 * 1,75}{15 * 1} = 105 \text{ чел./час}$$

Тогда общая трудоемкость определяется как:

$$t_{\text{пп}} = 18 + 45 + 90 + 180 + 105 = 438 \text{ чел./час}$$

4.3.2 Календарное планирование

Календарное планирование работ выполняется на основе предварительно определённой трудоёмкости каждого этапа разработки программно-алгоритмического обеспечения для полунатурного стенда управления квадрокоптером. В качестве входных данных используются результаты распределения трудозатрат по стадиям, представленные в предыдущем пункте, а также нормативные параметры рабочего времени.

Цель календарного планирования – определить длительность производственного цикла выполнения работ на каждом этапе и сформировать общий временной график реализации научно-технического проекта.

Трудоёмкость стадий учитывает специфику работ: разработка алгоритмов ориентации, моделирование в AirSim, интеграция аппаратных модулей, настройка связи со стендом, отладка и тестирование.

Для расчёта длительности производственного цикла j -ой стадии используется следующая формула:

$$T_{ц.j} = \frac{T_j}{t_{рд} q_j}, \quad (4.7)$$

где T_j – трудоёмкость j -ой стадии, чел./час;

$t_{рд}$ – продолжительность рабочего дня, час;

q_j – количество исполнителей, одновременно участвующих в выполнении работ на этапе, чел.

Результаты календарного планирования приведены в таблице 4.5

Таблица 4.5 – Расчет календарного плана работ

№ ста дии	Наименование стадий (этапов) работ	Удельный вес, %	Трудоёмкость стадии (этапа), чел./час	Количество исполнителей (при параллельных работах)	Длительность стадии, календ. дни
1	Анализ исходных данных, разработка ТЗ	5	9	2	5
2	Анализ ТЗ, принятие решений	5	9	2	5
3	Разработка алгоритмов и ПП	45	45	2	45
4	Разработка аппаратной части	20	90	2	20
5	Отладка ПП	20	90	2	20

Продолжение таблицы 4.5

6	Проведение эксперимента	4	90	2	4
7	Создание технической документации	1	105	2	1
	Итого	100	438		100

Графическое построение календарного графика проводится в осях: календарные даты, трудоемкость и даты окончания каждой стадии (этапа) работ. На рисунке 4.1 представлен календарный график, с помощью которого можно визуально оценить трудоемкость на каждой стадии проведенных работ.



Рисунок 4.1 – Календарный график

4.4 Определение затрат, себестоимости и цены

Определение затрат на ПП начинается с определения затрат на оплату труда сотрудника. Трудоемкость создания работы распределяется по основным этапам работ в соответствие со структурой трудовых затрат, на которых определяются ее исполнители и показатели оплаты труда. При наличии нескольких исполнителей, ставка оплаты труда определяется как средневзвешенная.

Зарплата персонала по стадиям (этапам) работ рассчитывается по формуле:

$$З_{зп. j} = T_j \overline{\tau_j}, \quad (4.8)$$

где T_j – трудоёмкость этапа в чел./дня;

$\overline{\tau_j}$ – средняя дневная тарифная ставка оплаты работ j -го этапа работы, руб.

Результаты расчетов затрат на оплату труда сотрудников за каждый выполненный этап с учетом премиальных выплат, которые составляют 20% приведены в таблице 4.6

Таблица 4.6 – Расчет заработной платы специалистов за каждый этап работ

№ эта па	Трудоёмкость, чел./дн.	Исполнители		Ставка, руб./день	Заработная плата, руб.	Заработ ная плата с премией , руб.
		Числ (чел)	Должность			
1	52	2	К.т.н. доцент кафедры	500	26000	31200
			Инженер- программи ст			
2	65	2	К.т.н. доцент кафедры	500	32500	39000
			Инженер- программи ст			
3	130	2	К.т.н. доцент кафедры	500	65000	78000
			Инженер- программи ст			
4	260	2	К.т.н. доцент кафедры	500	13000	156000

Продолжение таблицы 4.6

			Инженер-программист			
5	151	1	Инженер-программист	500	75500	90600
ИТОГО					330000	396000

Итоговая величина заработной платы сотрудников за все этапы работы составляет:

$$З_{зп} = 330000 \text{ руб.}$$

Итоговая величина заработной платы сотрудников за все этапы работы с учетом премиальных выплат составляет:

$$З_{зп+п} = 396000 \text{ руб.}$$

Для того, чтобы определить себестоимость НТПр, необходимо определить затраты на создание ПП, которое состоит из:

- Заработной платы основных исполнителей ($З_{зп}$);
- Отчисление на страховые взносы (30,2% от $З_{зп}$);
- Накладных расходов ($P_{накл}$) определяется по формуле:

$$P_{накл} = З_{зп} * K_{накл} \quad (4.9)$$

где $K_{накл}$ – коэффициент накладных расходов, для ПП принимается на уровне $1 \leq K_{накл} \leq 2$. Примем $K_{накл} = 1,5$.

Подставив значение коэффициента в формулу 4.7 получаем:

$$P_{накл} = 330000 * 1,5 = 495000 \text{ руб.}$$

Отчисление на страховые взносы (30,2%):

$$З_{страх} = \frac{З_{зп} * 30,2\%}{100\%}$$

Тогда:

$$З_{\text{страх}} = 396000 * 0,302 = 119592 \text{ руб.}$$

Прочие затраты зависят от объема и вида работ и составляют 10% от заработной платы:

$$З_{\text{пр}} = \frac{З_{\text{зп}} * 10\%}{100\%}$$

Тогда:

$$З_{\text{пр}} = 330000 * 0,1 = 33000 \text{ руб.}$$

Затраты на создание программного обеспечения представлены в таблице 4.7

Таблица 4.7 – Поэлементная структура затрат на ПП

Номер п/п	Наименование статей затрат	Затраты (руб.)	Удельный вес (%)
1	Заработная плата с премией	396000	39
2	Страховые взносы	119592	10
3	Накладные расходы	495000	48
5	Прочие расходы	33000	3
Итого		$З_{\text{зп}} = 1043592$	100

Цена на продукцию первоначально разработанного ПП ($Ц_{\text{пп}}$), определяется с учетом рентабельности разработки по формуле:

$$Ц_{ПП} = З_{ПП} + \frac{З_{ЗП} \rho_{ЗП}}{100}, \quad (4.10)$$

где $З_{ПП}$ – затраты на создание программных продуктов;

$З_{ЗП}$ – заработная плата основных исполнителей – разработчиков ПП;

$\rho_{ЗП} = 200\%$ – рентабельность разработки ПП по отношению к оплате труда основных исполнителей, обеспечивающих безубыточную деятельность.

Таким образом, получаем:

$$Ц_{ПП} = 1043592 + \frac{330000 \cdot 200}{100} = 1703592 \text{ руб.}$$

4.5 Определение и оценка показателей экономической эффективности

Основными показателями экономической эффективности являются: годовой экономический эффект ($\mathcal{E}$), уровень экономической эффективности (E), срок окупаемости вложений (T_{OK}). Экономический эффект определяется как годовая экономия на текущих расходах, причем во внимание принимаются изменения статьи затрат.

Разрабатываемый программный продукт автоматизирует процессы моделирования, ускоряет проверку алгоритмов ориентации, сокращает время поиска ошибок, интеграции и тестирования, а также снижает трудозатраты на проведение виртуальных экспериментов, заменяя часть реальных испытаний в стендовых условиях.

Для оценки экономической эффективности создаваемого ПП необходимо выяснить механизм его действия на экономические показатели в сферах применения ПП. Так как использование разработанного алгоритма ПП приведет к сокращению времени решения комплекса задач по моделированию движения квадрокоптера, то будем использовать формулу:

$$\Delta_{\text{ППГ}} = \sum_{i=1}^n \Delta T_{Mi} * C_{BT}, \quad (4.11)$$

где ΔT_{Mi} – экономия машинного времени по i -ой задаче, час.;

C_{BT} – стоимость одного машинного часа, руб.;

n – количество задач (комплекса задач), решаемых за год.

Для определения экономического эффекта предположим, что данный ПП используют в среднем 1825 раз за год (5 тестов ежедневно). Стоимость одного машинного часа приблизительно 120 рублей. Для решения одной задачи аналогу требовалось 40 минут, в то время как новому продукту для решения аналогичной задачи требуется лишь 20 минут, так как значительно повышен уровень автоматизации стенда, интеграции с AirSim и обработки данных датчиков ИМИ MPU-6050, следовательно, экономия машинного времени увеличивается в 2 раза.

Исходя из вышеописанных условий, получаем исходные данные:

$$n = 1825, C_{BT} = 120, \Delta T_{Mi} = 2.$$

При постановке значений получаем:

$$\Delta_{\text{ППГ}} = 1825 * 2 * 120 = 438000 \text{ руб.}$$

Оценим уровень экономической эффективности разработки ПП и срок окупаемости затрат на эту разработку.

Экономическая эффективность:

$$E_{\text{ПП}} = \frac{\Delta_{\text{ППГ}}}{C_{\text{ПП}}} \quad (4.12)$$

где $\Delta_{\text{ППГ}}$ – экономический эффект разработки;

$C_{\text{ПП}}$ – цена разработки, руб.

Уровень экономической эффективности затрат на разработку имеет значение:

$$E_{\text{ПП}} = \frac{438000}{1703592} = 0,257.$$

Срок окупаемости, лет:

$$T_{\text{ОК}} = \frac{1}{E_{\text{ПП}}} \quad (4.13)$$

$$T_{\text{ОК}} = \frac{1}{0,26} = 3.9 \text{ года}$$

Срок окупаемости ПП 4 года.

4.6 Вывод по оценке экономической эффективности разработки программного продукта

В экономической части дипломного проекта проведено технико-экономическое обоснование создания программного обеспечения для трёхмерной визуализации и моделирования движения квадрокоптера на основе полунатурного стенда с тремя степенями свободы и виртуальной среды AirSim, интегрированной с движком Unreal Engine.

При выполнении работы был определен индекс научно-технической прогрессивности. Он имеет больший уровень научно-технической прогрессивности, чем НТПр, выбранная в качестве аналога, следовательно, разработка новой продукции целесообразна.

В результате расчетов были получены следующие значения:

- Индекс научно-технического уровня $J_{\text{НТП}} = 1,3$;
- Затраты на создание ПП $З_{\text{ПП}} = 1043592$ руб.;
- Экономическая эффективность $Э_{\text{ППГ}} = 438000$ руб.;
- Показатель экономической эффективности $E_{\text{ПП}} = 0,257$.

При таком значении экономической эффективности срок окупаемости ПП составит 4 года.

Таким образом, разработка программного обеспечения для полунатурного моделирования квадрокоптера является научно-технически прогрессивной, экономически целесообразной и востребованной. Использование созданного ПО снижает трудозатраты инженеров, уменьшает стоимость экспериментов, увеличивает скорость разработки и проверки алгоритмов автономного управления квадрокоптера.

ЗАКЛЮЧЕНИЕ

В рамках данного дипломного проекта была рассмотрена и решена актуальная задача разработки интегрированной системы полунатурного моделирования квадрокоптера, предназначенной для исследования, отладки и тестирования алгоритмов управления и детекции препятствий в условиях, приближенных к реальным. Актуальность работы обусловлена возрастающим интересом к автономным беспилотным летательным аппаратам, а также необходимостью повышения безопасности и эффективности их эксплуатации в сложной и неопределённой среде.

В ходе выполнения работы был проведён анализ современных подходов к детекции препятствий и автономной навигации квадрокоптеров, а также рассмотрены методы моделирования и визуализации движения летательных аппаратов. Анализ показал, что наиболее перспективным направлением является использование мультисенсорных систем в сочетании с алгоритмами компьютерного зрения и нейросетевыми методами обработки данных, а также применение виртуальных сред моделирования для отладки и верификации алгоритмов управления.

Основным практическим результатом дипломного проекта стала разработка полунатурного стенда с тремя степенями свободы, предназначенного для исследования углового положения квадрокоптера по каналам тангажа, крена и курса. Конструкция стенда отличается простотой, модульностью и устойчивостью, что позволяет использовать его в лабораторных условиях без значительных затрат на изготовление и обслуживание. Применение 3D-печатных элементов и подшипников обеспечило необходимую точность вращения и надёжность конструкции.

В состав разработанной системы были интегрированы квадрокоптер, микроконтроллер ESP32 и инерциальный датчик MPU-6050, обеспечивающие измерение ориентации и передачу данных в режиме реального времени. Получаемые данные используются для визуализации

движения квадрокоптера в виртуальной среде AirSim на базе игрового движка Unreal Engine. Такое решение позволило реализовать гибридную модель, сочетающую реальные аппаратные компоненты и виртуальную среду, что значительно расширяет возможности экспериментальных исследований и снижает риски при реальных испытаниях.

Отдельное внимание в работе было уделено вопросам охраны труда и обеспечения безопасных условий работы. Проведённый анализ условий труда показал необходимость улучшения параметров освещённости рабочего места. В рамках раздела были выполнены расчёты системы освещения и предложены современные светодиодные светильники, обеспечивающие соответствие нормативным требованиям. Реализация предложенных мероприятий способствует снижению зрительного напряжения, повышению комфорта труда и, как следствие, улучшению качества выполняемых работ и снижению вероятности ошибок при проведении экспериментов.

В экономической части дипломного проекта было выполнено технико-экономическое обоснование разработки программного обеспечения для трёхмерной визуализации и моделирования движения квадрокоптера. Проведённые расчёты показали, что разработка обладает высоким уровнем научно-технической прогрессивности и является экономически целесообразной. Полученные значения показателей экономической эффективности свидетельствуют о возможности практического внедрения разработанного программного продукта и его конкурентоспособности по сравнению с существующими аналогами.

Цель дипломного проекта была достигнута, а поставленные задачи – выполнены в полном объёме. Разработанная интегрированная система полунатурного моделирования может быть использована как в научно-исследовательских целях, так и в образовательной практике для изучения принципов управления и навигации квадрокоптеров.

Перспективы дальнейшего развития работы связаны с расширением функциональных возможностей стенда, интеграцией дополнительных

сенсоров, реализацией более сложных алгоритмов управления и обхода препятствий, а также применением методов машинного обучения и адаптивного управления. Полученные в ходе выполнения дипломного проекта результаты могут служить основой для дальнейших исследований и практических разработок в области автономных беспилотных летательных аппаратов.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Jinpeng Zhang, Honghai Zhang, Jinlun Zhou, Mingzhuang Hua, Gang Zhong “Adaptive Collision Avoidance for Multiple UAVs in Urban Environments” – 2023
2. Hartmut Fricke, Stanley Förster, Robert Bruehl “Mid-air collisions with drones Assessment of collision scenarios and of drone operation risks in urban areas” – 2021
3. Ahmad Merei, Hamid Mcheick, Alia Ghaddar “A Survey on Obstacle Detection and Avoidance Methods for UAVs” – 2025, [Электронный ресурс] URL: <https://doi.org/10.3390/drones9030203>, (дата обращения: 22.10.2025)
4. Ulzhalgas Seidaliyeva, Lyazzat Iipbayeva, Dana Utebayeva, Nurzhigit Smailov, Eric T Matson, Yerlan Tashtay, Mukhit Turumbetov “LiDAR Technology for UAV Detection: From Fundamentals and Operational Principles to Advanced Detection and Classification Techniques” – 2025; [Электронный ресурс] URL: <https://doi.org/10.3390/s25092757>, (дата обращения: 26.10.2025)
5. Gang Wang, Yanfei Chen, Pei An, Hanyu Hong, Jinghu Hu, Tiange Huang “UAV-YOLOv8: A Small-Object-Detection Model Based on Improved YOLOv8 for UAV Aerial Photography Scenarios” – 2025; [Электронный ресурс] URL: <https://doi.org/10.3390/s23167190>, (дата обращения: 22.10.2025)
6. Флах П. Машинное обучение. Наука и искусство построения алгоритмов, которые извлекают знания из данных. ДМК Пресс, 2015. 402 с.
7. Ultralytics. 2023 “YOLOv8 Docs.” [Электронный ресурс] URL: <https://docs.ultralytics.com/ru>, (дата обращения: 13.11.2025)
8. Редько, В.Г. Эволюция, нейронные сети, интеллект: Модели и концепции эволюционной кибернетики / В.Г. Редько. - М.: Ленанд, 2015. - 224 с.
9. Welcome to AirSim / [Электронный ресурс] // GitHub : [сайт]. – URL: <https://github.com/madratman/AirSim/blob/docs/docs/docs/index.rst>, (дата обращения: 13.11.2025)

10. СанПиН 1.2.3685-21 «Гигиенические нормативы и требования к обеспечению безопасности труда»
11. СП 2.2.3670-20 «Санитарно-эпидемиологические требования к условиям труда»
12. СП 52.13330.2016 «Естественное и искусственное освещение»
13. ГОСТ 12.1.005-88 «ССБТ. Общие санитарно-гигиенические требования к воздуху рабочей зоны»
14. ГОСТ 12.0.003-2015 «Система стандартов безопасности труда. Опасные и вредные производственные факторы. Классификация»
15. Приказом Минтруда России от 21.11.2023 №817н
16. Unreal Engine версии 4.27 / [Электронный ресурс] // Unreal Engine : [сайт]. – URL: <https://www.unrealengine.com/en-US/blog/unreal-engine-4-27-released>, (дата обращения: 20.11.2025)

ПРИЛОЖЕНИЕ А

Программный модуль управления дроном в среде AirSim

```
import socket
import os
import time
import math
import threading
import sys
from datetime import datetime
from dataclasses import dataclass
from typing import Optional, Tuple, Dict, Any
import airsims

# Для Windows используем msvcrt, для Linux - select
try:
    import msvcrt
    WINDOWS = True
except ImportError:
    import select
    import termios
    import tty
    WINDOWS = False

# Импорты для YOLOv8
import cv2
import numpy as np
import torch
from ultralytics import YOLO
from collections import defaultdict
from threading import Thread, Lock

# =====
# КОНСТАНТЫ И КОНФИГУРАЦИЯ
# =====

class Config:
    """Конфигурация программы"""
    ESP32_IP = "172.20.10.3"
    PORT = 3333
    LOG_DIR = "logs"
    TARGET_ALTITUDE = -10 # Высота в AirSim (отрицательная = вверх)
    MIN_THROTTLE = 1000
    MAX_THROTTLE = 2000
    DEFAULT_THROTTLE = 1100
    MAX_ANGLE_LIMIT = 60.0
    MIN_SPEED_THRESHOLD = 0.5
    DEAD_ZONE_DEFAULT = 8.0
    SENSITIVITY_DEFAULT = 1.5
    SOCKET_TIMEOUT = 0.1
    MAIN_LOOP_SLEEP = 0.05
    KEEP_ALIVE_INTERVAL = 0.5

# YOLOv8 настройки
YOLO_MODEL_PATH = r"C:\Users\mkravtsov\Desktop\diplomaa\diplomaa\Best.pt"
YOLO_CONFIDENCE = 0.5
YOLO_FRAME_WIDTH = 640
YOLO_FRAME_HEIGHT = 360

# =====
# КЛАСС ДЛЯ YOLOv8 ДЕТЕКЦИИ
# =====

class YOLOv8Detector:
    """Класс для детекции объектов с помощью YOLOv8"""

    def __init__(self, model_path: str = Config.YOLO_MODEL_PATH,
                 conf_threshold: float = Config.YOLO_CONFIDENCE):
        self.model = YOLO(model_path)
        self.conf_threshold = conf_threshold
        self.detection_active = False
        self.current_frame = None
        self.detection_results = []
```

```

self.lock = Lock()
self.detection_thread = None
self.camera_client = None

# Проверка устройства
self.device = 'cuda' if torch.cuda.is_available() else 'cpu'
print(f"YOLOv8 использует устройство: {self.device}")
self.model.to(self.device)

# Цвета для аннотаций
self.colors = [
    (255, 0, 0), (0, 255, 0), (0, 0, 255), (255, 255, 0), (0, 255, 255),
    (255, 0, 255), (192, 192, 192), (128, 128, 128), (128, 0, 0), (128, 128, 0),
    (0, 128, 0), (128, 0, 128), (0, 128, 128), (0, 0, 128), (72, 61, 139),
    (47, 79, 79), (47, 79, 47), (0, 206, 209), (148, 0, 211), (255, 20, 147)
]

def connect_to_airsim(self, airsims_client):
    """Подключение к камере AirSim"""
    self.camera_client = airsims_client

def start_detection(self):
    """Запуск детекции в отдельном потоке"""
    if self.detection_active:
        return

    self.detection_active = True
    self.detection_thread = Thread(target=self._detection_loop, daemon=True)
    self.detection_thread.start()
    print("YOLOv8 детекция запущена")

def stop_detection(self):
    """Остановка детекции"""
    self.detection_active = False
    if self.detection_thread:
        self.detection_thread.join(timeout=2)
    print("YOLOv8 детекция остановлена")

def _detection_loop(self):
    """Основной цикл детекции"""
    while self.detection_active:
        try:
            # Получение кадра из AirSim
            if self.camera_client:
                responses = self.camera_client.simGetImages([
                    airsims.ImageRequest("0", airsims.ImageType.Scene, False, False)
                ])

            if responses and responses[0]:
                # Конвертация изображения
                img1d = np.frombuffer(responses[0].image_data_uint8, dtype=np.uint8)
                img_rgb = img1d.reshape(responses[0].height, responses[0].width, 3)

                # Изменение размера для ускорения обработки
                frame_resized = cv2.resize(img_rgb,
                                             (Config.YOLO_FRAME_WIDTH,
                                              Config.YOLO_FRAME_HEIGHT))

                # Детекция
                results = self.model(frame_resized,
                                     conf=self.conf_threshold,
                                     device=self.device,
                                     verbose=False)[0]

                with self.lock:
                    self.current_frame = img_rgb # Сохраняем оригинальный кадр
                    self.detection_results = results

            time.sleep(0.033) # ~30 FPS

        except Exception as e:
            print(f"Ошибка детекции: {e}")
            time.sleep(1)

def get_detection_display(self):
    """Получение кадра с bounding boxes"""
    with self.lock:
        if self.current_frame is None or self.detection_results is None:

```



```

        return None

# Создание копии кадра для отрисовки
display_frame = self.current_frame.copy()

# Отрисовка bounding boxes
if hasattr(self.detection_results, 'boxes'):
    boxes = self.detection_results.boxes
    for box in boxes:
        # Координаты (масштабированные)
        x1, y1, x2, y2 = box.xyxy[0].cpu().numpy()
        conf = box.conf[0].cpu().numpy()
        cls = int(box.cls[0].cpu().numpy())

        # Масштабирование координат до оригинального размера
        original_height, original_width = self.current_frame.shape[:2]
        x1 = int(x1 * original_width / Config.YOLO_FRAME_WIDTH)
        y1 = int(y1 * original_height / Config.YOLO_FRAME_HEIGHT)
        x2 = int(x2 * original_width / Config.YOLO_FRAME_WIDTH)
        y2 = int(y2 * original_height / Config.YOLO_FRAME_HEIGHT)

        # Отрисовка прямоугольника
        color = self.colors[cls % len(self.colors)]
        cv2.rectangle(display_frame,
                      (int(x1), int(y1)),
                      (int(x2), int(y2)),
                      color, 2)

        # Текст с классом и уверенностью
        class_name = self.detection_results.names[cls]
        label = f'{class_name} ({conf:.2%})'
        cv2.putText(display_frame, label,
                    (int(x1), int(y1) - 10),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.5, color, 2)

    return display_frame

def get_detected_objects_count(self):
    """Получение количества обнаруженных объектов"""
    with self.lock:
        if self.detection_results and hasattr(self.detection_results, 'boxes'):
            return len(self.detection_results.boxes)
        return 0

# =====
# КЛАССЫ ДАННЫХ
# =====

@dataclass
class ESP32State:
    """Состояние ESP32"""
    motors_armed: bool = False
    stream_active: bool = False
    throttle_value: int = Config.DEFAULT_THROTTLE
    last_command: Optional[str] = None
    last_command_time: Optional[float] = None

@dataclass
class PIDCoeffs:
    """PID коэффициенты"""
    Kp: float
    Ki: float
    Kd: float

@dataclass
class CalibrationData:
    """Данные калибровки"""
    zero_pitch: float = 0.0
    zero_roll: float = 0.0
    max_velocity: float = 8.0
    dead_zone: float = Config.DEAD_ZONE_DEFAULT
    sensitivity: float = Config.SENSITIVITY_DEFAULT
    invert_pitch: bool = False
    invert_roll: bool = True

@dataclass
class SensorData:
    """Данные сенсоров"""

```

```

yaw: float
pitch: float
roll: float
timestamp: float

@dataclass
class ControlData:
    """Контрольные данные"""
    velocity_x: float
    velocity_y: float
    control_pitch: float
    control_roll: float

# =====
# КЛАСС ЛОГГЕРА
# =====

class Logger:
    """Класс для логирования данных с корректным временем"""

    def __init__(self, log_dir: str = Config.LOG_DIR):
        self.log_dir = log_dir
        os.makedirs(self.log_dir, exist_ok=True)

        # Запоминаем время старта программы
        self.program_start_time = time.time()
        self.start_datetime = datetime.now()

        timestamp = self.start_datetime.strftime("%Y%m%d_%H%M%S")
        self.filename = f"angles_log_{timestamp}.txt"
        self.filepath = os.path.join(self.log_dir, self.filename)

        self._write_header()

    def _write_header(self):
        """Запись заголовка в лог-файл"""
        header = ("#" * 80 + "\n"
                  f"# Flight Log Session Started: {self.start_datetime.strftime('%Y-%m-%d %H:%M:%S')}\n"
                  f"# Log File: {self.filename}\n"
                  "#" * 80 + "\n\n"
                  "Absolute Time\tTime From Start(s)\tRaw Yaw\tRaw Pitch\tRaw Roll\t"
                  "Rel Pitch\tRel Roll\tControl Pitch\tControl Roll\tVelocity_X\tVelocity_Y\n")

        with open(self.filepath, 'w', encoding='utf-8') as f:
            f.write(header)

    def log_angles(self, sensor_data: SensorData, control_data: ControlData,
                   rel_pitch: float, rel_roll: float):
        """Логирование углов и контрольных данных с корректным временем"""
        current_time = time.time()
        elapsed_time = current_time - self.program_start_time

        # Форматируем абсолютное время
        abs_time = datetime.fromtimestamp(current_time).strftime("%H:%M:%S.%f")[:-3]

        data_line = (f'{abs_time}\t'
                     f'{elapsed_time:.3f}\t'
                     f'{sensor_data.yaw:.2f}\t'
                     f'{sensor_data.pitch:.2f}\t'
                     f'{sensor_data.roll:.2f}\t'
                     f'{rel_pitch:.2f}\t'
                     f'{rel_roll:.2f}\t'
                     f'{control_data.control_pitch:.2f}\t'
                     f'{control_data.control_roll:.2f}\t'
                     f'{control_data.velocity_x:.2f}\t'
                     f'{control_data.velocity_y:.2f}\t')

        with open(self.filepath, 'a', encoding='utf-8') as f:
            f.write(data_line + "\n")

    def log_calibration(self, calibration: CalibrationData):
        """Логирование калибровочных данных"""
        current_time = datetime.now().strftime("%H:%M:%S")
        elapsed_time = time.time() - self.program_start_time

        log_line = (f'# [{current_time}] [{elapsed_time:.2f}s] CALIBRATION_DATA:\n'
                    f'# zero_pitch={calibration.zero_pitch:.2f}\n'
                    f'# zero_roll={calibration.zero_roll:.2f}\n")

```

```

        f"# sensitivity={calibration.sensitivity:.2f}\n"
        f"# dead_zone={calibration.dead_zone:.2f}\n")

    with open(self.filepath, 'a', encoding='utf-8') as f:
        f.write(log_line)

def get_log_path(self) -> str:
    """Получение пути к лог-файлу"""
    return self.filepath

# =====
# КЛАСС ДЛЯ РАБОТЫ С ESP32
# =====

class ESP32Controller:
    """Класс для управления ESP32"""

    def __init__(self, ip: str = Config.ESP32_IP, port: int = Config.PORT):
        self.ip = ip
        self.port = port
        self.state = ESP32State()
        self.sock = self._create_socket()

    def _create_socket(self) -> socket.socket:
        """Создание UDP сокета"""
        sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
        sock.bind(("", self.port))
        sock.settimeout(Config.SOCKET_TIMEOUT)
        return sock

    def send_command(self, command: str, silent: bool = False) -> bool:
        """Отправка команды на ESP32"""
        try:
            self.sock.sendto(command.encode('utf-8'), (self.ip, self.port))
            self.state.last_command = command
            self.state.last_command_time = time.time()

            self._update_state(command)

            if not silent:
                print(f"\n📡 Sent -> {command}")
            return True

        except Exception as e:
            if not silent:
                print(f"\n❌ Ошибка отправки команды '{command}': {e}")
            return False

    def _update_state(self, command: str):
        """Обновление состояния на основе команды"""
        if command == "ARM":
            self.state.motors_armed = True
        elif command == "DISARM":
            self.state.motors_armed = False
        elif command == "START":
            self.state.stream_active = True
        elif command == "STOP":
            self.state.stream_active = False
        elif command.startswith("SET"):
            parts = command.split()
            if len(parts) >= 5:
                try:
                    self.state.throttle_value = int(parts[4])
                except ValueError:
                    pass

    def set_target_angles(self, yaw: float = 0, pitch: float = 0,
                          roll: float = 0, throttle: int = Config.DEFAULT_THROTTLE) -> bool:
        """Установка целевых углов и газа"""
        if throttle < Config.MIN_THROTTLE or throttle > Config.MAX_THROTTLE:
            print(f"⚠️ Throttle должен быть между {Config.MIN_THROTTLE} и {Config.MAX_THROTTLE}")
            return False

        command = f"SET {yaw:.1f} {pitch:.1f} {roll:.1f} {throttle}"
        return self.send_command(command)

    def receive_data(self) -> Optional[Tuple[float, float, float]]:
        """Получение данных от ESP32"""

```

```

try:
    data, addr = self.sock.recvfrom(1024)
    msg = data.decode().strip()

    if ',' in msg:
        parts = msg.split(',')
        if len(parts) == 3:
            return tuple(float(part) for part in parts)

except socket.timeout:
    pass
except Exception as e:
    print(f"❌ Ошибка получения данных: {e}")

return None

def close(self):
    """Закрытие соединения"""
    try:
        self.sock.close()
    except:
        pass

# =====
# КЛАСС ДЛЯ РАБОТЫ С AirSim
# =====

class AirSimController:
    """Класс для управления дроном в AirSim"""

    def __init__(self):
        self.client = None
        self.connected = False
        self.in_air = False
        self.camera_pitch = 0
        self.camera_yaw = 0

    def connect(self) -> bool:
        """Подключение к AirSim"""
        try:
            print("🔌 Подключение к AirSim...")
            self.client = airsims.MultirotorClient()
            self.client.confirmConnection()
            self.client.enableApiControl(True)
            self.client.armDisarm(False)
            self.connected = True
            print("✅ AirSim подключен")
            return True

        except Exception as e:
            print(f"❌ Ошибка подключения к AirSim: {e}")
            self.connected = False
            return False

    def get_drone_state(self) -> Optional[str]:
        """Получение состояния дрона"""
        if not self.connected:
            return None

        try:
            state = self.client.getMultirotorState()
            return '➡ В воздухе' if state.landed_state == 0 else '🛑 На земле'
        except:
            return '❓ Неизвестно'

    def takeoff(self, target_altitude: float = Config.TARGET_ALTITUDE) -> bool:
        """Взлет дрона"""
        if not self.connected:
            print("❌ AirSim не подключен!")
            return False

        try:
            print("🛫 Взлетаем...")
            self.client.armDisarm(True)
            self.client.takeoffAsync().join()
            time.sleep(2)
            self.client.moveToZAsync(target_altitude, 2).join()

```

```

        self.in_air = True
        print(f"□ Взлет выполнен! Высота: {-target_altitude} метров")
        return True

    except Exception as e:
        print(f"□ Ошибка при взлете: {e}")
        return False

def land(self) -> bool:
    """Посадка дрона"""
    if not self.connected:
        print(" AirSim не подключен!")
        return False

    try:
        print("Начинаю посадку...")
        self.client.landAsync().join()
        time.sleep(2)
        self.client.armDisarm(False)
        self.in_air = False
        print("Посадка выполнена успешно!")
        return True

    except Exception as e:
        print(f"□ Ошибка при посадке: {e}")
        return False

def move_by_velocity(self, velocity_x: float, velocity_y: float,
                    yaw: float = 0) -> bool:
    """Движение дрона с заданной скоростью"""
    if not self.connected or not self.in_air:
        return False

    try:
        self.client.moveByVelocityZAsync(
            velocity_x,
            velocity_y,
            Config.TARGET_ALTITUDE,
            0.1,
            airsims.DrivetrainType.MaxDegreeOfFreedom,
            airsims.YawMode(False, yaw)
        )
        return True
    except:
        return False

def set_camera_orientation(self, pitch: float, yaw: float) -> bool:
    """Установка ориентации камеры"""
    if not self.connected:
        return False

    try:
        self.client.simSetCameraPose(
            "0",
            airsims.Pose(
                airsims.Vector3r(0, 0, 0),
                airsims.to_quaternion(
                    math.radians(pitch),
                    math.radians(0),
                    math.radians(yaw)
                )
            )
        )
        return True
    except:
        return False

# =====
# КЛАСС ДЛЯ ОБРАБОТКИ ДАННЫХ С ДАТЧИКОВ
# =====

class SensorProcessor:
    """Класс для обработки данных с датчиков"""

    def __init__(self, calibration: CalibrationData):
        self.calibration = calibration
        self.prev_yaw = 0.0
        self.prev_pitch = 0.0

```

```

self.prev_roll = 0.0

def process_sensor_data(self, raw_yaw: float, raw_pitch: float,
                        raw_roll: float) -> SensorData:
    """Обработка сырых данных с датчиков"""
    # Сглаживание данных
    yaw = 0.8 * self.prev_yaw + 0.2 * raw_yaw
    pitch = 0.8 * self.prev_pitch + 0.2 * raw_pitch
    roll = 0.8 * self.prev_roll + 0.2 * raw_roll

    self.prev_yaw, self.prev_pitch, self.prev_roll = yaw, pitch, roll

    return SensorData(
        yaw=yaw,
        pitch=pitch,
        roll=roll,
        timestamp=time.time()
    )

def calculate_relative_angles(self, sensor_data: SensorData) -> Tuple[float, float]:
    """Расчет относительных углов с учетом калибровки"""
    rel_pitch = sensor_data.pitch - self.calibration.zero_pitch
    rel_roll = sensor_data.roll - self.calibration.zero_roll

    # Применение мертвой зоны
    if abs(rel_pitch) < self.calibration.dead_zone:
        rel_pitch = 0
    if abs(rel_roll) < self.calibration.dead_zone:
        rel_roll = 0

    return rel_pitch, rel_roll

def calculate_control_data(self, rel_pitch: float, rel_roll: float,
                          drone_yaw: float) -> ControlData:
    """Расчет контрольных данных для управления дроном"""
    # Инвертируем оси при необходимости
    if self.calibration.invert_pitch:
        rel_pitch = -rel_pitch
    if self.calibration.invert_roll:
        rel_roll = -rel_roll

    # Применяем чувствительность
    control_pitch = rel_pitch * self.calibration.sensitivity
    control_roll = rel_roll * self.calibration.sensitivity

    # Ограничиваем углы управления
    control_pitch = max(-Config.MAX_ANGLE_LIMIT,
                       min(Config.MAX_ANGLE_LIMIT, control_pitch))
    control_roll = max(-Config.MAX_ANGLE_LIMIT,
                      min(Config.MAX_ANGLE_LIMIT, control_roll))

    # Преобразование в локальные скорости
    local_forward = control_pitch * self.calibration.max_velocity / Config.MAX_ANGLE_LIMIT
    local_right = control_roll * self.calibration.max_velocity / Config.MAX_ANGLE_LIMIT

    # Минимальная скорость для реакции
    if abs(local_forward) < Config.MIN_SPEED_THRESHOLD and \
        abs(local_right) < Config.MIN_SPEED_THRESHOLD:
        return ControlData(0, 0, control_pitch, control_roll)

    # Преобразование в глобальные скорости
    yaw_rad = math.radians(drone_yaw)
    global_x = local_forward * math.cos(yaw_rad) - local_right * math.sin(yaw_rad)
    global_y = local_forward * math.sin(yaw_rad) + local_right * math.cos(yaw_rad)

    # Корректировка минимальной скорости
    speed = math.sqrt(global_x**2 + global_y**2)
    if 0 < speed < Config.MIN_SPEED_THRESHOLD:
        scale = Config.MIN_SPEED_THRESHOLD / speed
        global_x *= scale
        global_y *= scale

    return ControlData(global_x, global_y, control_pitch, control_roll)

# =====
# КЛАСС ДЛЯ КАЛИБРОВКИ
# =====

```

```

class CalibrationManager:
    """Класс для управления калибровкой"""

    def __init__(self, esp32_controller: ESP32Controller):
        self.esp32 = esp32_controller
        self.calibration = CalibrationData()
        self.calibration_done = False

    def prepare_calibration(self) -> bool:
        """Подготовка к калибровке"""
        print("\n" + "="*60)
        print(" ПОДГОТОВКА К КАЛИБРОВКЕ")
        print("="*60)

        print("🔌 Проверка подключения к ESP32...")

        # Попробуем получить данные от ESP32
        data_received = False
        for i in range(10):
            data = self.esp32.receive_data()
            if data:
                data_received = True
                yaw, pitch, roll = data
                print(f"✅ Данные получены: Pitch={pitch:.1f}°, Roll={roll:.1f}°, Yaw={yaw:.1f}°")
                break
            time.sleep(0.5)

        if i == 4:
            print("Включаю стрим данных...")
            self.esp32.send_command("START", silent=True)

        if not data_received:
            print("❌ Не удалось получить данные от ESP32")
            return False

        print("\n✅ Подготовка к калибровке завершена!")
        return True

    def perform_calibration(self) -> bool:
        """Выполнение калибровки"""
        print("\n" + "="*60)
        print(" НАЧАЛО КАЛИБРОВКИ")
        print("="*60)

        # Обратный отсчет
        print("\n✅ Начинаю через:")
        for i in range(5, 0, -1):
            print(f" {i}...")
            time.sleep(1)

        print(" НАЧАЛО!")

        # Сбор данных
        pitch_samples = []
        roll_samples = []

        print("\n Сбор данных калибровки...")

        start_time = time.time()
        sample_count = 0
        progress_bar_length = 40

        while time.time() - start_time < 5:
            elapsed = time.time() - start_time
            progress = int((elapsed / 5) * progress_bar_length)

            data = self.esp32.receive_data()
            if data:
                yaw, pitch, roll = data
                pitch_samples.append(pitch)
                roll_samples.append(roll)
                sample_count += 1

            bar = █ * progress + ░ * (progress_bar_length - progress)
            print(f"\r [{bar}] {elapsed:.1f}/5.0с | Образцов: {sample_count}",
                  end="", flush=True)
            time.sleep(0.05)

```

```

print()

if not pitch_samples or not roll_samples:
    print("❑ Ошибка калибровки: не удалось собрать данные")
    return False

# Вычисляем средние значения
self.calibration.zero_pitch = sum(pitch_samples) / len(pitch_samples)
self.calibration.zero_roll = sum(roll_samples) / len(roll_samples)

# Вычисляем стандартное отклонение
if len(pitch_samples) > 1:
    pitch_std = math.sqrt(sum((x - self.calibration.zero_pitch)**2
                              for x in pitch_samples) / (len(pitch_samples) - 1))
    roll_std = math.sqrt(sum((x - self.calibration.zero_roll)**2
                              for x in roll_samples) / (len(roll_samples) - 1))

    print(f"\n Статистика:")
    print(f"   Pitch std: {pitch_std:.2f}° {'❑' if pitch_std < 2 else '❐'}")
    print(f"   Roll std: {roll_std:.2f}° {'❑' if roll_std < 2 else '❐'}")

print(f"\n❑ КАЛИБРОВКА ЗАВЕРШЕНА:")
print(f"   🌀 Zero Pitch: {self.calibration.zero_pitch:.1f}°")
print(f"   🌀 Zero Roll: {self.calibration.zero_roll:.1f}°")
print(f"   📊 Образцов собрано: {sample_count}")

self.calibration_done = True
return True

# =====
# КЛАСС ДЛЯ УПРАВЛЕНИЯ МЕНЮ
# =====

class MenuManager:
    """Класс для управления меню и вводом с клавиатуры"""

    def __init__(self):
        self.current_mode = 1 # 1 - главное меню, 2 - меню камеры
        self.program_running = True
        self.telemetry_enabled = False
        self.print_telemetry_flag = False
        self.menu_shown = False
        self.camera_menu_shown = False
        self.show_menu_after_command = False
        self.telemetry_line_shown = False
        self.object_detection_flag = False

        if not WINDOWS:
            # Сохраняем настройки терминала для Linux
            self.old_settings = termios.tcgetattr(sys.stdin)
            tty.setcbreak(sys.stdin.fileno())

    def get_key(self) -> Optional[str]:
        """Считывание клавиши без ENTER"""
        try:
            if WINDOWS:
                if msvcrt.kbhit():
                    key = msvcrt.getch()
                    if key == b'\xe0':
                        key = msvcrt.getch()
                        if key == b'H': return 'up'
                        if key == b'P': return 'down'
                        if key == b'K': return 'left'
                        if key == b'M': return 'right'
                    else:
                        return key.decode('utf-8').lower()
            else:
                if select.select([sys.stdin], [], [], 0)[0]:
                    key = sys.stdin.read(1)
                    if key == '\x1b':
                        if select.select([sys.stdin], [], [], 0.1)[0]:
                            next1, next2 = sys.stdin.read(2)
                            if next1 == '[':
                                if next2 == 'A': return 'up'
                                if next2 == 'B': return 'down'
                                if next2 == 'D': return 'left'
                                if next2 == 'C': return 'right'

```



```

        else:
            return key.lower()
    except:
        pass
    return None

def clear_telemetry_line(self):
    """Очищает строку телеметрии"""
    if self.telemetry_line_shown:
        print('\r' + ' ' * 120 + '\r', end="", flush=True)
        self.telemetry_line_shown = False

def cleanup(self):
    """Очистка ресурсов"""
    if not WINDOWS:
        termios.tcsetattr(sys.stdin, termios.TCSADRAIN, self.old_settings)

# =====
# ГЛАВНЫЙ КЛАСС ПРИЛОЖЕНИЯ
# =====

class DroneControlSystem:
    """Главный класс системы управления дроном"""

    def __init__(self):
        self.logger = Logger()
        self.esp32 = ESP32Controller()
        self.airsim = AirSimController()
        self.menu = MenuManager()

        # PID коэффициенты
        self.pid_coeffs = {
            'pitch': PIDCoeffs(Kp=4.0, Ki=0.02, Kd=0.5),
            'roll': PIDCoeffs(Kp=4.0, Ki=0.02, Kd=0.5),
            'yaw': PIDCoeffs(Kp=2.0, Ki=0.005, Kd=0.3)
        }

        # Калибровка
        self.calibration_manager = CalibrationManager(self.esp32)
        self.calibration = self.calibration_manager.calibration

        # Обработка данных
        self.sensor_processor = SensorProcessor(self.calibration)

        # YOLOv8 детектор
        self.yolo_detector = YOLOv8Detector()
        self.object_detection_active = False
        self.detection_display_thread = None

        # Время
        self.start_time = time.time()
        self.last_command_time = time.time()

        # Флаг для обновления телеметрии
        self.last_telemetry_update = 0
        self.telemetry_update_interval = 0.2 # Обновлять телеметрию каждые 200 мс

    def show_status(self):
        """Показывает текущий статус системы"""
        print("\n" + "="*60)
        print(" ТЕКУЩИЙ СТАТУС СИСТЕМЫ")
        print("="*60)

        print(f"ESP32:")
        print(f" Моторы: {'ARM' if self.esp32.state.motors_armed else 'DISARM'}")
        print(f" Стрим: {'ВКЛ' if self.esp32.state.stream_active else 'ВЫКЛ'}")
        print(f" Throttle: {self.esp32.state.throttle_value} мкс")

        print("-" * 60)
        print(f"AirSim:")
        print(f" Подключение: {'' if self.airsim.connected else ''}")
        if self.airsim.connected:
            drone_state = self.airsim.get_drone_state()
            if drone_state:
                print(f" Дрон: {drone_state}")
            else:
                print(f" Дрон: Неизвестно")

```

```

print("-" * 60)
print(f"Калибровка:")
status = ' ☐ ВЫПОЛНЕНА' if self.calibration_manager.calibration_done else ' ☐ НЕ ВЫПОЛНЕНА'
print(f" Статус: {status}")
if self.calibration_manager.calibration_done:
    print(f" Zero Pitch: {self.calibration.zero_pitch:.1f}°")
    print(f" Zero Roll: {self.calibration.zero_roll:.1f}°")

print("-" * 60)
print(f"YOLOv8 Детекция:")
status = ' АКТИВНА' if self.object_detection_active else ' НЕАКТИВНА'
print(f" Статус: {status}")
if self.object_detection_active:
    objects_count = self.yolo_detector.get_detected_objects_count()
    print(f" Обнаружено объектов: {objects_count}")

print("=*60)

def show_main_menu(self):
    """Показывает главное меню"""
    if not self.menu.menu_shown:
        self.menu.clear_telemetry_line()
        print("\n" + "="*70)
        print(" ГЛАВНОЕ МЕНЮ УПРАВЛЕНИЯ:")
        print("=*70)

    menu_items = [
        "[1] Включить телеметрию",
        "[2] Настройка камеры",
        "[5] YOLOv8 Детекция объектов",
        "-" * 70,
        "[3] Управление дроном (установка углов)",
        "    Задает target angles на ESP32",
        "-" * 70,
        "[4] Показать PID коэффициенты",
        "-" * 70,
        "[a] ARM (включить моторы)",
        "[z] DISARM (выключить моторы)",
        "-" * 70,
        "[s] START стрим",
        "[x] STOP стрим",
        "-" * 70,
        "[t] Takeoff (AirSim)",
        "[l] Land (AirSim)",
        "-" * 70,
        "[i] Показать текущий статус",
        "[ ] Пробел - выключить телеметрию",
        "-" * 70,
        "[r] Переподключиться к AirSim",
        "[q] Посадка и выход"
    ]

    for item in menu_items:
        print(f" {item}")

    print("=*70)

    if self.calibration_manager.calibration_done:
        print(" ☐ Калибровка выполнена. Система готова к работе.")
    else:
        print(" ☐ Калибровка не выполнена. Сначала выполните калибровку!")

    print("=*70)
    print("⏏ Нажмите клавишу (цифра/буква)")

    self.menu.menu_shown = True

def drone_control_menu(self):
    """Меню управления дроном"""
    self.menu.print_telemetry_flag = False
    self.menu.clear_telemetry_line()

    print("\n" + "="*60)
    print("УПРАВЛЕНИЕ ДРОНОМ (Target Angles)")
    print("=*60)

    # Ввод значений
    try:

```

```

yaw = self._get_input_float("Введите Yaw (рыскание) в градусах [0]: ", 0.0)
pitch = self._get_input_float("Введите Pitch (тангаж) в градусах [0]: ", 0.0)
roll = self._get_input_float("Введите Roll (крен) в градусах [0]: ", 0.0)

throttle = self._get_input_int(
    f"Введите Throttle ({Config.MIN_THROTTLE}-{Config.MAX_THROTTLE}) "
    f"[{self.esp32.state.throttle_value}]: ",
    self.esp32.state.throttle_value,
    Config.MIN_THROTTLE,
    Config.MAX_THROTTLE
)

# Проверка диапазонов
if abs(pitch) > 30:
    print(f"Pitch ({pitch}°) выходит за рекомендуемый диапазон ±30°")
if abs(roll) > 30:
    print(f"Roll ({roll}°) выходит за рекомендуемый диапазон ±30°")

print(f"\n Будет отправлена команда: SET {yaw:.1f} {pitch:.1f} {roll:.1f} {throttle}")
print(f" ESP32 будет пытаться удерживать эти углы")

confirm = input("Подтвердить отправку? (y/n): ").strip().lower()
if confirm == 'y':
    if self.esp32.set_target_angles(yaw, pitch, roll, throttle):
        print(f"Команда отправлена: Yaw={yaw:.1f}°, "
              f"Pitch={pitch:.1f}°, Roll={roll:.1f}°, "
              f"Throttle={throttle} мкс")
    except ValueError as e:
        print(f"Ошибка ввода: {e}")
    except Exception as e:
        print(f"Ошибка: {e}")

def show_pid_info(self):
    """Показывает информацию о PID коэффициентах"""
    self.menu.print_telemetry_flag = False
    self.menu.clear_telemetry_line()

    print("\n" + "="*60)
    print(" PID КОЭФФИЦИЕНТЫ (ТОЛЬКО ДЛЯ ПРОСМОТРА)")
    print("="*60)
    print("Эти коэффициенты запрограммированы в ESP32")
    print("и регулируют стабилизацию дрона по углам.")
    print("-" * 60)
    print(" Текущие значения:")
    print(f" Pitch (тангаж):")
    print(f" Kp = {self.pid_coeffs['pitch'].Kp} - пропорциональный коэффициент")
    print(f" Ki = {self.pid_coeffs['pitch'].Ki} - интегральный коэффициент")
    print(f" Kd = {self.pid_coeffs['pitch'].Kd} - дифференциальный коэффициент")
    print()
    print(f" Roll (крен):")
    print(f" Kp = {self.pid_coeffs['roll'].Kp}")
    print(f" Ki = {self.pid_coeffs['roll'].Ki}")
    print(f" Kd = {self.pid_coeffs['roll'].Kd}")
    print()
    print(f" Yaw (рыскание):")
    print(f" Kp = {self.pid_coeffs['yaw'].Kp}")
    print(f" Ki = {self.pid_coeffs['yaw'].Ki}")
    print(f" Kd = {self.pid_coeffs['yaw'].Kd}")
    print("-" * 60)
    print("i Изменение PID коэффициентов требует перепрошивки ESP32")
    print("="*60)

    input("\nНажмите Enter для возврата в меню...")

def _get_input_float(self, prompt: str, default: float) -> float:
    """Получение float ввода с обработкой по умолчанию"""
    value = input(prompt).strip()
    return float(value) if value else default

def _get_input_int(self, prompt: str, default: int, min_val: int, max_val: int) -> int:
    """Получение int ввода с проверкой диапазона"""
    while True:
        value = input(prompt).strip()
        if not value:
            return default
        try:

```

```

        int_val = int(value)
        if min_val <= int_val <= max_val:
            return int_val
        else:
            print(f"❑ Значение должно быть от {min_val} до {max_val}")
    except ValueError:
        print("❑ Введите целое число")

def menu_thread_func(self):
    """Поток обработки меню"""
    while self.menu.program_running:
        key = self.menu.get_key()

        if key:
            self.menu.clear_telemetry_line()

            if self.menu.current_mode == 1:
                self._handle_main_menu_key(key)
            elif self.menu.current_mode == 2:
                self._handle_camera_menu_key(key)

        time.sleep(0.1)

def _handle_main_menu_key(self, key: str):
    """Обработка клавиш главного меню"""
    handlers = {
        '1': self._handle_telemetry_enable,
        '2': self._handle_camera_mode,
        '3': self._handle_drone_control,
        '4': self._handle_pid_info,
        '5': self._handle_object_detection,
        'a': self._handle_arm,
        'z': self._handle_disarm,
        's': self._handle_start_stream,
        'x': self._handle_stop_stream,
        't': self._handle_takeoff,
        'l': self._handle_land,
        'i': self._handle_status,
        'v': self._handle_telemetry_disable,
        'r': self._handle_reconnect,
        'q': self._handle_quit
    }

    handler = handlers.get(key)
    if handler:
        handler()
    else:
        print(f"\n❑ Неизвестная команда: {key}")
        self.menu.show_menu_after_command = True

def _handle_camera_menu_key(self, key: str):
    """Обработка клавиш меню камеры"""
    step = 5
    changed = False

    if key == 'up':
        self.airsim.camera_pitch += step
        changed = True
    elif key == 'down':
        self.airsim.camera_pitch -= step
        changed = True
    elif key == 'left':
        self.airsim.camera_yaw -= step
        changed = True
    elif key == 'right':
        self.airsim.camera_yaw += step
        changed = True
    elif key == 'b':
        self.menu.current_mode = 1
        self.menu.menu_shown = False
        self.menu.camera_menu_shown = False
        self.menu.print_telemetry_flag = self.menu.telemetry_enabled
        print("\n↩ Возврат в главное меню")
        return

    if changed:
        if self.airsim.set_camera_orientation(self.airsim.camera_pitch,
                                             self.airsim.camera_yaw):

```

```

        print(f"\r 📷 Камера: Pitch={self.airsim.camera_pitch:3}°, "
              f"Yaw={self.airsim.camera_yaw:3}°", end="", flush=True)

# =====
# ОБРАБОТЧИКИ КОМАНД
# =====

def _handle_telemetry_enable(self):
    if not self.calibration_manager.calibration_done:
        print("\n❑ Сначала выполните калибровку!")
        self.menu.show_menu_after_command = True
        return

    self.menu.print_telemetry_flag = True
    self.menu.telemetry_enabled = True
    print("\nТелеметрия ВКЛЮЧЕНА (нажмите пробел для отключения)")
    self.menu.show_menu_after_command = True

def _handle_camera_mode(self):
    self.menu.current_mode = 2
    self.menu.menu_shown = False
    self.menu.camera_menu_shown = False
    self.menu.clear_telemetry_line()
    print("\nРЕЖИМ НАСТРОЙКИ КАМЕРЫ")
    print(" ↑/↓ — наклон камеры (вверх/вниз)")
    print(" ←/→ — поворот камеры (влево/вправо)")
    print(" [b] — возврат в меню")
    print(f"\r 📷 Камера: Pitch={self.airsim.camera_pitch:3}°, "
          f"Yaw={self.airsim.camera_yaw:3}°", end="", flush=True)

def _handle_drone_control(self):
    if not self.calibration_manager.calibration_done:
        print("\n❑ Сначала выполните калибровку!")
        self.menu.show_menu_after_command = True
        return

    if not self.esp32.state.stream_active:
        print("\n❑ Стрим данных не активен!")
        print(" Включите стрим командой [s]")
        self.menu.show_menu_after_command = True
        return

    self.drone_control_menu()
    self.menu.show_menu_after_command = True

def _handle_pid_info(self):
    self.show_pid_info()
    self.menu.show_menu_after_command = True

def _handle_object_detection(self):
    """Обработка запуска/остановки детекции объектов"""
    if not self.airsim.connected:
        print("\n❑ Сначала подключитесь к AirSim!")
        self.menu.show_menu_after_command = True
        return

    if not self.object_detection_active:
        # ЗАПУСК ДЕТЕКЦИИ
        print("\n🚀 ЗАПУСК YOLOv8 ДЕТЕКЦИИ")
        print("-" * 40)
        print("• Нажмите [5] повторно для остановки")
        print("• Нажмите ESC в окне детекции для выхода")
        print("• Обнаруженные объекты отображаются в bounding boxes")
        print("-" * 40)

        # Подключаем детектор к AirSim
        self.yolo_detector.connect_to_airsim(self.airsim.client)
        self.object_detection_active = True

        # Запускаем детекцию
        self.yolo_detector.start_detection()

        # Запускаем отображение в отдельном потоке
        self.detection_display_thread = Thread(target=self._display_detection,
                                                daemon=True)
        self.detection_display_thread.start()

```

```

        print("❑ Детекция запущена. Окно OpenCV должно открыться.")

    else:
        # ОСТАНОВКА ДЕТЕКЦИИ
        print("\n● Останавливаю детекцию...")
        self.object_detection_active = False
        self.yolo_detector.stop_detection()
        cv2.destroyAllWindows()
        print("❑ Детекция остановлена")

    self.menu.show_menu_after_command = True

def _display_detection(self):
    """Отображение детекции в отдельном окне"""
    window_name = 'YOLOv8 Object Detection - AirSim'
    cv2.namedWindow(window_name, cv2.WINDOW_NORMAL)

    last_objects_count = 0
    last_update_time = time.time()

    while self.object_detection_active and self.yolo_detector.detection_active:
        try:
            frame = self.yolo_detector.get_detection_display()
            if frame is not None:
                # Добавляем информацию о количестве объектов
                objects_count = self.yolo_detector.get_detected_objects_count()

                # Обновляем счетчик в консоли каждые 2 секунды
                current_time = time.time()
                if objects_count != last_objects_count or current_time - last_update_time > 2:
                    print(f"r Обнаружено объектов: {objects_count}", end="", flush=True)
                    last_objects_count = objects_count
                    last_update_time = current_time

                # Добавляем текст на кадр
                info_text = f'Objects: {objects_count}'
                cv2.putText(frame, info_text,
                           (10, 30), cv2.FONT_HERSHEY_SIMPLEX,
                           1, (0, 255, 0), 2)

                # Показываем кадр
                cv2.imshow(window_name, frame)

                # Выход по ESC или кнопке 5
                key = cv2.waitKey(1) & 0xFF
                if key == 27: # ESC
                    self.object_detection_active = False
                    break

        except Exception as e:
            print(f"Ошибка отображения: {e}")
            break

    cv2.destroyAllWindows()

def _handle_arm(self):
    if not self.calibration_manager.calibration_done:
        print("\n❑ Сначала выполните калибровку!")
        self.menu.show_menu_after_command = True
        return

    if not self.esp32.state.stream_active:
        print("\n❑ Стрим данных не активен!")
        print("Включите стрим командой [s]")
        self.menu.show_menu_after_command = True
        return

    print("\n❑ АРМИРОВАНИЕ МОТОРОВ")
    confirm = input("Продолжить? (y/n): ").strip().lower()
    if confirm == 'y':
        if self.esp32.send_command("ARM"):
            print("❑ Команда ARM отправлена на ESP32")
        else:
            print("❑ ARM отменен")
        self.menu.show_menu_after_command = True

def _handle_disarm(self):
    print("\n❑ DISARM МОТОРОВ")

```

```

confirm = input("Продолжить? (y/n): ").strip().lower()
if confirm == 'y':
    if self.esp32.send_command("DISARM"):
        print(" Команда DISARM отправлена на ESP32")
    else:
        print("❑ DISARM отменен")
    self.menu.show_menu_after_command = True

def _handle_start_stream(self):
    if self.esp32.send_command("START"):
        print("❑ Стрим данных START отправлен")
    self.menu.show_menu_after_command = True

def _handle_stop_stream(self):
    if self.esp32.send_command("STOP"):
        print(" Стрим данных STOP отправлен")
    self.menu.show_menu_after_command = True

def _handle_takeoff(self):
    if not self.airsim.connected:
        print(" AirSim не подключен! Используйте [r]")
        self.menu.show_menu_after_command = True
        return

    if not self.esp32.state.motors_armed:
        print(" Сначала взведите моторы [a]!")
        self.menu.show_menu_after_command = True
        return

    print("\n ВЗЛЕТ ДРОНА")
    confirm = input("Выполнить взлет? (y/n): ").strip().lower()
    if confirm == 'y':
        if self.airsim.takeoff():
            print(" Взлет выполнен успешно!")
        else:
            print(" Ошибка при взлете")
    else:
        print(" Взлет отменен")
    self.menu.show_menu_after_command = True

def _handle_land(self):
    if not self.airsim.connected:
        print(" AirSim не подключен!")
        self.menu.show_menu_after_command = True
        return

    if not self.airsim.in_air:
        print("❑ Дрон уже на земле!")
        self.menu.show_menu_after_command = True
        return

    print("\n Начинаю посадку...")
    if self.airsim.land():
        print("❑ Посадка выполнена успешно!")
    else:
        print("❑ Ошибка при посадке")
    self.menu.show_menu_after_command = True

def _handle_status(self):
    self.show_status()
    self.menu.show_menu_after_command = True

def _handle_telemetry_disable(self):
    if self.menu.telemetry_enabled:
        self.menu.telemetry_enabled = False
        self.menu.print_telemetry_flag = False
        self.menu.clear_telemetry_line()
        print("\n Телеметрия ВЫКЛЮЧЕНА")
        self.menu.show_menu_after_command = True

def _handle_reconnect(self):
    print("\n Переподключение к AirSim...")
    if self.airsim.connect():
        print("❑ AirSim переподключен")
    else:
        print(" Не удалось переподключиться к AirSim")
    self.menu.show_menu_after_command = True

```

```

def _handle_quit(self):
    print("\n Запуск посадки и выход...")
    self.menu.program_running = False

def run(self):
    """Запуск основной программы"""
    print("\n" + "="*60)
    print(" СИСТЕМА УПРАВЛЕНИЯ ДРОНОМ С YOLOv8")
    print("="*60)
    print(f" Логирование начато: {os.path.basename(self.logger.get_log_path())}")
    print(f" YOLOv8 модель: {Config.YOLO_MODEL_PATH}")

    # Запуск калибровки
    if not self._run_calibration():
        print("\n Калибровка не выполнена. Программа завершена.")
        return

    # Подключение к AirSim
    print("\n Подключение к AirSim...")
    self.airsim.connect()

    # Запуск потока меню
    menu_thread = threading.Thread(target=self.menu_thread_func, daemon=True)
    menu_thread.start()

    # Основной цикл
    self._main_loop()

def _run_calibration(self) -> bool:
    """Выполнение калибровки"""
    print("\n" + "="*60)
    print(" ОБЯЗАТЕЛЬНАЯ КАЛИБРОВКА ПРИ ЗАПУСКЕ")
    print("="*60)

    print("\nШАГ 1: ПОДГОТОВКА К КАЛИБРОВКЕ")
    print("-" * 40)
    input("\n⏎ Нажмите Enter, когда будете готовы начать подготовку...")

    if not self.calibration_manager.prepare_calibration():
        return False

    print("\n ШАГ 2: ВЫПОЛНЕНИЕ КАЛИБРОВКИ")
    print("-" * 40)
    input("\n⏎ Нажмите Enter, когда будете готовы начать калибровку...")

    if not self.calibration_manager.perform_calibration():
        print("\n❌ КАЛИБРОВКА НЕ УДАЛАСЬ!")
        return False

    print("\n❌ КАЛИБРОВКА УСПЕШНО ЗАВЕРШЕНА!")
    return True

def _main_loop(self):
    """Основной цикл обработки данных"""
    try:
        while self.menu.program_running:
            current_time = time.time()

            if self.menu.show_menu_after_command:
                self.menu.menu_shown = False
                self.menu.show_menu_after_command = False

            # Показ меню
            if self.menu.current_mode == 1 and not self.menu.menu_shown:
                self.show_main_menu()

            # Получение данных от ESP32
            data = self.esp32.receive_data()
            if data:
                self._process_sensor_data(data, current_time)

            # Обновление телеметрии (с ограничением частоты)
            if (self.menu.print_telemetry_flag and self.menu.telemetry_enabled and
                self.menu.current_mode == 1 and
                current_time - self.last_telemetry_update > self.telemetry_update_interval):

                # Показываем телеметрию только если есть данные
                if hasattr(self, 'last_sensor_data'):

```



```

        self._print_telemetry_line()
        self.last_telemetry_update = current_time

    # Keep-alive для AirSim
    if (current_time - self.last_command_time > Config.KEEP_ALIVE_INTERVAL and
        self.airsim.connected and self.airsim.in_air):
        self.airsim.move_by_velocity(0, 0, 0)
        self.last_command_time = current_time

    time.sleep(Config.MAIN_LOOP_SLEEP)

except KeyboardInterrupt:
    print("\n Остановка по Ctrl+C...")
except Exception as e:
    print(f"\n❌ Критическая ошибка: {e}")
finally:
    self.shutdown()

def _process_sensor_data(self, raw_data: Tuple[float, float, float], current_time: float):
    """Обработка данных с датчиков"""
    raw_yaw, raw_pitch, raw_roll = raw_data

    # Проверка на NaN
    if any(math.isnan(x) for x in [raw_yaw, raw_pitch, raw_roll]):
        return

    # Обработка данных
    sensor_data = self.sensor_processor.process_sensor_data(raw_yaw, raw_pitch, raw_roll)
    rel_pitch, rel_roll = self.sensor_processor.calculate_relative_angles(sensor_data)
    control_data = self.sensor_processor.calculate_control_data(
        rel_pitch, rel_roll, sensor_data.yaw
    )

    # Сохраняем последние данные для телеметрии
    self.last_sensor_data = sensor_data
    self.last_control_data = control_data
    self.last_rel_pitch = rel_pitch
    self.last_rel_roll = rel_roll

    # Логирование с корректным временем
    self.logger.log_angles(sensor_data, control_data, rel_pitch, rel_roll)

    # Управление дроном в AirSim
    if self.airsim.connected and self.airsim.in_air:
        self.airsim.move_by_velocity(
            control_data.velocity_x,
            control_data.velocity_y,
            sensor_data.yaw
        )
        self.last_command_time = current_time

def _print_telemetry_line(self):
    """Вывод одной строки телеметрии"""
    if not hasattr(self, 'last_sensor_data'):
        return

    # Очищаем предыдущую строку
    if self.menu.telemetry_line_shown:
        print("\r" + ' ' * 120 + "\r", end="", flush=True)

    # Получаем количество объектов, если детекция активна
    objects_info = ""
    if self.object_detection_active:
        objects_count = self.yolo_detector.get_detected_objects_count()
        objects_info = f" | Objects: {objects_count}"

    # Выводим новую строку
    print(f"\r Raw: P={self.last_sensor_data.pitch:6.1f}° R={self.last_sensor_data.roll:6.1f}° "
          f"Y={self.last_sensor_data.yaw:6.1f}° | Rel: P={self.last_rel_pitch:6.1f}° "
          f"R={self.last_rel_roll:6.1f}° | V: X={self.last_control_data.velocity_x:5.2f} "
          f"Y={self.last_control_data.velocity_y:5.2f} m/s{objects_info}", end="", flush=True)

    self.menu.telemetry_line_shown = True

def shutdown(self):
    """Безопасное завершение работы"""
    print("\n🛑 Безопасное завершение...")

```

```

# Остановка детекции, если активна
if self.object_detection_active:
    print(" Остановка YOLOv8 детекции...")
    self.object_detection_active = False
    self.yolo_detector.stop_detection()
    cv2.destroyAllWindows()

# Посадка дрона
if self.airsim.connected and self.airsim.in_air:
    self.airsim.land()

# Отключение ESP32
try:
    print(" Отключение ESP32...")
    self.esp32.send_command("STOP", silent=True)
    time.sleep(0.1)
    self.esp32.send_command("DISARM", silent=True)
except:
    pass

# Закрытие соединений
self.esp32.close()
self.menu.cleanup()

print(f"\n Лог сохранен в: {self.logger.get_log_path()}")
print("\n □ Программа завершена")

# =====
# ЗАПУСК ПРОГРАММЫ
# =====

if __name__ == "__main__":
    app = DroneControlSystem()
    app.run()

```